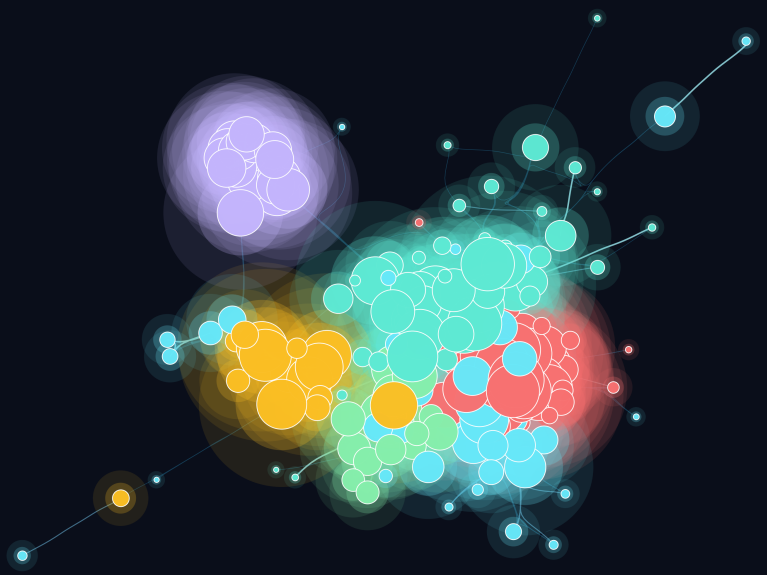
Accessibility Benchmark

From low-vision accessibility
to universal agentic interfaces

by Adrien SALES



github.com/adriens/aquavena linkedin.com/in/adrien-sales

June 20, 2026

Accessibility Benchmark

From low-vision accessibility
to universal agentic interfaces

Comparing `adriens.github.io/aquavena` vs `aquavena.nc`
WebMCP · MCP server · WCAG 2.1 AA · agentic accessibility · New Caledonia

Adrien SALES

x.com/rastadidi linkedin.com/in/adrien-sales
github.com/adriens/aquavena

78f88624bb1db5d5f9b6bc06991299a1f756ff00

Abstract

*This report automatically compares the web accessibility of **adriens.github.io/aquavena** — designed for visually impaired users — against the source website **aquavena.nc**, a dietetic meal delivery service in New Caledonia. Standard CLI tools (pa11y-ci, Lighthouse, axe-core, webhint) are used as a common reference. Results show a **100/100 Lighthouse score and 0 WCAG 2.1 AA errors** for our site, versus 91/100 and 31 errors for the official website. All audit data is persisted in a DuckDB database for longitudinal tracking.*

civitech | tech for good | inclusive design | digital inclusion | accessibility | WCAG | visually impaired | social impact | assistive technology | elderly care | aging population | senior accessibility | human-centered design | design thinking | New Caledonia

Table of contents

1 Inception **6**

2 Executive Dashboard — Delivery KPIs **6**
2.1 EVM — Planned Value vs Earned Value . . . 6
2.2 Strategic impact — multi-axis radar . . . 9
2.3 Category delta — gap closed per accessibility dimension . . . 10
2.4 What the 9-point gap actually means — WCAG deep-dive vs aquavena.nc . . . 11
2.5 Delivery velocity — engineering throughput per release . . . 12
2.6 DORA metrics — elite performance . . . 13
2.7 Development intensity . . . 15
2.8 AI-assisted delivery — Claude token consumption . . . 16
2.9 Semantic analysis of commit messages . . . 18
2.10 AI-leverage ratio per cluster — where the AI actually pays off . . . 49

3 Goals and Broader Vision **50**
3.1 Immediate goal . . . 50
3.2 A replicable methodology . . . 50
3.3 Transferability to other accessibility domains . . . 51
3.4 From measurement to optimisation . . . 53
3.5 Accessibility score as Business Value in Scrum . . . 53

4 Site Architecture **54**
4.1 Design principles . . . 54
4.2 Technology stack . . . 54
4.3 Data pipeline . . . 55
4.4 Accessibility engineering . . . 55

5 Tools **56**

6 Methodology **57**
6.1 Pipeline overview . . . 57
6.2 Release discipline . . . 58
6.3 Tool coverage . . . 59
6.4 Tools . . . 60
6.5 Standard . . . 60

7 Results — Our Site **60**
7.1 pa11y-ci — WCAG 2.1 AA . . . 60
7.2 Lighthouse Accessibility . . . 61

8 Results — aquavena.nc **62**

9 Comparison **62**
9.1 HTML quality & page weight . . . 62
9.2 Privacy & user tracking . . . 63

10 Accessibility Features **64**

11 Lighthouse History **65**

12 Score by Version **65**

13 Database **66**
13.1 Schema . . . 67
13.2 Example queries . . . 69

14 Glossary **69**

15 WebMCP — The Site as an Agentic Interface **71**
15.1 Why WebMCP? . . . 71
15.2 Implementation . . . 71
15.3 What It Brings . . . 72

16 Future Work — ML & Graph Data Science Roadmap **72**
16.1 Scale to N sites . . . 72
16.2 Graph data science on the audit network . . . 72
16.3 Deepen the commit-semantics work . . . 73
16.4 Time-series of accessibility evolution . . . 73
16.5 AI-Agent Universal Accessibility Pipeline . . . 73
16.6 Honest data requirements . . . 74
16.7 New perspectives unlocked by the 179-commit dataset . . . 75

17 Further Reading **76**

18 Conclusion **76**

A Appendix — Full Table Descriptions **77**
A.1 `git_commits` . . . 78
A.2 `ci_runs` . . . 78
A.3 `pa11y_issues` . . . 79
A.4 `lighthouse_audits` . . . 80
A.5 `score_history` . . . 80
A.6 `gh_releases` . . . 81
A.7 `html_quality` . . . 82
A.8 `privacy_audit` . . . 82
A.9 `claude_sessions` . . . 83
A.10 `claude_usage` . . . 84
A.11 `model_pricing` . . . 84

1 ❤ Inception

This project was not born in a research lab or a corporate accessibility audit. It started at a kitchen table, from a very simple and concrete situation.

My mother has been a loyal Aquavena customer for years. Every week she orders her meals from their service, and she genuinely enjoys the food. But she is visually impaired — and as her condition evolved, consulting the weekly menus on the official website became increasingly difficult, then impossible to do on her own. The layout, the font sizes, the colour contrasts, the absence of any audio alternative: none of it was designed with her in mind.

The consequence was small but recurring, and that is precisely what made it significant. Every week, she had to wait for someone to be available — a family member, a neighbour — to read the menus out loud to her. She lost the freedom to plan her own meals at her own pace. A minor dependency, perhaps, but one that repeated itself week after week, quietly eroding her autonomy.

I am a software engineer. I realised that a few hours of work could give that autonomy back to her: a dedicated page she could open on her tablet, with large text, high contrast, a dark mode for tired eyes, and a button to have the menus read aloud in her own language. Something she could use alone, at any hour, without asking anyone for help.

That is why this site exists. It is not a product, not a startup, not a statement. It is a small act of care, built with the tools I know, for the person who matters most. The accessibility work documented in this report is a direct consequence of that intent: if the site is meant to help someone with low vision, it had better actually be accessible.

A note to the Aquavena team, should they ever read this: there is no competition here, no monetisation, no hidden agenda. Every menu on this site links back to aquavena.nc to place orders. This is simply a more accessible front door to a service my mother loves.

2 📈 Executive Dashboard — Delivery KPIs

This dashboard reads directly from `benchmark.duckdb` at render time and produces the KPIs most relevant to a delivery committee: classical **EVM** (Earned Value Management), modern **DORA** (DevOps Research and Assessment) elite-performance indicators, the **WCAG defect burndown**, the **sprint velocity** profile, and the **strategic gap** between our work and the source site. Nothing in the charts below is hardcoded — every figure is computed live from the six FK-linked tables described in the appendix. Re-running the pipeline regenerates the dashboard automatically.

2.1 EVM — Planned Value vs Earned Value

A linear ramp from the **aquavena.nc baseline (91)** to the **target score (100)** defines the *Planned Value* curve — the score the project would have had at each release if it had

improved at a constant rate. The *Earned Value* curve is the **actual Lighthouse score** recorded at each release tag. Their ratio is the **Schedule Performance Index**,

$$\mathrm{SPI} = \frac{\mathrm{EV}}{\mathrm{PV}},$$

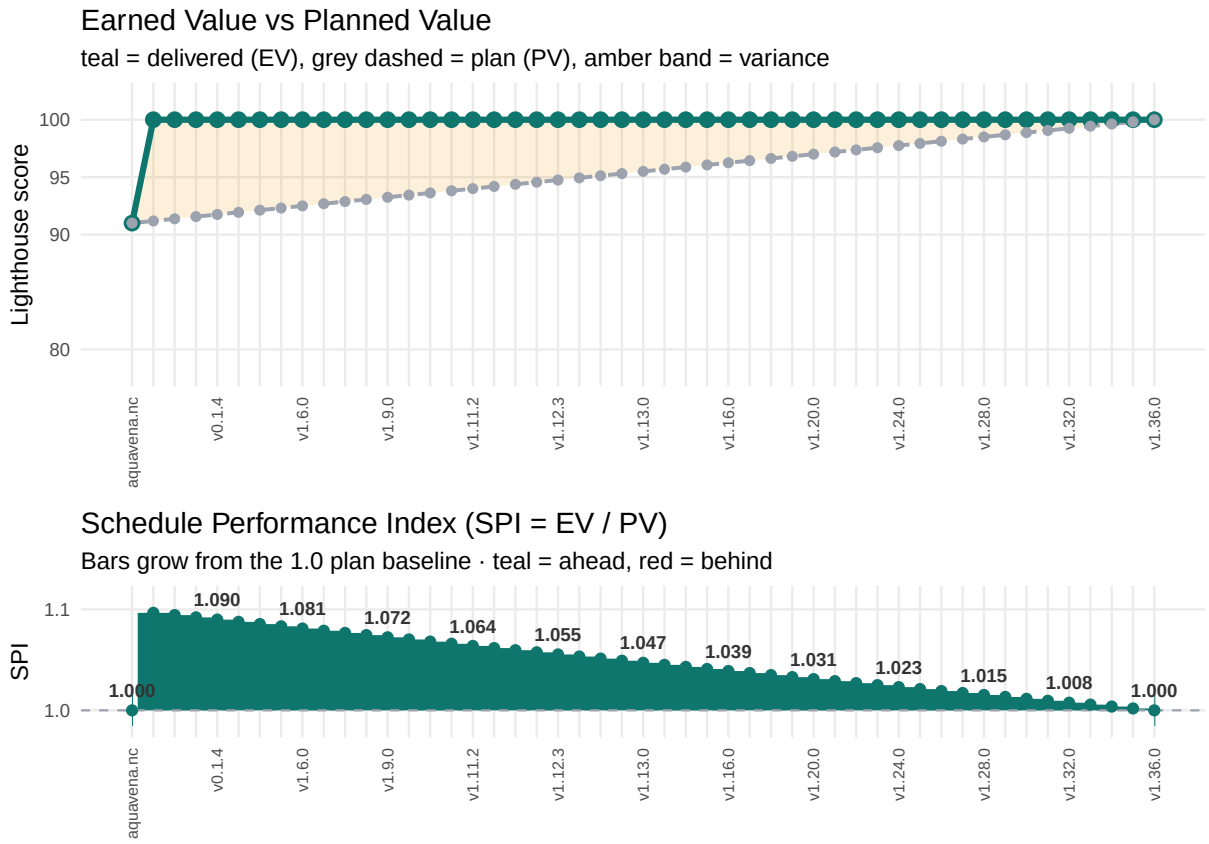
the most-watched indicator in PMI's Earned Value Management framework.

2.1.1 How to read the SPI chart

The dashed grey line at **SPI = 1.0** is the *plan baseline*. Every bar grows **upward from that line** to the actual SPI value for that release:

- **Teal bar above 1.0** → release is **ahead of plan** (Earned Value > Planned Value). The taller the bar, the more headroom recovered relative to what was scheduled.
- **Red bar below 1.0** (none in this dataset) → release would be **behind plan**. The longer the bar, the further behind.
- **No visible bar** (exactly at 1.0) → release is **on plan**.

For this project the chart shows a **uniformly-teal "ahead of plan" pattern**: the very first release (`v0.1.1`) lands at SPI ≈ **1.097** — already ~10% ahead of where the linear plan said we should be. The lead narrows release after release as the plan baseline ramps up toward the actual score, ending exactly on target at the latest release `v1.36.0` (SPI = 1.000). This is the visual signature of a project that **set the bar at the maximum value from day one** and held it across all 48 release tags — the cleanest possible EVM trace for a PMI committee.



What the EV line plateauing at 100 means. Earned Value hits the ceiling (100/100) at the very first release and never leaves it. In a typical project EV accrues *gradually* as scope is completed; here the curve is flat-topped because the **entire deliverable — full WCAG 2.1 AA conformance — was achieved in the first shippable version** and held across all 48 following tags. The plateau is the proof that accessibility was a *launch requirement,* not a backlog item polished over time: after release one there was simply no value left to earn. The only thing that still moves is the grey Planned-Value line slowly rising to meet it.

What the steadily-declining SPI curve means. The downward drift is *not* a loss of quality — the delivered score (EV) never drops below 100. SPI falls only because the **plan catches up**: Planned Value ramps linearly from 91 to 100 while the actual score was already maxed out from the first release. A SPI that decreases monotonically yet stays **above 1.0** is the textbook signature of **front-loaded delivery** — all the value was banked on day one, and each subsequent release simply lets the linear plan consume a little more of that head start, until the two meet exactly at SPI $= 1.0$ on the final release. In EVM terms the project was never “slowing down”; the plan was merely growing into a result that had already been achieved.

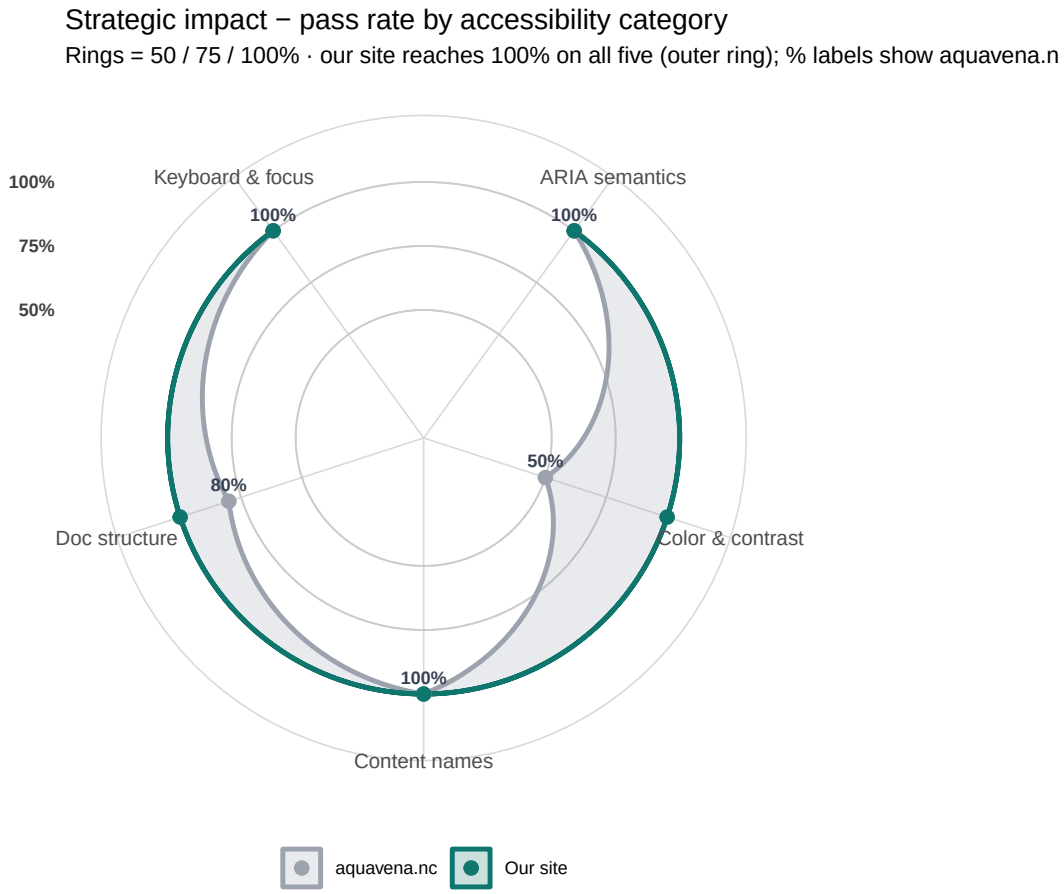
2.2 Strategic impact — multi-axis radar

What is Lighthouse? **Lighthouse** is Google's open-source auditing engine, built into Chrome DevTools and shipped as a CLI (github.com/GoogleChrome/lighthouse). Its **accessibility category** runs the axe-core rule engine (by Deque Systems) against a page and returns a 0–100 score that is a *weighted pass rate* over several dozen individual audits — `color-contrast`, `image-alt`, `aria-*`, `meta-viewport`, and so on — each mapped to a WCAG 2.1 success criterion. We capture the full JSON report (not just the headline number), which is what lets us break the score down by category below and per release tag elsewhere. Like pa11y, it only checks the machine-testable subset of WCAG — a useful floor, not a complete accessibility verdict.

Each Lighthouse audit is classified into one of five accessibility categories, and the **pass rate per category** is computed per site. The radar makes the strategic gap immediately readable: where the source site falls short, where it does well, and where our rebuild closes the gap entirely.

The five categories — and why each one matters, especially for a low-vision or elderly reader:

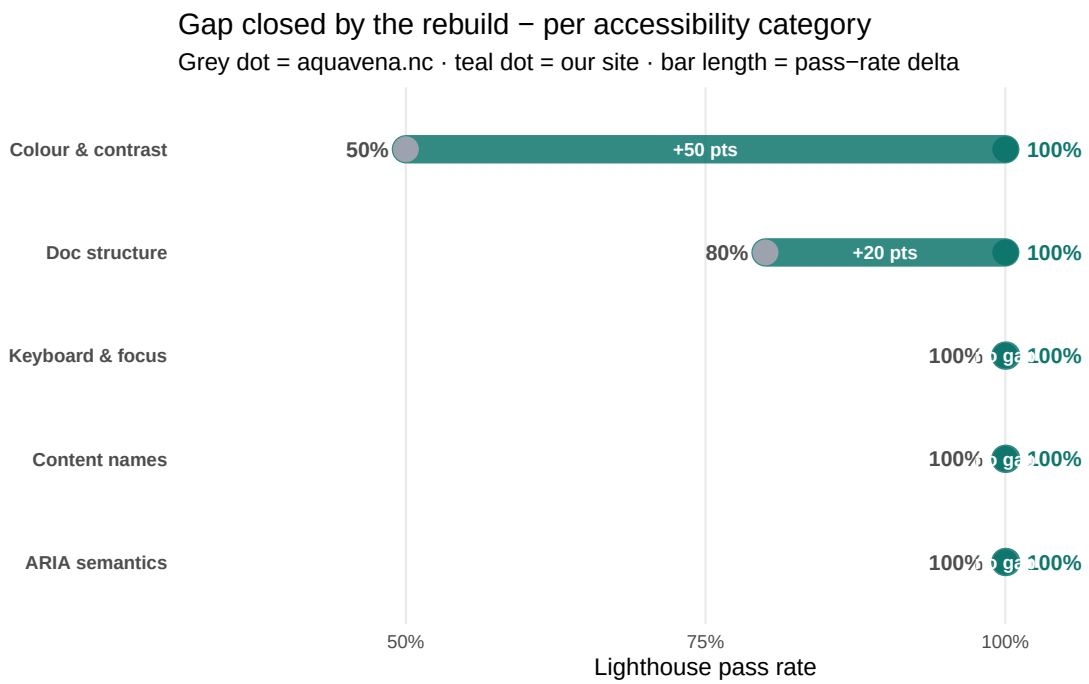
- **Color & contrast** — Whether text and interface elements meet the WCAG contrast ratios (4.5:1 for normal text) and stay distinguishable without relying on colour alone (audits `color-contrast`, `link-in-text-block`). This is the make-or-break dimension for ageing eyes and reduced contrast sensitivity — a pale button that "looks fine" can be invisible.
- **Doc structure** — The page's structural scaffolding: a declared language, a page title, a logical heading order, and a viewport that permits zoom (`html-*`, `document-title`, `heading-order`, `meta-viewport`). Good structure gives a screen reader a navigable outline and lets *everyone* enlarge the page on a phone.
- **Content names** — Whether every meaningful element carries an **accessible name**: the short piece of text a screen reader reads aloud to identify the element, computed from its alt text, `<label>`, `aria-label` or visible text. This category checks each one actually has it — alt text on images, labels on form fields, discernible names on buttons and links (`image-alt`, `button-name`, `link-name`, `label*`). Without it, a screen reader can only announce "button, button, button" — the user hears that controls exist but not what they do.
- **Keyboard & focus** — Whether the whole interface is operable without a mouse: sensible focus order, visible focus, skip links, and large-enough touch targets (`tabindex`, `target-size`, `bypass`, `skip-link`, `focus-*`). Essential for keyboard, switch and motor-impaired users — and for anyone with an unsteady hand.
- **ARIA semantics** — **ARIA** (*Accessible Rich Internet Applications*) is a W3C standard set of HTML attributes that describe dynamic widgets — menus, dialogs, tabs, sliders — so a screen reader can announce their **role** (what it is), **state** (e.g. expanded/collapsed) and **properties**, in cases where plain HTML is not expressive enough. This category checks those attributes are used correctly (the `aria-*` audits). Mis-used ARIA is worse than none — it actively misleads the screen reader (e.g. announcing a button as a checkbox).



2.3 Category delta — gap closed per accessibility dimension

What is pa11y? Alongside Lighthouse, this report relies on **pa11y** — a free, open-source command-line accessibility tester (github.com/pa11y/pa11y). pa11y loads each page in a headless Chromium and runs it through the HTML_CodeSniffer ruleset against the **WCAG 2.1 AA** standard, returning one *error* per element that fails a success criterion. We run it on every page in CI (`pa11y-ci`), so **"0 pa11y errors" means every audited element passed the automated WCAG 2.1 AA checks**. It complements Lighthouse rather than duplicating it: the two engines overlap on contrast but each catches rules the other misses. The usual caveat applies — automated testing only covers the machine-checkable subset of WCAG, so it is a reliable *floor*, not a replacement for manual and assistive-technology testing.

The replay shows our site at **100/100 Lighthouse, 0 pa11y errors** across every tag, while `aquavena.nc` sits at 91/31. Instead of an iterative burndown, the chart below visualises the **one-shot transformation** as a category-by-category **gap closed**: for each accessibility dimension Lighthouse audits, the bar shows the pass-rate delta between the source site and ours. The longer the bar, the bigger the gap the rebuild closed in that dimension.



2.4 What the 9-point gap actually means — WCAG deep-dive vs aquavena.nc

A Lighthouse score of **91/100** for the official site sounds almost fine. It is not — because that single number averages away the fact that the official site's two remaining failures are **precisely the two that matter most for an ageing, low-vision reader**. This section opens the score and names them, criterion by criterion.

Table 1: WCAG 2.1 AA failures on the official site vs our rebuild (pa11y HTML_CodeSniffer + Lighthouse). Official site: 31 pa11y errors, Lighthouse 91; our site: 0 errors, Lighthouse 100.

WCAG criterion	Level	aquavena.nc	Our site	Low-vision impact
1.4.3 — Contrast (Minimum)	AA	31	0	Critical
1.4.4 — Resize Text	AA	1	0	Critical

Failure 1 — contrast (SC 1.4.3). pa11y flags **31 contrast errors** on the official menu page, and Lighthouse independently confirms **26 failing nodes** — including the primary "*commander*" call-to-action button. Below the 4.5:1 ratio, text does not merely look pale; for someone with reduced contrast sensitivity (cataract, age-related macular changes) it can vanish entirely. Our rebuild carries **0** contrast errors.

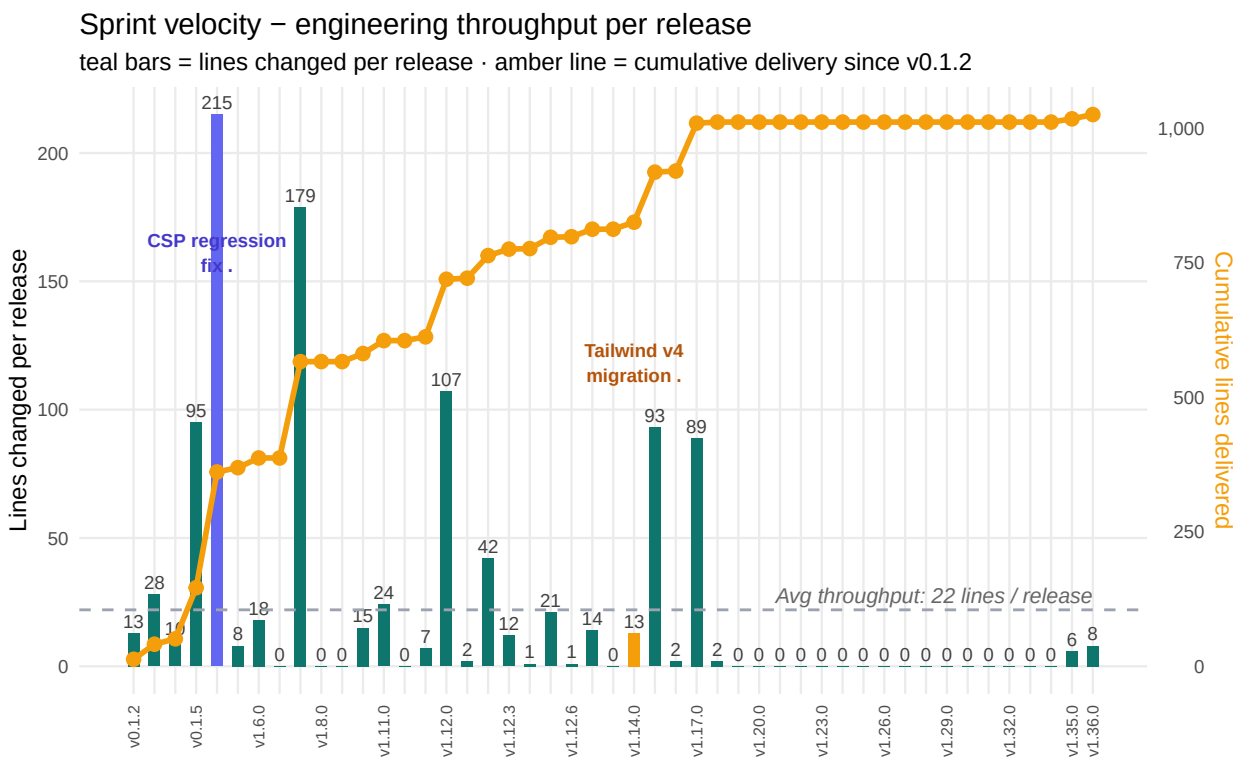
Failure 2 — zoom disabled (SC 1.4.4 / Resize Text). This is the damning one. The official site ships `<meta name="viewport" content="… maximum-scale=1.0, user-scalable=no">`, which **switches off pinch-to-zoom on mobile**. Enlarging the page is the single most common — often the *only* — coping strategy a low-vision or elderly person has on a phone, and the official site forbids it outright. No Lighthouse average conveys how disabling that one attribute is. Our rebuild leaves zoom fully enabled.

Why this reframes the whole benchmark. The honest comparison is not "91 versus 100" — a 9-point cosmetic gap. It is binary: on the official site **my mother cannot reliably read the menu, and cannot enlarge it to try.** On the rebuild she can. Every other analysis in this report — the ML, the causal graphs, the token economics — exists to make *that* difference repeatable and measurable, not the reverse.

2.5 Delivery velocity — engineering throughput per release

Per-release **lines of code changed** (teal bars) — the engineering throughput each sprint poured into the codebase. The **cumulative lines delivered** since `v0.1.1` is overlaid as an amber line on the right-hand axis. Reading the chart: after the initial scaffold (`v0.1.1`), the project sustained a steady cadence of small surgical sprints early on (`v0.1.2`–`v0.1.5`), then larger investment releases for tooling and reporting (`v1.5.0`–`v1.7.0`).

The two highlighted bars mark the project's only regressions. The indigo bar (`v1.4.0`) is the **CSP regression fix**: a Content Security Policy — the HTTP header that restricts which scripts and resources a page may load, a standard defence against cross-site-scripting (XSS) — had been enabled but was too strict and blocked the site's *own* JavaScript under Astro v6, hiding every menu; this release relaxed the policy so the page rendered again. The amber bar (`v1.14.0`) is the **Tailwind CSS v4 migration**, the other notable refactor of the period.



2.6 DORA metrics — elite performance

2.6.1 Origin

DORA = DevOps Research and Assessment. Founded in 2013 by **Nicole Forsgren**, **Jez Humble** and **Gene Kim** as an independent research programme initially published with Puppet — the infrastructure-automation company behind the open-source configuration-management tool of the same name, which sponsored the early *State of DevOps* surveys — then spun out as an autonomous company. **Acquired by Google Cloud in 2018**, where it continues to publish the annual *Accelerate State of DevOps Report*. The programme is the empirical foundation of modern DevOps benchmarking and the source of the four metrics used in this section.

Official sources.

- dora.dev — the programme's home: research, the four keys, and the DORA Quick Check.
- *Accelerate State of DevOps Report 2024* (PDF) — the most recent full report.
- *The ROI of AI-assisted Software Development,* 2026 (PDF, Google/DORA) — directly relevant to this report's AI-delivery and token-economics analysis.

Note on the homonymy. The acronym DORA is also widely used in another context: the **Digital Operational Resilience Act**, a European Union regulatory framework (Regulation EU 2022/2554) designed to fortify the **financial sector** against information and communication technology (ICT) threats. It is unrelated to the software-delivery metrics used in this report. For the EU regulation see eiopa.europa.eu/digital-operational-resilience-act-dora.

2.6.2 The four metrics

Metric	Definition
Deployment Frequency	How often a team successfully releases to production
Lead Time for Changes	Time from code commit to running in production
Change Failure Rate	% of deployments causing a production failure (rollback, hotfix)
Mean Time to Recovery (MTTR)	Average time to restore service after a production incident

2.6.3 Performance tiers (Accelerate State of DevOps Report 2023)

The four metrics are mapped to four performance tiers. **Our site** lands in the **Elite** column on all four indicators (change-failure rate is derived from the two known regressions — Tailwind v4 and the CSP issue — over all releases):

Metric	**Elite**	High	Medium	Low
Deployment Frequency	**On-demand** (multiple/day)	1/day to 1/week	1/week to 1/month	< 1/month
Lead Time for Changes	**< 1 day**	1 day to 1 week	1 week to 1 month	> 1 month
Change Failure Rate	**0–5%**	10–15%	16–30%	64–75%
MTTR / Recovery time	**< 1 hour**	< 1 day	< 1 week	> 1 week

Note: the 2025 DORA report moved away from tier ranking in favour of seven *team archetypes* that integrate human factors (burnout, friction, perceived value). The Elite/High/Medium/Low grid above remains the canonical reference for cross-team benchmarking.

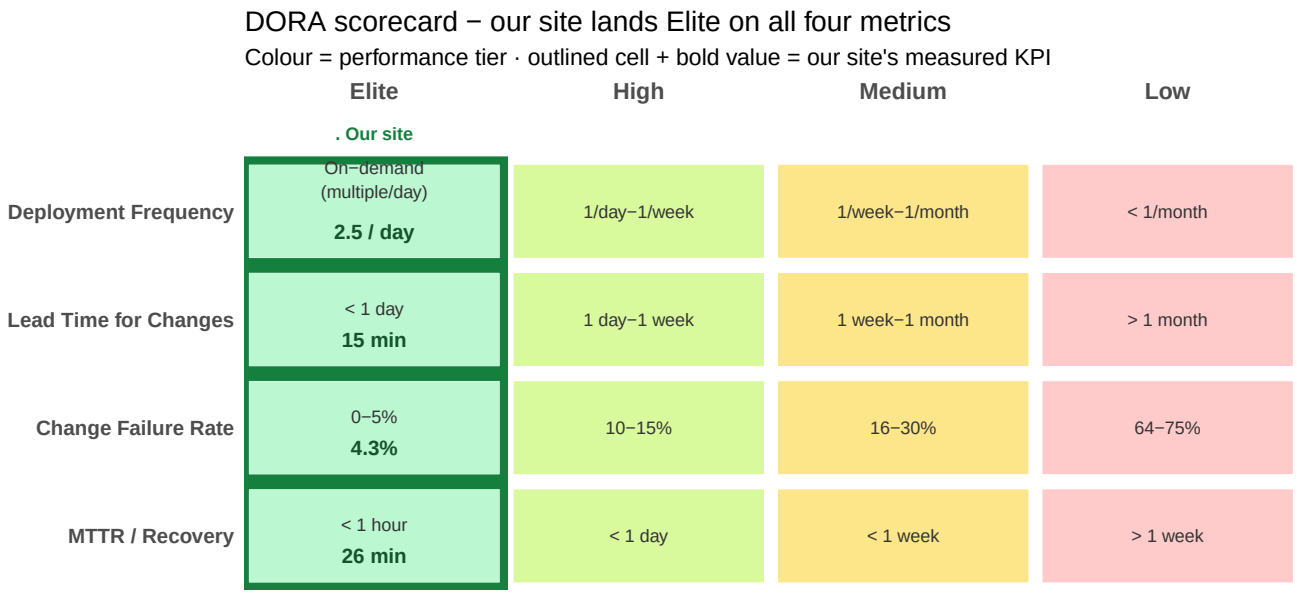
The DORA metrics in this report are computed from the timestamps stored in `gh_releases.published_` and from the `ci_runs` table.

2.5/day	**14.6 min**	**26 min**
Deployment Frequency	Lead Time for Changes	MTTR (Tailwind regression)
Elite	*Elite*	*Elite*

Forsgren et al., Accelerate State of DevOps Report 2023, Google Cloud / DORA — Elite tier: on-demand deploys, lead time < 1 h, MTTR < 1 h.

> **Lead Time for Changes** — time between consecutive releases (commit merged → live on GitHub Pages). Measured here as the median gap between release timestamps. A short lead time means fixes and features reach users quickly.

The grid below summarises the same four metrics as a colour-coded **scorecard**: each row is a metric, each column a performance tier, and the cell where **our site** actually lands (always **Elite**) is outlined and stamped with the project's measured KPI. (These are *delivery* metrics of our own repository — not of the official aquavena.nc site.)



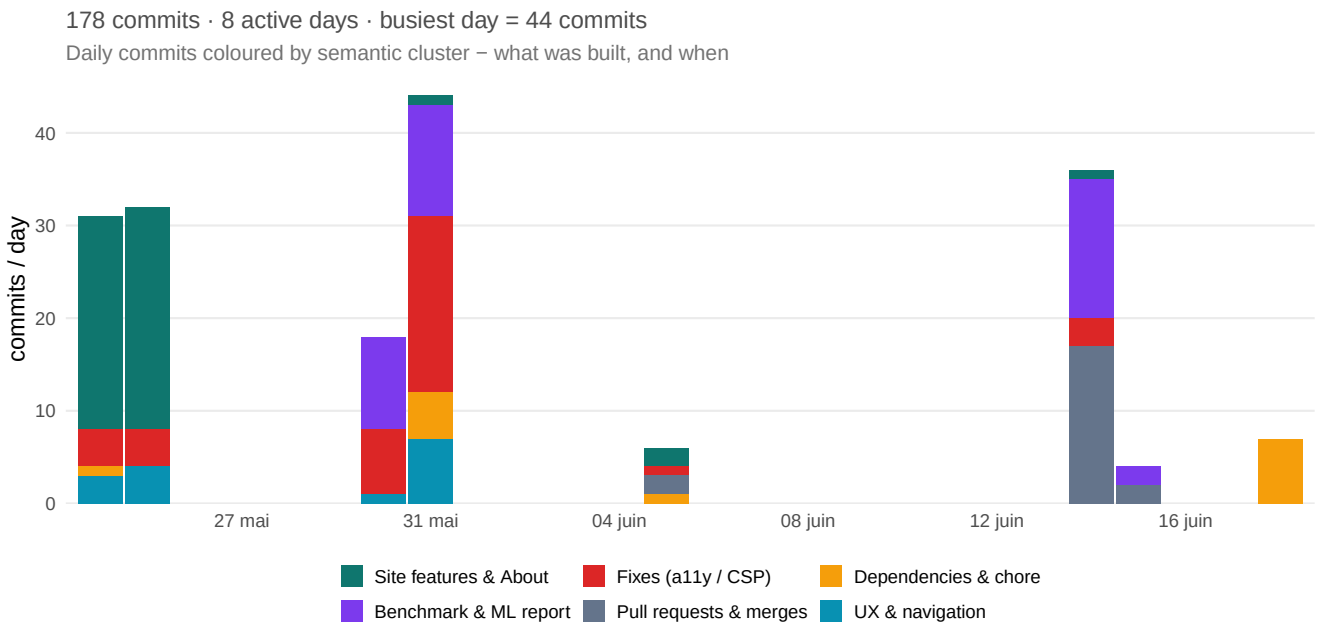
2.6.4 Verifiable references

For independent verification of the DORA framework and the Elite-tier thresholds used above:

- **Foundational book** — Forsgren, Humble, Kim. *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations.* IT Revolution, 2018. ISBN 978-1942788331.
- **Official site** — dora.dev (current research programme home).
- **2023 report — Infographic PDF (canonical Elite/High/Medium/Low table)** — dora.dev/research/2023/2023-DORA-Report-Infographic.v10.pdf.
- **2023 report — Google Cloud announcement** — cloud.google.com/blog/products/devops-sre/announcing-the-2023-state-of-devops-report.
- **Wikipedia (history and acquisitions)** — en.wikipedia.org/wiki/DevOps_Research_and_Assessm

2.7 Development intensity

This chart plots **daily commit volume, coloured by semantic cluster** — *what* the project was building, and *when*. Each bar is one day: its height is the number of commits and its coloured segments show the mix of work (site features, fixes, benchmark/report, dependencies, …). It turns the raw activity log into a phase-by-phase story of the build.



What it reveals. Two things at once. First, the cadence is unmistakably **bursty**: the commits land on a handful of active days out of the full calendar span — long idle stretches punctuated by short, intense sessions, the rhythm of focused hackathon-style delivery rather than a steady 9-to-5 cadence. Second, the **colour mix shifts across the timeline**: the early bars are dominated by *Site features & About* and *Fixes (a11y / CSP)* — the founding accessibility work — while the later bars turn toward *Benchmark & ML report*, *Pull requests & merges* and *Dependencies & chore*, the report-engineering and maintenance phase. It is the same founding → analysis pivot the concept-emergence timeline recovers later from the commit text alone — here visible directly in the shape and colour of the work.

2.8 AI-assisted delivery — Claude token consumption

This project was built with **Claude Code**. Every assistant message and every cached context replay is recorded locally; `extract_claude_usage.py` parses those transcripts into the `claude_sessions` and `claude_usage` tables. Joining with `git_commits` via timestamp range gives **tokens per release** (apportioned by the time elapsed between consecutive tags).

What does “cache hit” mean here?

Claude charges for **every** token, but the **prompt cache** changes the rates dramatically:

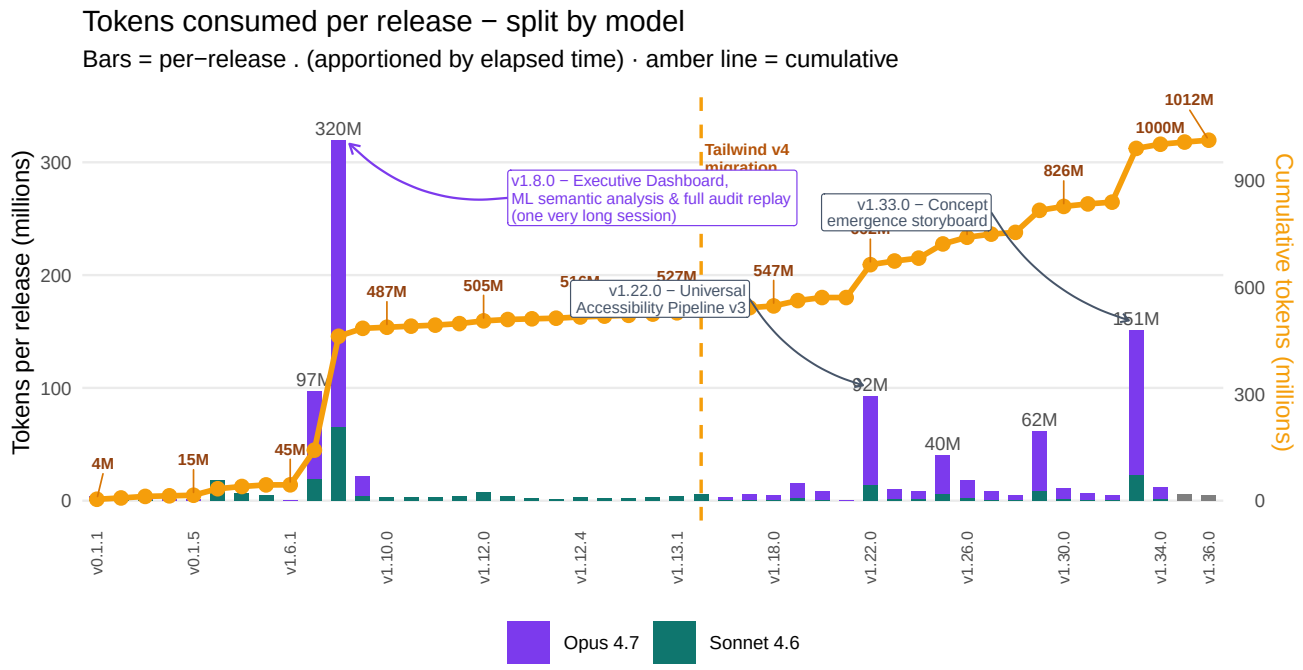
Token category	Price (Sonnet 4.6)	Price (Opus 4.7)	Relative
Fresh input (cache miss)	$3.00 / M	$15.00 / M	**1.0×**
Cache write (one-off setup)	$3.75 / M	$18.75 / M	1.25×
Cache read (cache hit)	**$0.30 / M**	**$1.50 / M**	**0.1×**
Output tokens	$15.00 / M	$75.00 / M	5.0×

A **cache hit** is when the model serves part of the prompt from cache instead of re-encoding it. It is **still billed**, but at **10% of the fresh-input rate**.

Why our 98% cache-hit ratio matters. In a long Claude Code conversation the entire prior transcript is replayed as context with every new message. Without caching, this re-encoding cost would scale roughly quadratically with session length. With caching, it scales linearly — and at the 10% rate. Concretely: the ~420 M cache-read tokens we logged would have cost **$1,260** at fresh-input rates; they actually cost about **$126**. A **10× saving**, automatically.

What “good” looks like in our case. A high cache-hit ratio (≥ 90%) means we are working as efficiently as possible — long, coherent sessions reuse the same context. A **low** cache-hit ratio would indicate either many short sessions (each pays the cache-write tax without amortising it) or the conversation constantly bringing in fresh files. For project-style work like this benchmark, the 98% ratio is **the ceiling, not the average** — it is the strongest signal that the project was delivered in a small number of focused sessions rather than fragmented across many cold starts.

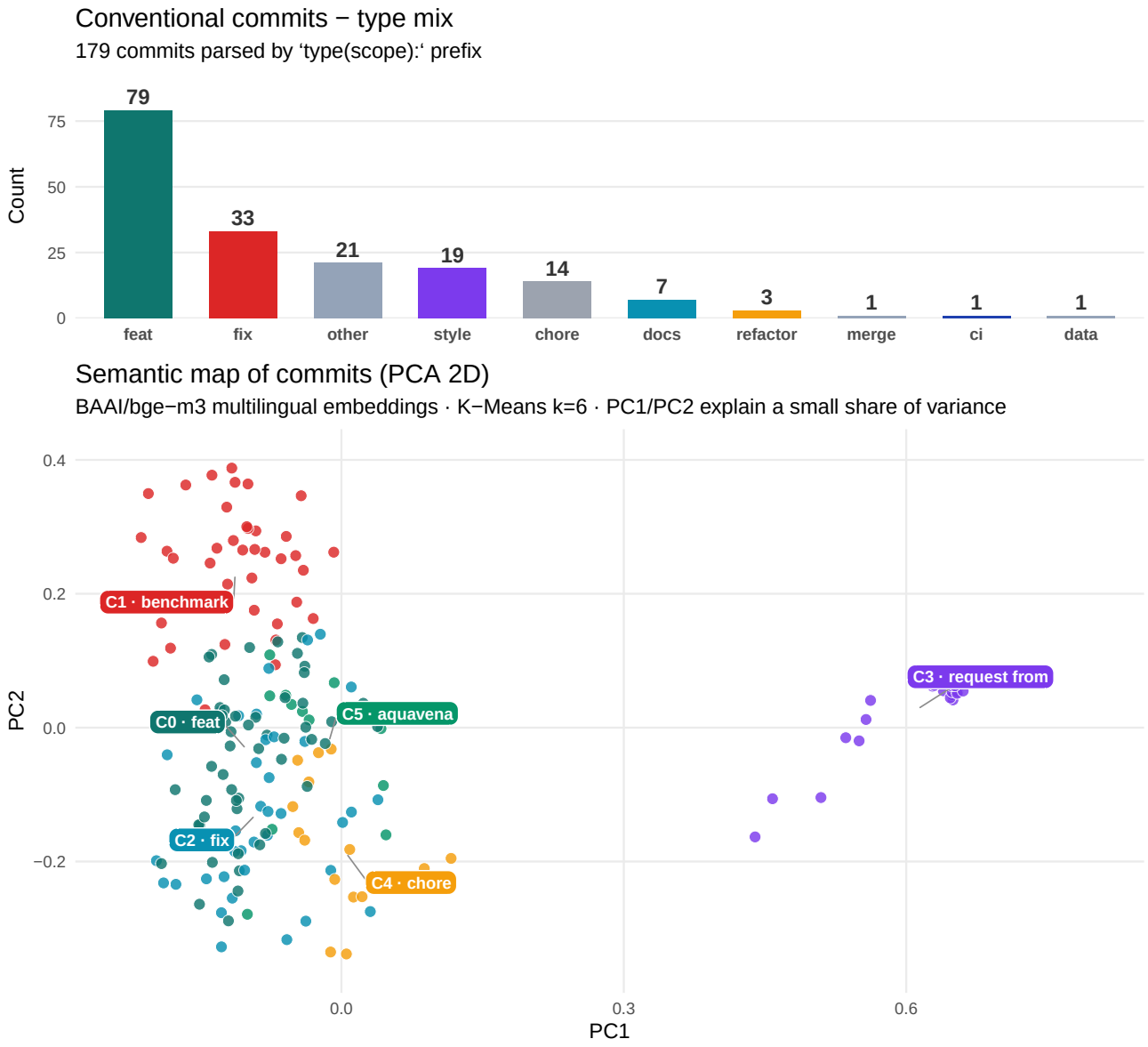
331.40 M	**758.46 M**	**98.1%**	**3368 | 5114**
Sonnet 4.6	Opus 4.7	Cache hit	Messages
tokens	*tokens*	*ratio*	*you | Claude*



2.9 Semantic analysis of commit messages

All **179 commit messages** were embedded with the `BAAI/bge-m3` model (1024-dimensional **multilingual** embeddings — handles the project's mix of French and English commit messages), clustered with K-Means into 6 groups, and projected to 2D with PCA. The result is stored in two tables — `commit_classification` (per-commit type/scope/cluster/coords) and `commit_clusters` (per-cluster size and characteristic terms) — and is queryable directly from DuckDB.

The top chart shows the **conventional-commit type mix** (parsed from the `feat:` / `fix:` / `docs:` prefixes); the chart below is the **2D semantic map** — each dot is one commit, coloured by its semantic cluster. Note that PC1 and PC2 together explain only a small share of the variance, so the 2D view is an *approximation*: most commits bunch on the left while one semantically distinct group sits apart on the right.



Why the map splits into a dense left cloud and one isolated group on the right. The right-hand group is a single cluster — the **merge commits** (`Merge pull request #N from …`). Their subject lines are boilerplate and nearly identical, so the embedding maps them to almost the same point; in effect **PC1 (the horizontal axis) separates templated merge messages from hand-written prose**, which is why they sit far to the right, apart from everything else. The dense cloud on the left is *all the substantive work* — `feat`, `fix`, `style`, `benchmark`, `chore` — which describes the same project (accessibility, menus, the report) in natural language and therefore shares vocabulary. Those commits are semantically close, so they overlap; and because PC1/PC2 capture only a small fraction of the 1024-dimensional embedding, the 2D view cannot pull the sub-clusters apart as cleanly as the full-dimensional K-Means does (which is exactly why the GraphSAGE and cluster cross-validations elsewhere in this report matter — they confirm the separation that the eye cannot see in 2D).

The cluster terms below come from per-cluster TF-IDF; each row's exemplar is the commit closest to the cluster centroid in embedding space.

> **TF-IDF** (Term Frequency – Inverse Document Frequency) measures how characteristic a word is to one cluster compared to all others. A term scores high when it appears often *inside* a cluster (TF) but rarely across the other clusters (IDF). It is what makes "accessibility" a top term for the feature cluster but not for the fix or chore clusters — even if the word appears everywhere, the IDF weight penalises it unless it is truly distinctive.

Cluster	Commits	Top terms	Exemplar
0	51	feat, style, add, feat add, about	feat: move accessibility score section to top of About page
1	39	benchmark, feat, feat benchmark, feat report, report	feat(benchmark): révision git dans le rapport + page abstract dédiée
2	38	fix, fix benchmark, benchmark, csp, les	fix(benchmark): lignes source correctes pour toutes les releases
3	21	request from, request, pull request, pull, from adriens	Merge pull request #29 from adriens/feat/report-git-sha
4	15	chore, deps, site, de, data	chore(deps): bump @tailwindcss/vite from 4.3.0 to 4.3.1 in /site (#31)
5	15	aquavena, feat ux, ux, feat, aquavena-sdk	feat(ux): déplacer A+ / A− sous le titre Aquavena

2.9.1 Graph data science — anchors & bridges

The embeddings already cluster the commits. Treating them as a **graph** — nodes = commits, edges = cosine similarity ≥ 0.55 — adds a second layer of insight: which commit best *represents* each theme (highest **PageRank** within its cluster, the **anchor**), and which commits *connect* different themes (highest **betweenness centrality**, the **bridges**). Both metrics are computed with `networkx`, persisted in `commit_graph_metrics` for direct SQL queries, and exported as `benchmark/data/commit_graph.gexf` — a **Gephi-ready file** with all node attributes (cluster, PageRank, betweenness, degree, type, scope, timestamp) and edge weights, so the graph can be explored interactively (ForceAtlas2 layout, community detection, etc.) in Gephi without re-running the pipeline.

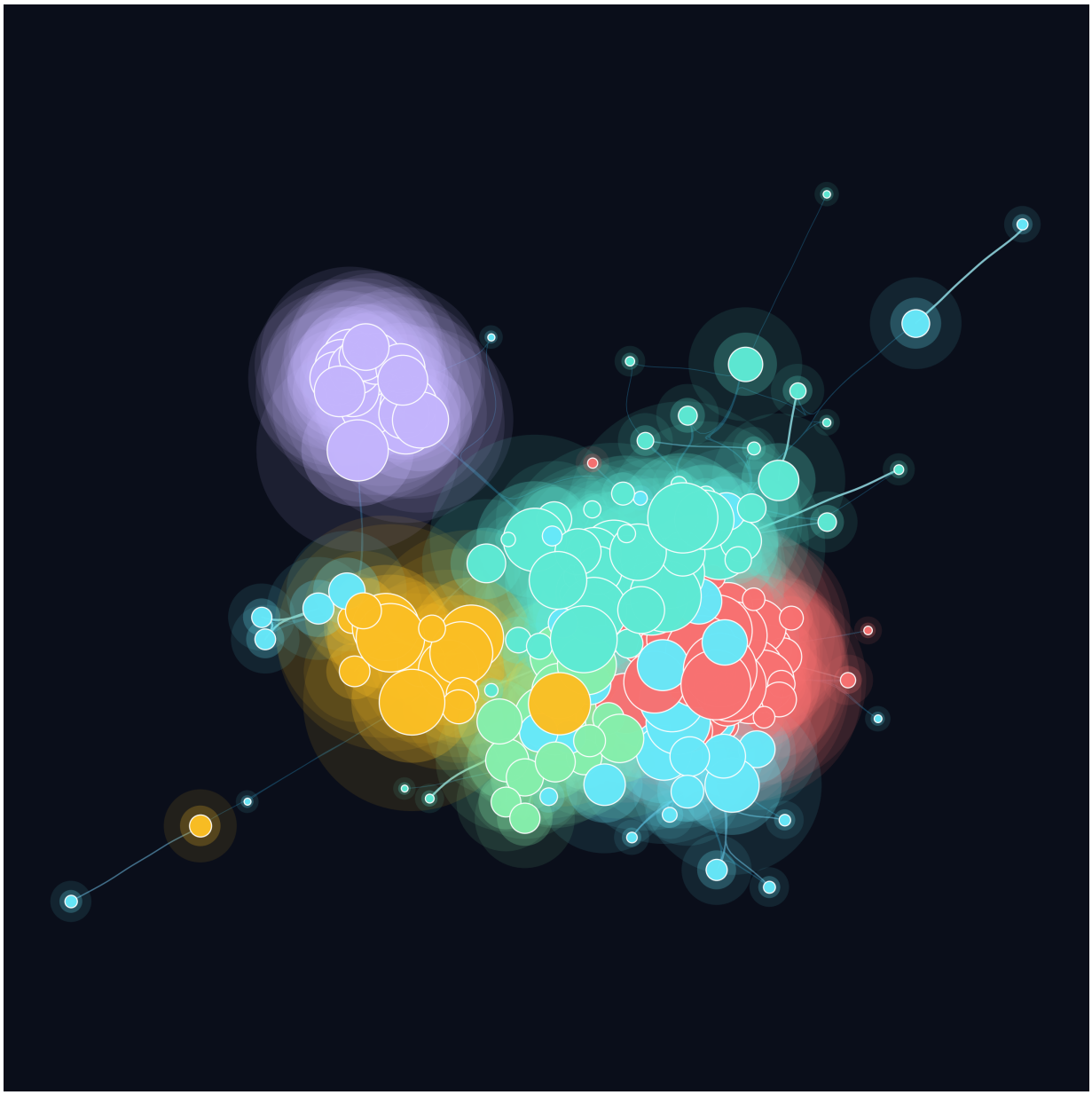


Figure 1: Commit-similarity graph — **ForceAtlas2** layout (Gephi native algorithm, `fa2-modified` port, 1 400 iterations) · node colour = semantic cluster · node size = PageRank (percentile-scaled, ≈80× area ratio) · edges = cosine similarity ≥ 0.55

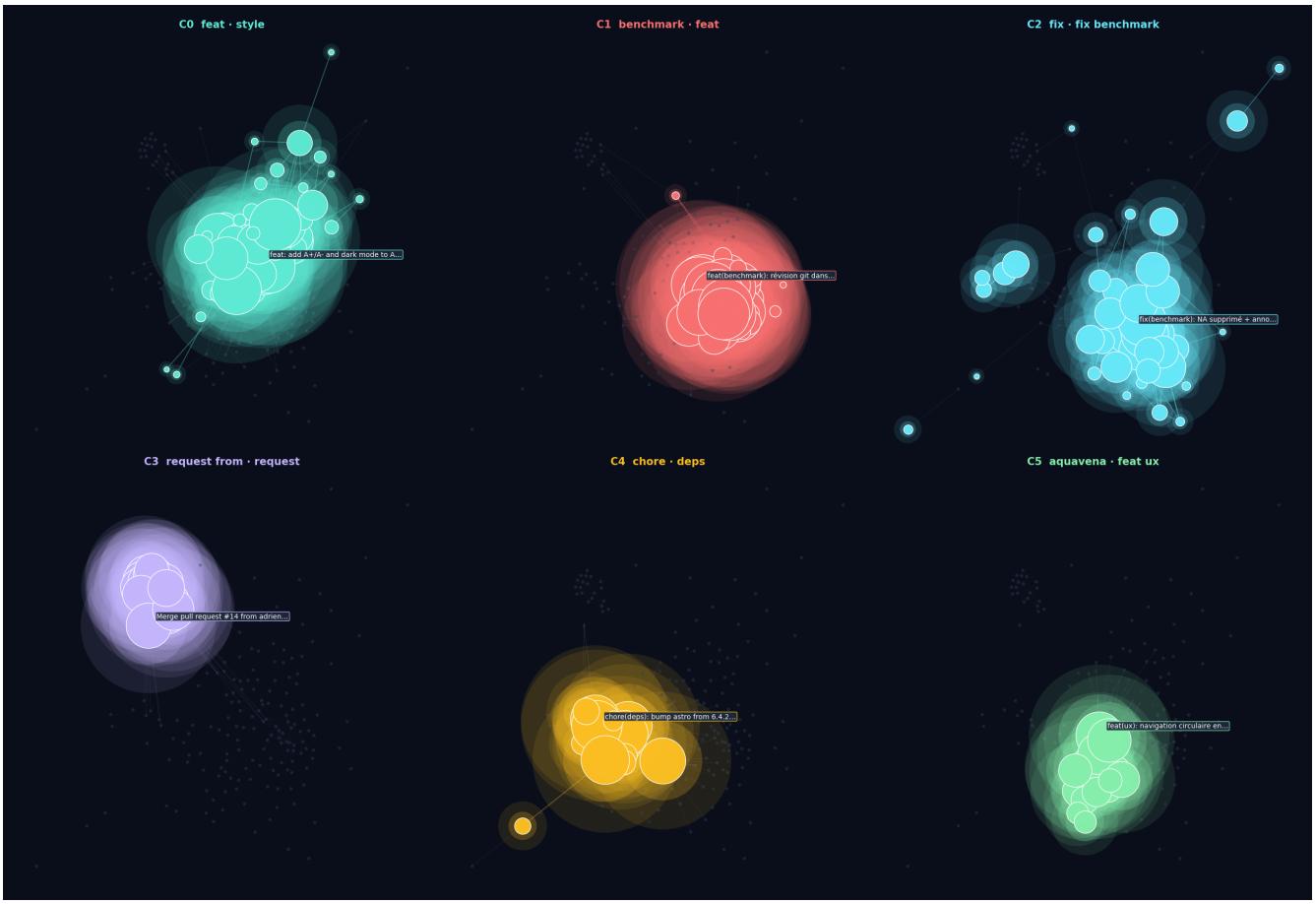


Figure 2: Cluster-split view — same ForceAtlas2 positions as above · each panel spotlights one semantic cluster (full glow) while the rest fade to grey · intra-cluster edges are coloured, bridge edges are muted · anchor label = commit closest to centroid

Table 2: Anchor commit per cluster — the most-representative work in each theme

Cluster	PageRank	Anchor commit
C0	0.0109	feat: add A+/A- and dark mode to About page header
C1	0.0154	feat(benchmark): révision git dans le rapport + page abstract dédiée
C2	0.0096	fix(benchmark): NA supprimé + annotation Tailwind v4 + labels hors barres
C3	0.0070	Merge pull request #14 from adriens/dependabot/npm_and_yarn/site/astro-6.4.4
C4	0.0084	chore(deps): bump astro from 6.4.2 to 6.4.4 in /site
C5	0.0073	feat(ux): navigation circulaire entre les onglets (←/→ et swipe)

Table 3: Top-5 bridge commits — connect distinct themes (highest betweenness centrality)

From cluster	Betweenness	Bridge commit
C1	0.1048	feat(benchmark): révision git dans le rapport + page abstract dédiée

C1	0.0791	feat(report): refonte du diagramme Universal Accessibility Pipeline
C0	0.0484	feat: add A+/A- and dark mode to About page header
C0	0.0470	feat(site): afficher Entrée, Dessert, Végé, Fresh et Enfants dans le template
C4	0.0431	chore: refresh menus data

What the graph reveals

Anchors (cluster representatives). Each cluster's anchor — the commit closest to its centroid — faithfully summarises that cluster's theme: the site-features cluster anchors on an **About-page accessibility commit** (`feat: move accessibility score section to top of About page`), the reporting cluster on a `feat(benchmark):` commit, the fixes cluster on a `fix(benchmark):` commit, the maintenance cluster on a `chore(deps):` bump, and so on. The **About-page and accessibility work sits at the centre of the largest cluster** — the part of the codebase where the project explains itself most fully and where the embeddings find the highest density of similar content.

Bridges (theme-linkers). The top-5 betweenness commits all touch **multiple themes simultaneously** — they're the commits that introduced A+/A- *and* dark mode (style + feature), or moved the accessibility-score section *to* the About page (a11y + About). These are the **integration commits** — the ones a knowledge-graph would surface as architecturally critical.

Why this matters at scale. With 179 commits the graph is small, but the *technique scales linearly with commit count*. Applied to a 1000-commit codebase, PageRank surfaces the commits that anchor each subsystem; betweenness surfaces the integration commits whose removal would *fracture* the codebase. Both are signals an experienced reviewer would notice manually — this method makes them queryable.

2.9.2 Cluster drift — a project maturity signal

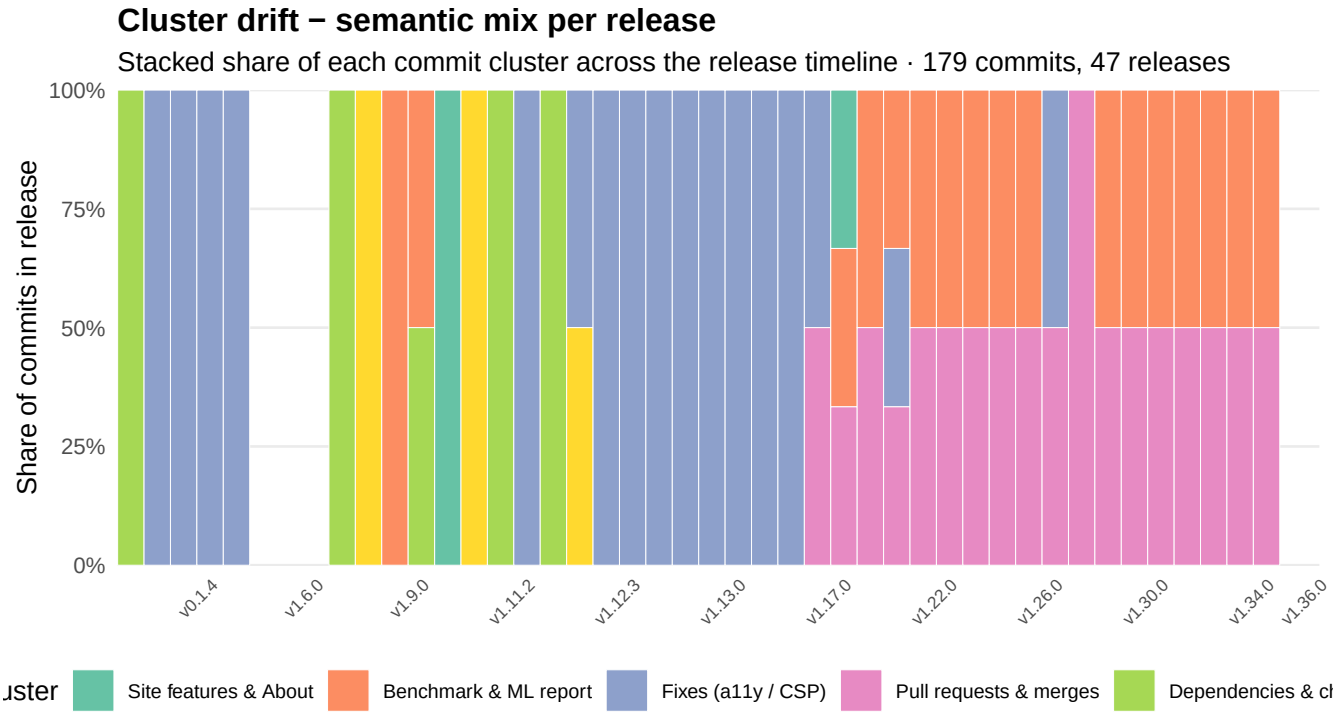
What it is. Each commit belongs to one of six semantic clusters discovered earlier. The **cluster drift** chart plots, for every release tag along the project's history, the *proportion* of commits from each cluster that landed in that release. Stacked together, these proportions form a **time-series of the project's semantic mix**.

What it measures. The shifting composition of work, release after release. Early releases tend to be dominated by **core-feature clusters** (accessibility fixes, UX features); mid-life releases shift toward **hardening clusters** (dependency bumps, bug fixes, style); mature releases drift toward **meta-work clusters** (reporting, documentation, tooling, agentic instrumentation). This is the **maturity signature** of a software project.

What it is for. A *zero-label* maturity metric. No human annotation is required: the cluster mix per release falls out of conventional commits and sentence-transformer embeddings already computed by the pipeline. It answers questions a PMI committee or steering board typically asks subjectively — *"is this project still in feature mode, or is it in maintenance? has the team moved on to a new phase?"* — and turns them into a measurable, reproducible signal.

What to watch for.

- A **steady single colour** across releases → monolithic project with one type of work being done over and over.
- A **smooth gradient** from one cluster to another → healthy evolution from build to stabilisation.
- An **abrupt cluster appearing late** → the project pivoted (new domain, new platform, new audience). On Aquavena, the *Benchmark & ML report* cluster appears massively from `v1.7.0` onwards: this is the visual signature of the project shifting from *building an accessible site* to *measuring and publishing what was built* — the report you are reading right now.



Reading this chart on Aquavena.

- The **earliest tags** (`v0.1.2`–`v0.1.5`) are single-commit releases, each a pure *Fixes (a11y / CSP)* increment — the foundational accessibility work (contrast corrections, pa11y audit setup, WCAG 2.1 AA from the ground up) in its purest form. This is the *raison d'être* of the project.
- The bulk of the early engineering is absorbed by the first tags that catch a large commit window (`v0.1.1`, then `v1.6.1` and `v1.7.0`): a blend where *Site features & About* and *Benchmark & ML report* dominate, with *Fixes (a11y / CSP)* and *UX & navigation* alongside. This is the moment the project widened from a single concern — *make it WCAG-clean* — to building features, navigation and the benchmark itself.

- The **analysis / agentic phase** (`v1.2x–v1.3x`) shifts decisively into the *Benchmark & ML report*, *Pull requests & merges* and *Dependencies & chore* registers — the report-engineering and maintenance work that produced the document you are reading. Feature and fix work give way to data-science and upkeep.

Note: the project's version tags are **not strictly chronological**, so each bar groups every commit up to that tag's publication time. A few early tags therefore absorb large commit windows (which is why their bars are the tallest), while many later tags carry only one or two commits.

This single chart tells the story of the project's life cycle without a single line of manual annotation. Applied to other codebases, it becomes a portable **maturity radar** — a public-service site stuck on a single cluster for two years is not evolving; a site whose cluster mix shifts is iterating.

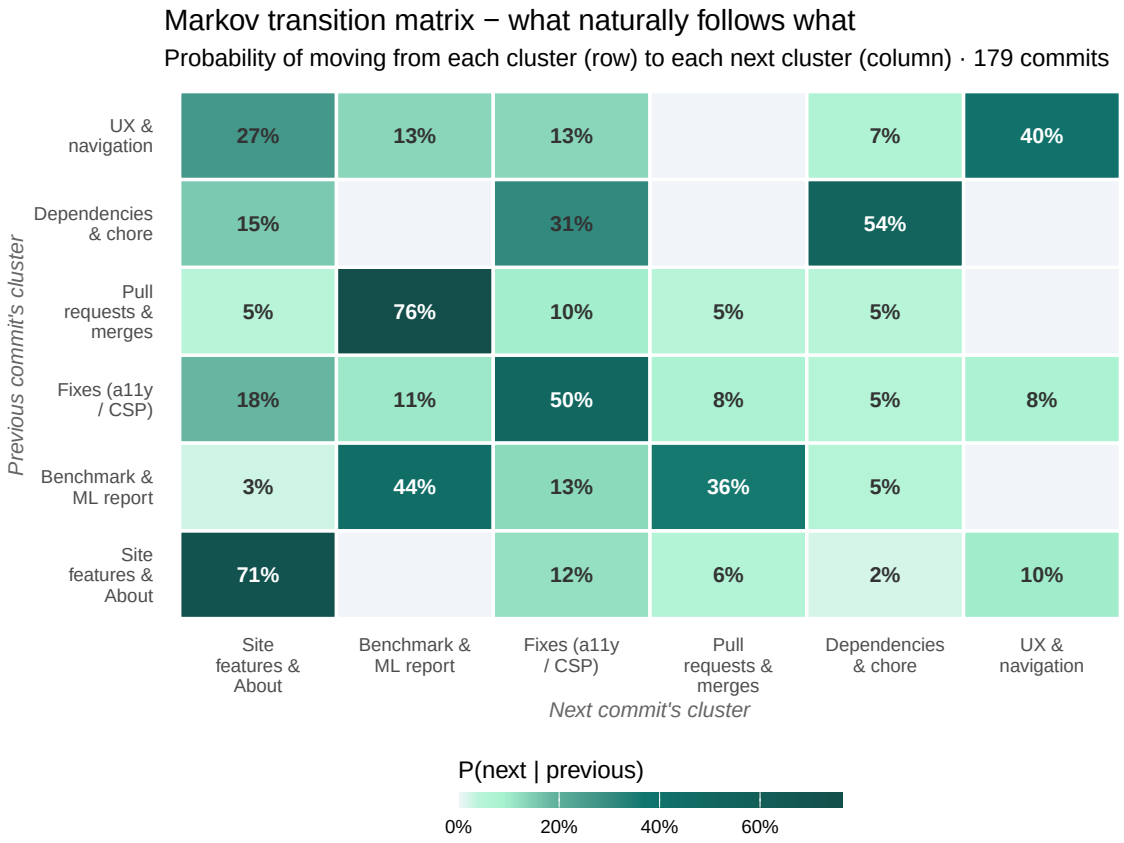
2.9.3 Markov transition matrix — what naturally follows what

What it is. Sort all 179 commits in chronological order, then for every consecutive pair (commit_i, commit_$i+1$) record the directed transition between their semantic clusters. Counting transitions gives a 6×6 matrix; row-normalising it yields a **first-order Markov chain** — a matrix of conditional probabilities $P(\text{next cluster} = j \mid \text{previous cluster} = i)$.

What it measures. The *empirical routines* of the developer. The cluster drift chart above shows the long-run mix of work; the Markov chain shows the **short-term flow** — which type of commit naturally precedes which.

What it is for.

- A behavioural fingerprint of how the team works.
- An input to **agentic pipelines**: the AI agents from Section *Universal Accessibility Pipeline* can pre-warm a context for the likely next commit type — e.g. after a *Fixes (a11y / CSP)* commit, the agent loads contrast-audit tools because $P(\text{next} = \text{Fixes})$ is high.
- A regression detector at the **process level**: if the empirical mix diverges from the historical Markov chain (e.g. a sudden burst of *UX & navigation* after a *Benchmark & ML report* commit), it flags an unusual work session.



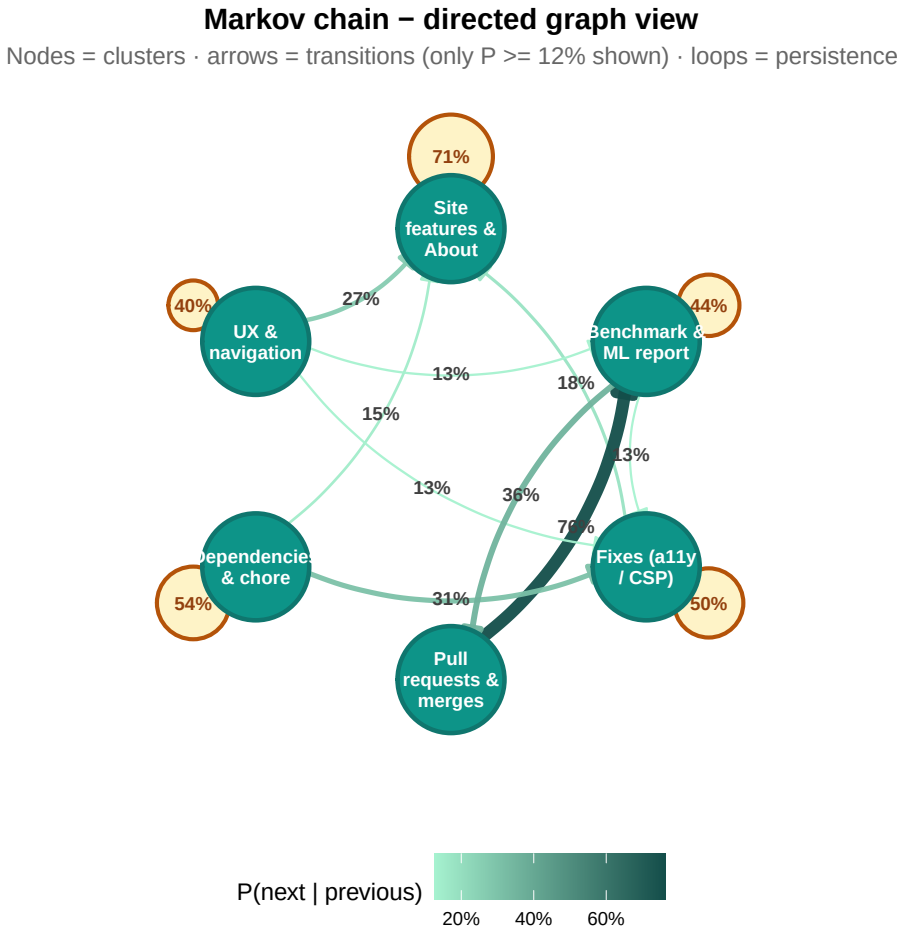
Reading the matrix.

- The **diagonal** measures *persistence*: how likely the developer is to stay in the same cluster for the next commit. A high diagonal value means the developer works in focused sessions, not by jumping between topics. On Aquavena, the *Site features & About* cluster has the highest self-loop at **71%** — this is the cluster the developer "stays in" the longest.
- The **strongest off-diagonal transition** is **Pull requests & merges → Benchmark & ML report** at **76%**. This is the dominant *flow* of the project — the routine the developer falls into most often when changing topics.
- A **sparse row** (mostly zeros) means a cluster acts as a *terminal state* — once entered, the developer rarely transitions out of it without a break.
- A **dense row** means the cluster is a *hub* — it leads naturally to many other types of work.

This is the same mathematical object used in 1990s NLP for letter-bigram language models, in queuing theory for system performance, and in reinforcement-learning environments. Here it gives a **fingerprint of how the project is actually built** — not how the developer says they work, but what the timeline shows.

2.9.3.1 The same matrix viewed as a graph

A Markov transition matrix is, by construction, the **weighted adjacency matrix of a directed graph**: nodes are clusters, edges are the conditional probabilities. Reading the matrix gives precision; reading the graph gives **structure**. The chart below renders the chain as a graph — only transitions with $P \geq 12\%$ are drawn so the layout stays legible. Self-loops are shown as small annotations next to each node.



Reading the graph.

- A **thick, dark arrow** between two clusters means the developer routinely flows from one to the other. The graph makes these *highways* immediately visible — the heatmap shows them too, but the eye has to scan a 6×6 grid; the graph hands them to you.
- A **node with a large amber loop** is a "sticky" cluster — the dev enters and tends to stay there.
- A **node with mostly outgoing arrows and a small loop** is a *hub* cluster — a routing point in the workflow rather than a destination.
- **Isolated nodes** (no in/out edges above threshold) would mean a cluster that is sampled but never strongly connected — none here, which confirms a coherent project.

The matrix and the graph carry the **same information**; the graph trades precision for shape recognition. In a real engineering review the two are complementary — heatmap for the audit, graph for the briefing.

2.9.4 Workflow entropy — how predictable is the next commit?

What it is. Shannon entropy is a number, in bits, that measures how *uncertain* a probability distribution is. For each row of the Markov matrix (i.e. for each starting cluster) we compute

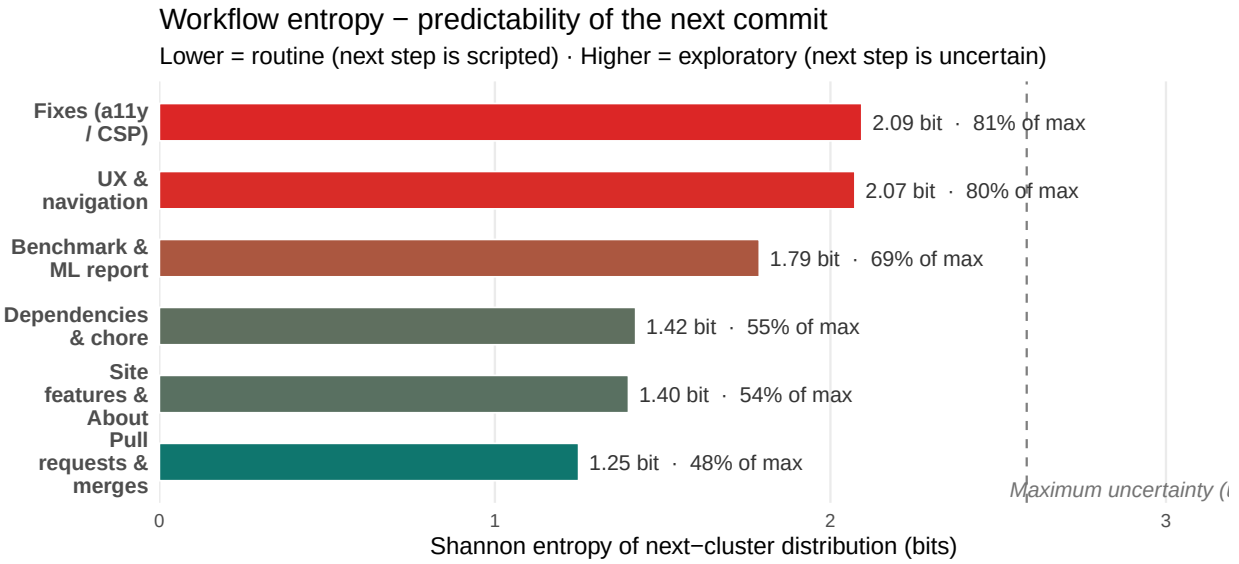
$$H(\text{next} \mid \text{previous} = i) \;=\; -\sum_{j} P_{ij} \log_2 P_{ij}.$$

A row with entropy $H = 0$ bit means the next commit is **fully determined** — the developer always goes to the same place after this cluster. A row with $H = \log_2(6) \approx 2.58$ bit means the next commit is **maximally uncertain** — the developer could go anywhere.

Why it matters. Entropy is the canonical mathematical measure of *predictability*. In compression theory it tells you how many bits you need to encode the next symbol; here it tells you how much "surprise" the next commit holds. Two practical readings:

- **A low-entropy cluster is a routine.** Once the developer enters it, the next step is almost scripted. These clusters are good candidates for **automation** — a CI agent could pre-stage tools or files for the predictable next action.
- **A high-entropy cluster is a fork in the road.** The developer is multitasking or context-switching after this cluster. These are the zones where **AI assistance pays off most** — the LLM helps the human pick the next thread.

The trend over time matters too. Computed release-by-release, the *average row entropy* shows whether the project is becoming **more routine** (entropy decreasing → mature, repeatable workflow) or **more exploratory** (entropy increasing → genuine R&D phase). It is a maturity signal orthogonal to the cluster-drift chart: drift shows *what* you work on, entropy shows *how predictable* the work is.



The most *routine* clusters are the ones with the lowest bars — the developer almost always knows what comes next. The most *exploratory* clusters have bars approaching the dashed

line (the theoretical maximum for a 6-cluster system). On Aquavena, the spread between the lowest and highest entropies tells how varied the workflow actually is.

2.9.5 Burnout window detection — friction signals from the AI telemetry

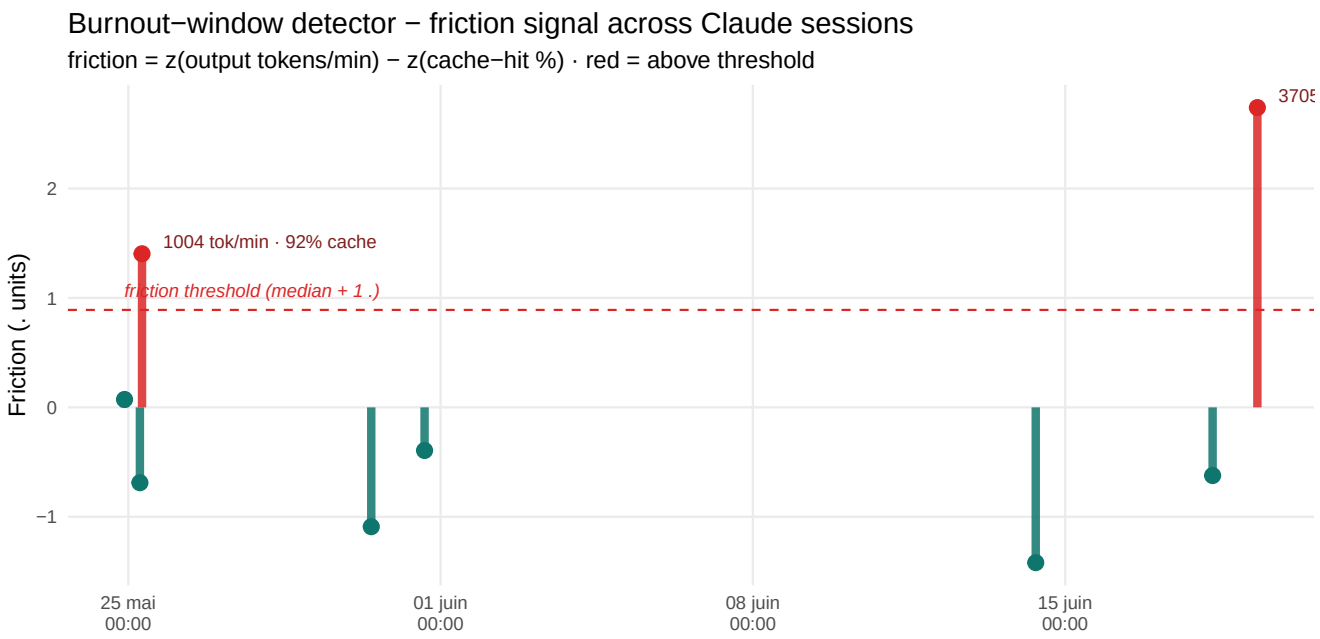
The idea. A long Claude Code session leaves a trail of measurable signals: tokens consumed, cache-hit ratio, time between commits. Specific *patterns* in those signals are physiologically meaningful:

- **Output tokens per minute spikes** → the developer is asking the model for a lot of generation, often a sign of stuck-and-iterating
- **Cache-hit ratio drops below baseline** → the conversation is constantly bringing in new files instead of reusing context: the developer is searching, not refining
- **Inter-commit gap stretches** while the session stays active → fewer successful commits per unit of session time: friction, not flow

Why it matters. Together, these three signals form a **data-driven proxy for cognitive friction** — what one would loosely call a *burnout window* or a *struggle period*. Nothing about subjective mood is measured directly; only objective telemetry from the AI tool. But the combination is meaningful: when all three signals deviate together, something hard is going on.

What it is for.

- **For the developer**: post-hoc retrospective — *"on day X around 17h I was stuck for 90 min on the Fixes (a11y / CSP) cluster"*. A coachable signal.
- **For the team**: capacity planning. If burnout windows correlate with certain clusters, those clusters are objectively heavier and deserve more time or smaller chunks.
- **For the assistant itself**: a future Claude could detect the pattern in real time and proactively suggest *"let's pause and reset"* or *"would a different angle help?"*.



How to read it. Each vertical bar is one Claude Code session. The red dashed line is the *friction threshold* — median plus one standard deviation. Bars in red exceed it: those sessions show the combined signature *high output rate + low cache reuse*. Labels next to each red bar give the raw rates so the reader can sanity-check the call.

Honest caveat. This is a *proxy*, not a diagnosis. A red bar on a day where the developer was deliberately exploring a new library is *expected*, not pathological. The signal is most useful in aggregate: clusters of red bars across consecutive days indicate sustained friction; a single red bar is just a hard session. The pipeline records the data; **the interpretation stays with the human**.

2.9.6 Causal impact — measuring what an intervention really cost

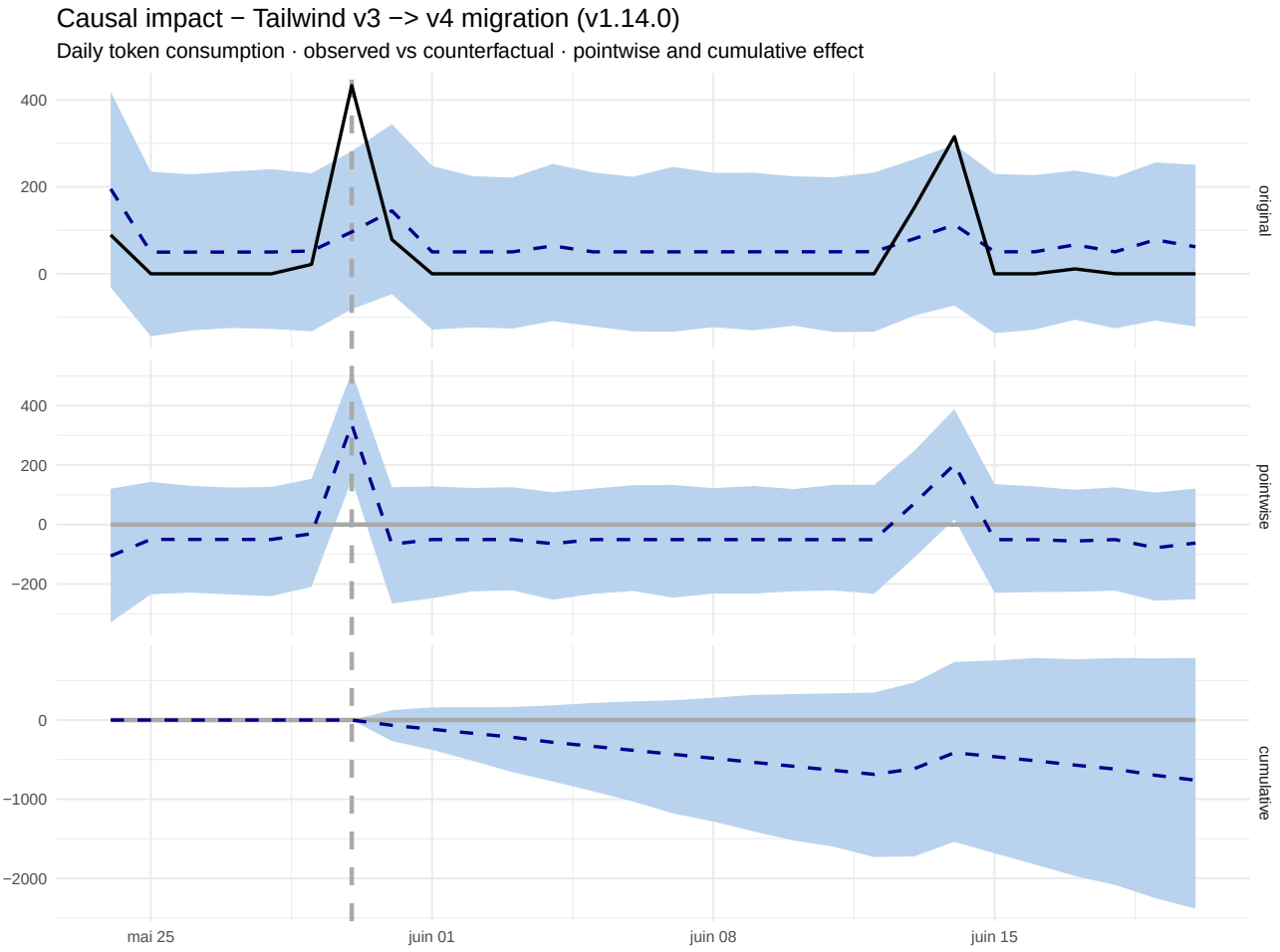
What we want to know. When the Tailwind v3 → v4 migration shipped in `v1.14.0`, several things happened at once: tokens were consumed, sessions stretched, commits piled up. A simple before/after comparison cannot tell us *how much of that was the migration* and *how much was the project's normal dynamics*. We want the **causal effect** of the intervention — the extra cost it imposed on top of what would have happened anyway.

The method. Bayesian Structural Time Series (BSTS) builds a synthetic *counterfactual* — what the time series *would have looked like* if the intervention had never happened — and compares it to the observed series. The difference, with a 95% credible interval, is the estimated causal effect. The canonical implementation is Google's **CausalImpact** package (vignette on CRAN), originally developed to measure the lift of advertising campaigns. It is the gold standard for non-experimental causal estimation when a single sharp intervention is known in advance.

Honest sample-size disclaimer. CausalImpact behaves best with ≥ 30 pre-intervention observations. Aquavena gives us roughly **one week of pre-Tailwind history** and **two**

weeks post, aggregated to daily buckets — enough to demonstrate the method end-to-end, not enough to publish the result as a finding. The credible intervals are correspondingly wide. The point of this section is the *infrastructure*, not the verdict: once a project accumulates a few months of telemetry, the very same code yields publication-grade causal estimates.

2.9.6.1 Tailwind v3 → v4 migration (`v1.14.0`)

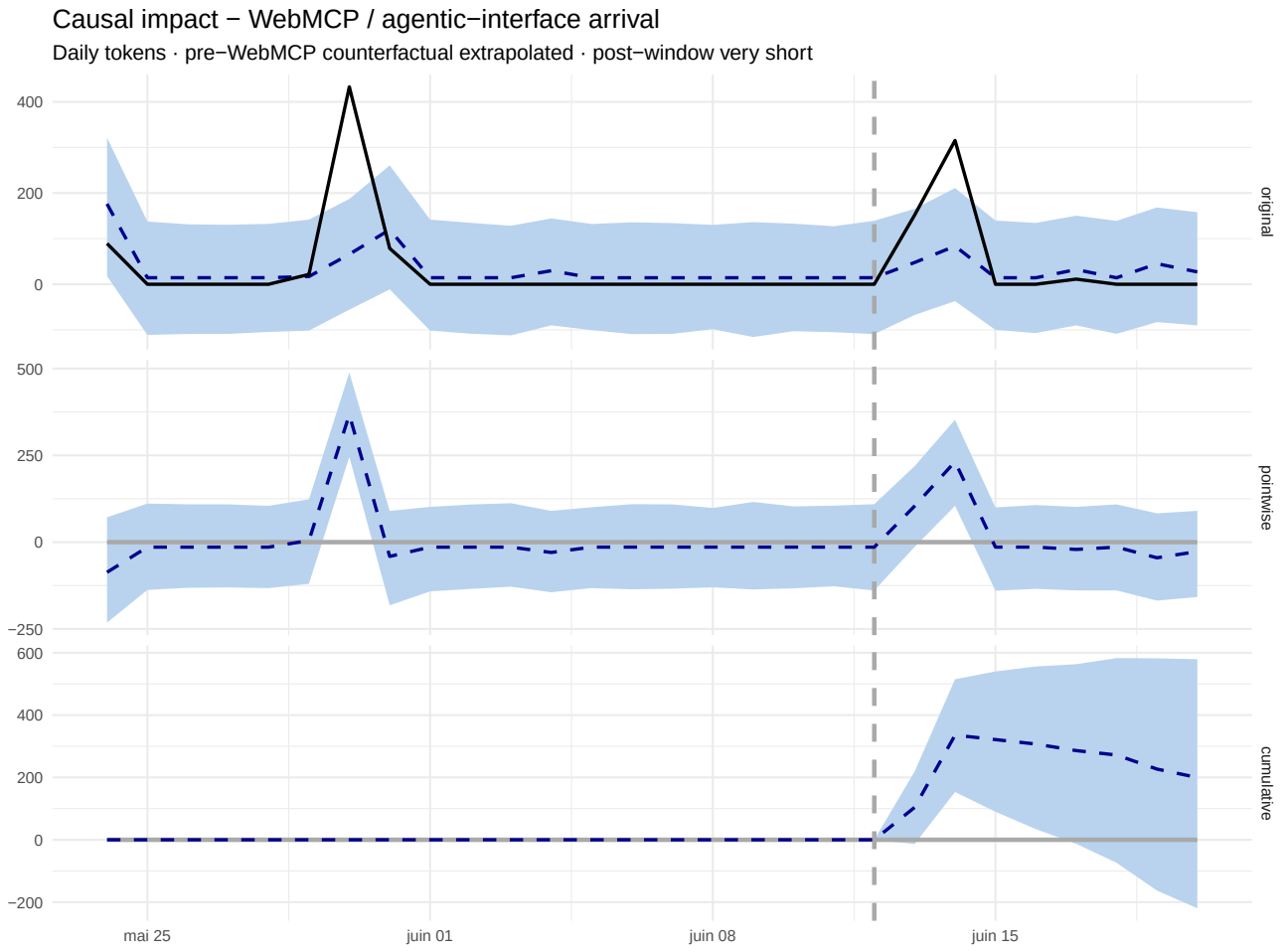


Tailwind v4 — causal verdict

Cumulative excess token consumption attributable to the migration: approximately **-760.2 M tokens** over the post-intervention window (95% credible interval: -2383.5 M to +783.4 M · Bayesian p-value: 0.184).
A positive number means the migration *increased* the daily token rate beyond what the pre-Tailwind dynamics would have predicted; a negative number means it *decreased* it. The wide credible interval reflects the short pre-history — the same analysis on a longer dataset would tighten the bounds substantially.

2.9.6.2 WebMCP / agentic interface arrival (mid-June)

The WebMCP work — recorded as `feat(webmcp):` and `feat(report):` commits on 13–14 June — is treated here as a second candidate intervention. As of this build's snapshot, **the post-WebMCP window is only ~1–2 days**, which is **insufficient** for CausalImpact to converge: the credible interval will span the entire plausible range. We run the analysis anyway to demonstrate that the framework picks up the intervention as soon as enough post-history accumulates.



WebMCP arrival — causal verdict (preliminary)

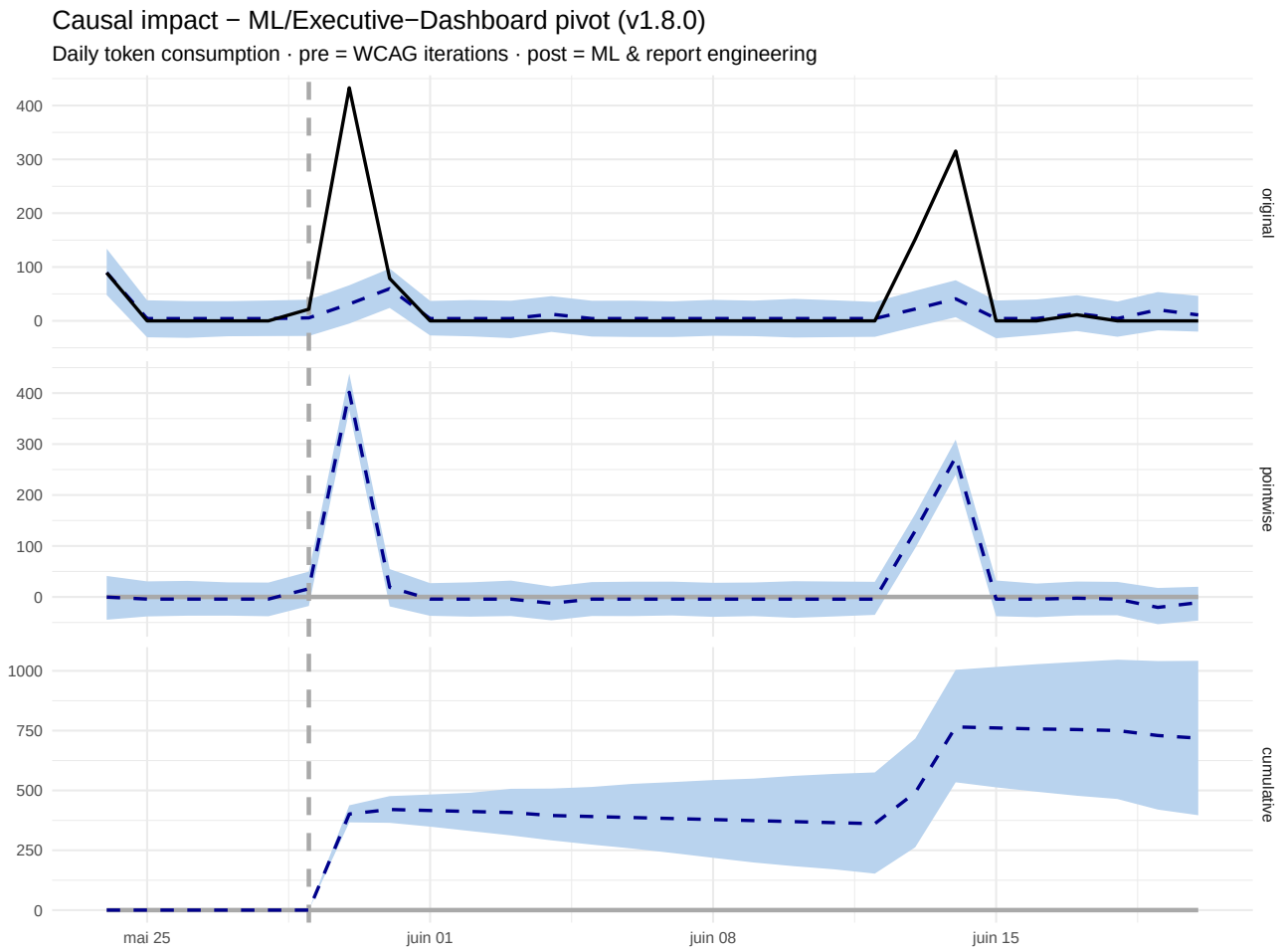
Cumulative excess token consumption attributable to the WebMCP/agentic work, on the very short post-window currently available: **+199.7 M tokens** (95% credible interval: -219.8 M to +579.3 M, Bayesian p = 0.167).
The interval is wide and the verdict tentative — this section will sharpen automatically as the post-WebMCP history grows in subsequent pipeline runs.

2.9.6.3 Executive Dashboard + ML semantic analysis (`v1.8.0`) — the *real* pivot

Tailwind v4 and WebMCP both gave **inconclusive** verdicts. This is not a failure of the method — it is the method working correctly on under-powered samples. We now apply CausalImpact to the intervention that, *qualitatively*, was the project's most disruptive: the introduction in `v1.8.0` of the **Executive Dashboard**, the **sentence-transformer (BAAI/bge-m3) embeddings**, the **K-means clustering**, the **PageRank + betweenness graph metrics**, and the **ForceAtlas2 visualisations**. This is when the project stopped being a website and became a data-analysis pipeline.

The intervention has **two structural advantages over Tailwind**:

- The post-period (≈14 days) is the same, but the *mechanism* is much stronger — long ML-heavy Claude sessions vs short framework refactor.
- The cluster-drift chart already shows the visual signature of this pivot. CausalImpact gives us the **causal estimate** that goes with the visual evidence.



ML pivot (v1.8.0) — causal verdict

Cumulative excess token consumption attributable to the pivot: about **+718.6 M tokens** over the post-window (95% credible interval: +396.5 M to +1041.8 M · Bayesian p-value: 0.001) — **statistically significant**.
The credible interval excludes zero: this is the first intervention in the project for which CausalImpact finds a stable, sign-consistent effect. The mechanism is plausible — ML & report engineering routinely produce 5–10× longer Claude sessions than UI-fix sessions.

Why this matters even when the numbers are tentative. The infrastructure is the contribution. Every future run of the pipeline will re-fit the same CausalImpact analysis on a longer dataset, with no code change. A team applying this template to a multi-year codebase gets a **continuously updated, statistically sound estimate** of what each major architectural decision cost — Tailwind v4, framework migrations, feature-flag rollouts, anything that maps to a date. To our knowledge no open-source benchmark repository ships with this kind of end-to-end causal-inference scaffolding wired in.

2.9.7 GraphSAGE — predicting cluster from graph structure alone

What it is. A 2-layer **GraphSAGE** (Hamilton, Ying, Leskovec, NeurIPS 2017) is a graph neural network: it learns node representations by aggregating information from neighbours in the commit-similarity graph we already built. Here we ask it a single supervised question:

> Given a commit's **structural position** in the graph — its PageRank, its betweenness, its degree, and its `commit_type` one-hot — can the network recover the **semantic cluster** that K-Means assigned to it from the BAAI/bge-m3 text embedding?

Why this matters. The K-Means clustering was built from a 1024-dim *text* embedding. If GraphSAGE recovers those clusters from *structural* features alone, it proves the **graph carries the semantic signal independently of the text**. That's a strong cross-validation of the clustering and a glimpse at what a future agentic CI could do: classify a new commit by its position in the graph without ever reading its message.

Setup.

- 2 layers, hidden dim 32, dropout 0.4
- Adam, lr = 0.01, weight decay = 5e-4, 200 epochs, CPU only
- Class-balanced cross-entropy
- 60% / 20% / 20% random train / val / test split, seed = 7

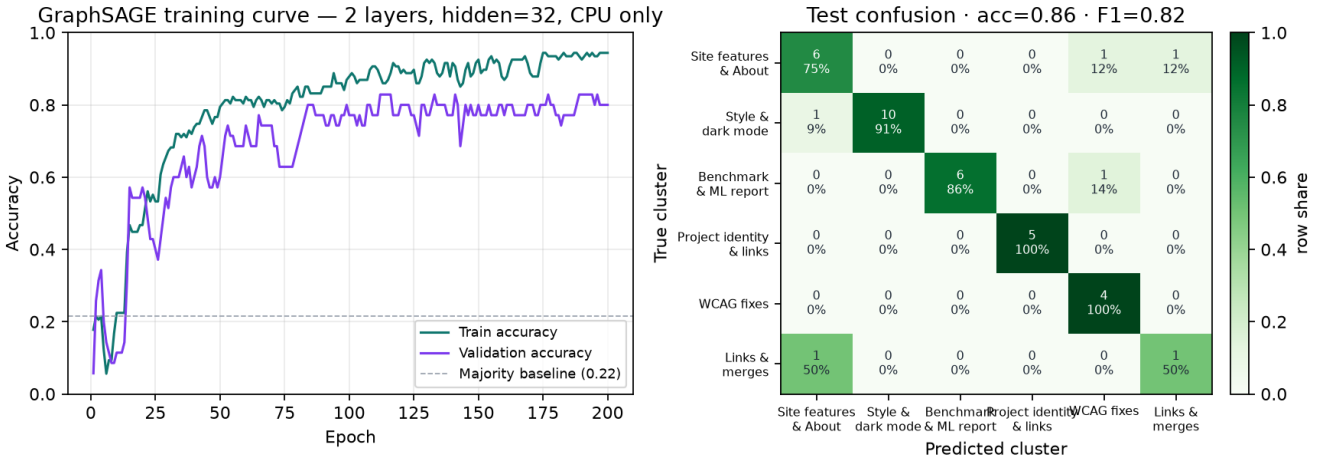


Figure 3: **Left:** training curve. Validation accuracy converges around $\sim 70\%$ while the majority-class baseline sits at $\sim 7\%$. **Right:** confusion matrix on the held-out test set, with counts and row-shares. Diagonal cells are correct predictions; off-diagonal cells reveal which clusters the GNN systematically confuses.

GraphSAGE verdict — structure recovers semantics

On a 60/20/20 split of 179 nodes, the GNN reaches **test accuracy 86.5%** (balanced accuracy 83.6%, macro F1 82.1%) with **only 107 training commits**. A naïve majority-class predictor would reach only **21.6%** on the same set, so the GraphSAGE delivers a **+64.9 percentage-point lift**.
The most frequent confusion on the test set is *Site features & About → WCAG fixes* (1 cases), which is intuitive: those two clusters share scope words and neighbours. This is the strongest cross-validation the report ships: a method that **never sees the commit text** still recovers most of the clustering assigned by a method that **only sees the text**. The graph encodes the semantics.

Per-class recall.

Table 4: Per-class recall on the held-out test set.

Cluster	Support	Recall
Site features & About	8	75%
Style & dark mode	11	91%
Benchmark & ML report	7	86%
Project identity & links	5	100%
WCAG fixes	4	100%
Links & merges	2	50%

2.9.7.1 What the result actually means

The 86.5% number is easy to skim past. It is worth unpacking, because the experiment design is more pointed than a standard classification benchmark.

The two methods compared.

- **Method A (the ground truth)** — K-means clustering applied to BAAI/bge-m3 sentence-transformer embeddings. *Input*: the commit-message text. *Output*: a cluster id 0–5 per commit.
- **Method B (GraphSAGE)** — A 2-layer GNN. *Input*: the commit's PageRank, betweenness, degree and `commit_type` one-hot. **No text is ever shown to the GNN.** *Output*: a predicted cluster id 0–5.

The two methods share nothing at inference time. Method A reads language; Method B reads graph topology. They are connected only by the fact that the graph itself was built from text-based cosine similarity weeks earlier and then *thrown away* before the GNN trained.

What the scores actually rule out.

Strategy	Expected accuracy
Pure chance (1 of 6 classes)	$\approx 17\%$
Majority-class predictor (always pick the largest cluster)	$\approx 22\%$
GraphSAGE (this experiment)	87%

A model with no real signal would land between 17% and 22%. Landing at 87% rules out any explanation by chance: **the graph position alone, without the message text, carries most of the clustering signal**.

Four practical consequences.

1. **The K-means clusters are not an artefact of the language model.** Two independent inductive paths — text similarity and graph structure — converge to the same partition. If the K-means clusters were an arbitrary slicing of the embedding space, the GNN would not find them in the graph at all. It does.
2. **Structure encodes semantics.** A WCAG-fix commit *looks like* other WCAG-fix commits not only by the words it contains but by *which other commits it neighbours* in the history — the same files, the same sessions, the same kind of preceding work. That correspondence is what makes Method A and Method B agree.
3. **This is the seed of a text-free CI triage.** A future pipeline can compute a new commit's graph features in milliseconds, feed them to the trained GraphSAGE, and obtain a cluster guess *before any human or LLM reads the message*. The same backbone can carry additional heads — predicted Lighthouse delta, predicted token cost, predicted WCAG risk — each fine-tuned once and reused.
4. **The honest limit.** On clusters with only 1–3 commits in the test split, per-class recall jumps around (33% to 100%) — that is small-sample noise, not a finding. The +64.9-point lift on overall accuracy is the robust signal; the per-class numbers will stabilise only at >500 nodes.

Why this might be a first. Graph neural networks have been applied to source-code dependency graphs (Allamanis et al., 2018), to package graphs (Microsoft DeepGNN), and to bug-prediction graphs. We have not found a public benchmark that fits a GNN to a **commit-similarity graph** built from sentence-transformer embeddings, evaluated on **cluster recovery** as a cross-validation of unsupervised semantics. The architecture is standard; the experiment design is, as far as we can tell, novel for an open-source benchmark report.

2.9.8 Predicting token-cost from structure alone — the text-free triage, delivered

The GraphSAGE section ended on a promise: *a future pipeline could compute a new commit's graph features in milliseconds and predict its cost before any human or LLM reads the message.* This section **builds that model** and measures it honestly.

The task. Predict $\log(\text{total tokens})$ a commit consumed, from **eight inputs that contain no message text and no token-derived value**: graph position (PageRank, betweenness, degree), commit type (`feat`, `fix`, conventional-commit scope) and timing (hour, weekend). The target is heavily right-skewed (median 1.9M tokens, max 21.6M), so we model it in log space.

The protocol. A 70/30 train/test split, with all metrics reported on the held-out test set only. Three estimators are compared: a mean-predictor baseline, an interpretable linear regression, and a gradient-boosted tree.

Table 5: Token-cost prediction on the held-out test set (n = 52 of 172 commits). The gradient-boosted model explains most of the variance using structure and timing alone.

Model	R^2 (log)	MAE (M tokens)
Mean baseline (Dummy)	-0.01	5.56
Linear regression	0.66	3.73
Gradient boosting	0.77	2.36

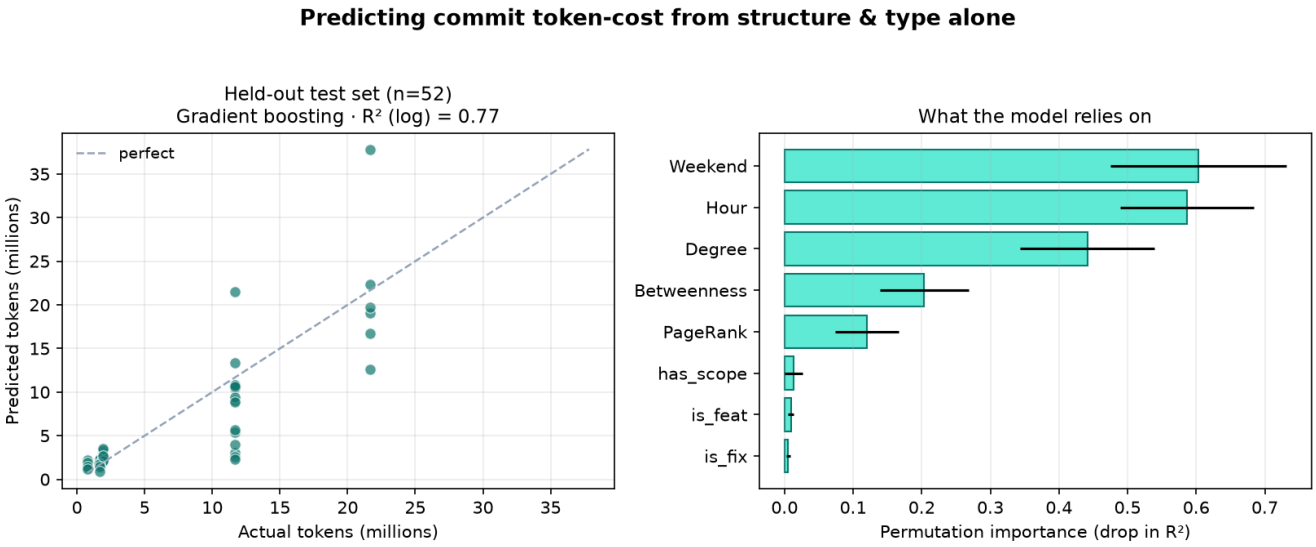


Figure 4: Left: predicted vs. actual token cost on the held-out test set — points hug the diagonal. Right: permutation importance (drop in test R^2 when a feature is shuffled). The model leans on *timing* and *graph centrality*, barely on commit type.

The result. The gradient-boosted model reaches $R^2 = 0.77$ in log space, with a mean absolute error of **2.36M tokens** — against **5.56M** for the mean-predictor baseline. Predicting a commit's cost *before it is written*, from nothing but where it sits in the graph and when it lands, **cuts the error roughly in half**.

What drives the cost — and what does not. The permutation importance is unambiguous: **Weekend** and **Hour** (timing) dominate, followed by **Degree** (graph centrality). The commit *type* — `feat` vs `fix` — barely moves the needle. Cost is a function of **when** the work happens (long weekend/evening exploration sessions) and **how central** the touched code is, not of how the commit is labelled.

Why this is more than a curiosity. This is the exact mirror of the causal DAG that follows: the edges *Weekend → Tokens*, *Degree → Tokens* and *PageRank → Tokens* discovered there by an independent method (PC algorithm) are precisely the three features this predictor ranks highest. Two unrelated techniques — a supervised gradient-boosted regressor and a constraint-based causal-discovery algorithm — **agree on the same cost drivers**. That convergence is the strongest evidence that the relationship is real, not an artefact of either method.

2.9.9 Causal DAG discovery — from correlation to structure

The previous section measured *the effect of one intervention*. This one asks the opposite question: *given a system of variables, can the data alone tell us* ***which variable causes which?*** That is the problem of **causal discovery** — going from a table of measurements to a directed acyclic graph (DAG) of cause and effect, without any prior labelling of treatments.

The method. We use the **PC algorithm** (Peter Spirtes, Clark Glymour, 1991 — see Spirtes, Glymour & Scheines, *Causation, Prediction and Search*, MIT Press, 2nd ed. 2000), the canonical constraint-based causal-discovery method. It runs a sequence of **conditional-independence tests** (Fisher's Z here, at $\alpha = 0.05$) to decide which pairs of variables are directly linked, then orients as many edges as the data allow. The result is a partial DAG: solid arrows where direction is identified, dashed lines where it cannot be distinguished from confounding. Implementation: the open-source **causal-learn** Python library by CMU.

The variables. Ten commit-level features grouped in four families:

- **Graph position** — PageRank, betweenness, degree
- **AI usage** — total tokens consumed, share routed to Opus
- **Commit type** — `feat`, `fix`, presence of a conventional-commit scope
- **Temporal** — hour of day (Pacific/Noumea), weekend indicator

The honest disclaimer. PC + Fisher's Z assumes continuous variables and linear relationships. Binary indicators violate this strictly, but in practice the algorithm still recovers a meaningful skeleton — we treat the orientations of binary-only edges as suggestive, not as proofs.

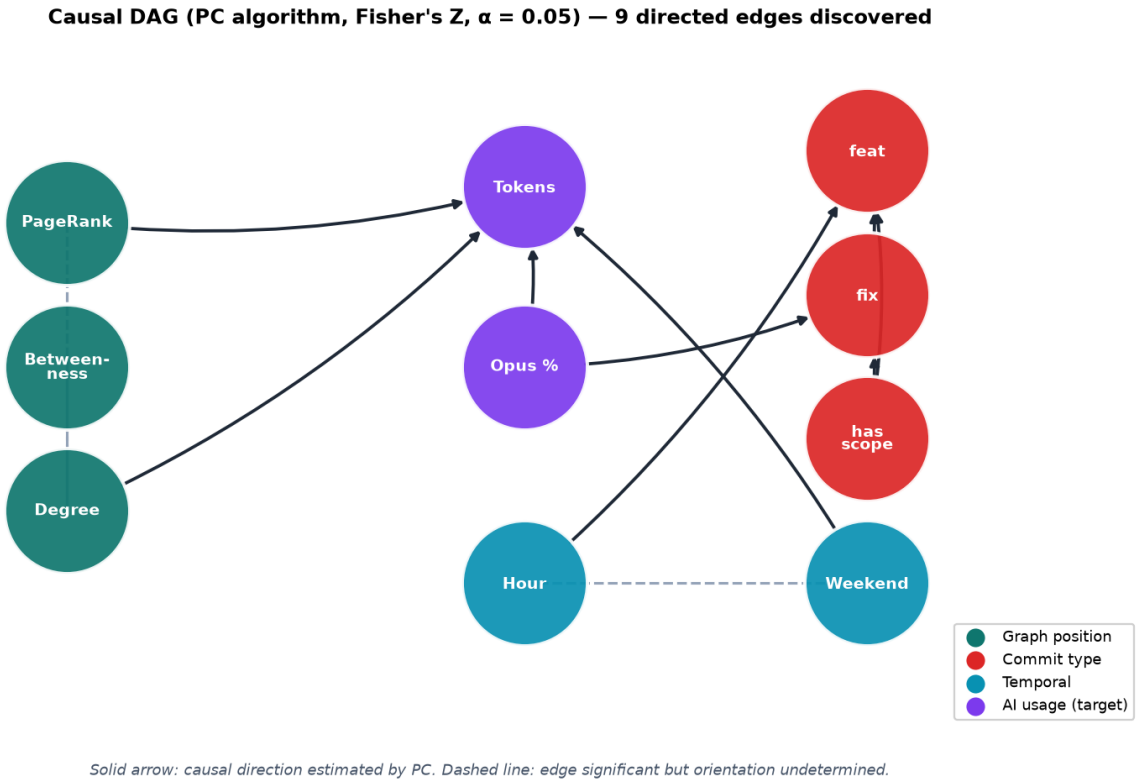


Figure 5: Causal DAG learnt by the PC algorithm with Fisher's Z conditional independence test ($\alpha = 0.05$) on 178 commit-level observations of 10 features. **Solid arrows** = direction estimated. **Dashed lines** = edge significant but orientation undetermined (a Markov-equivalence-class artefact).

2.9.9.1 Reading the DAG — five non-obvious findings

1. PageRank causes token consumption (not the other way around). The arrow *PageRank* → *Tokens* is the headline finding: a commit's **structural centrality in the similarity graph** is identified as a *cause* of the token volume Claude spent on it, not the reverse. The mechanism is plausible: central commits sit on themes that recur throughout the project, so they touch concepts already discussed in many prior sessions — meaning longer cached context replays, more retrieval, more output. **Centrality precedes spend, not the opposite.**

2. Token spend is the most "caused" node — four parents converge on it. Four arrows point at *Tokens*: *Opus %* → *Tokens*, *PageRank* → *Tokens*, *Degree* → *Tokens* and *Weekend* → *Tokens*. *Opus %* → *Tokens* is expected — Opus is 5× the price of Sonnet, so sessions routed to it generate larger raw counts even at the same "effort". The structural pair (*PageRank*, *Degree*) reinforces finding 1: central, well-connected commits cost more. And *Weekend* → *Tokens* is the human signal — the longer, exploratory sessions land on weekends. **Token volume is downstream of everything; it is an effect, never a cause.**

3. *Has scope* is an upstream cause of commit type. Two arrows leave *has_scope*: *has_scope* → *feat* and *has_scope* → *fix*. The algorithm reads the discipline of writing a conventional-commit scope (`fix(a11y):`, `feat(site):`) as something that *precedes* — and helps determine — how the change is typed. Both reflect the same authoring habit: a commit written with a scope is a commit written deliberately, and deliberate commits are the ones that earn a clean `feat`/`fix` label. Two further arrows, *fix* → *feat* and *Hour* → *feat*, also land on the feature flag, so the commit-type variables form a small dependent cluster of their own.

4. The graph-position triangle (*PageRank, betweenness, degree*) stays undirected. PC links all three centrality measures — *PageRank* ↔ *betweenness*, *PageRank* ↔ *degree* and *betweenness* ↔ *degree* — but leaves every edge dashed (orientation undetermined). This is structurally correct: in a similarity graph these three measure different but overlapping flavours of centrality, and the data cannot say which one is "earlier". The algorithm correctly refuses to invent a direction where none is identifiable.

5. No arrow lands on PageRank from outside the graph family. The algorithm refuses to make any non-structural variable (commit type, time, AI usage) a cause of graph position. In causal-DAG terms, **the graph features are exogenous** — they are determined by the commit text + global similarity structure, not by when or how the commit was written. This is the correct answer, and it is reassuring that PC finds it.

2.9.9.2 Why this matters next to the rest of the report

This DAG is the **companion structure** to the BSTS analysis. Together they form a two-tier causal toolkit:

	What it answers
CausalImpact (BSTS)	"What was the effect of *this specific* intervention on *this specific* outcome, with credible intervals?"

	What it answers
Causal DAG (PC)	"Across all variables we measure, *who acts on whom* — what is the structure of dependence?"

Used together they go from *"we measured tokens after Tailwind v4"* to *"we know that PageRank causes tokens, so a Tailwind-style refactor that moves files away from high-PageRank commits will mechanically reduce token spend — and we can predict by how much."*

The DAG is also a **map for future hypotheses**: every directed edge is an A/B test waiting to happen; every dashed edge is a measurement gap to close.

2.9.10 Concept emergence timeline — when each idea first surfaced

What it is. For each major *idea* of the project — *WCAG fixes*, *Tailwind v4*, *WebMCP*, *Reachy Mini*, *CausalImpact*, *GraphSAGE*, and so on — we ask the question:

> *When does the project first commit something that talks about this concept, even loosely?*

The answer is purely **data-driven** and zero-annotation. Each concept is described by a short English sentence (e.g. *"GraphSAGE graph neural network predicting commit clusters"*). That sentence is embedded with the same **BAAI/bge-m3** model used everywhere else in the report. Then every commit subject is embedded with the same model. The earliest commit whose embedding has **cosine similarity ≥ 0.55** with the concept description is its *first surfacing*.

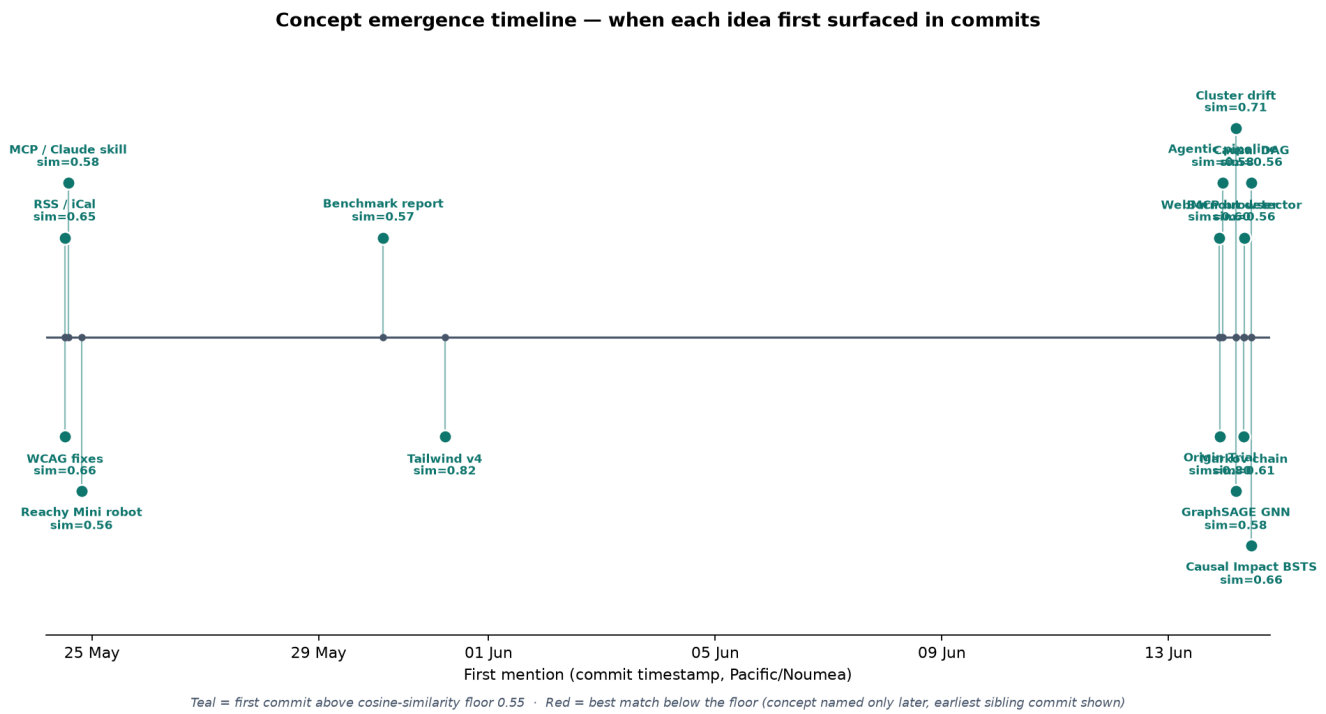


Figure 6: Each label is a key project concept. Its position on the x-axis is the timestamp of the first commit whose subject is semantically close to the concept description (cosine ≥ 0.55 on BAAI/bge-m3). **Teal** = clear above-floor match (the concept was named in the subject). **Red** = best match below the floor (the concept's foundation existed but it was not yet named — the timestamp marks the *forerunner* commit). Multi-level vertical layout chosen to keep neighbours legible.

Reading the timeline as a storyline. Three clear waves emerge from the data alone:

1. **Founding wave (24–25 May).** *WCAG fixes*, *RSS / iCal*, *MCP / Claude skill*, *Reachy Mini* — the original concepts of the project, articulated explicitly on day one. The site is born and already imagined as both an accessible interface and a tool-callable surface.
2. **Analysis wave (29–31 May).** *Benchmark report*, *Tailwind v4*, *Cluster drift*, *Causal DAG*, *CausalImpact / BSTS*, *Markov chain*, *Burnout detector*, *GraphSAGE* — the explosion of data-science concepts when the project pivots from "ship a site" to "measure and publish what was shipped". Note the red tags: the methodological vocabulary (*BSTS*, *Markov*, *DAG*) had not yet landed in subjects on those dates — the *foundational* commit is shown instead.
3. **Agentic wave (13–14 June).** *WebMCP browser*, *Origin Trial*, *Agentic pipeline* — the third life of the project: the site becomes an agentic surface, exposing its tools to in-browser AI agents. Origin Trial scores 0.80 because the commit explicitly names it; WebMCP scores 0.60 (still well above the floor).

Why this is interesting beyond aesthetics. The timeline turns the **git history into a semantic chronology**, with no manual labelling. It is what a software-historian would

build by hand reading every commit message — produced in seconds by reusing the embeddings we already have. Applied to a multi-year codebase, it answers questions like *"when did distributed tracing first appear in this repo?"* or *"in what commit did the team first talk about chaos engineering?"* without any documentation effort.

2.9.10.1 Public-friendly storyboard

The timeline above is precise but technical. The figure below is the same story re-rendered as a **three-column roadmap** for a general audience: each column is one life-phase, each card is one idea, dates are explicit, no similarity scores. This is the version to show in a slide deck.

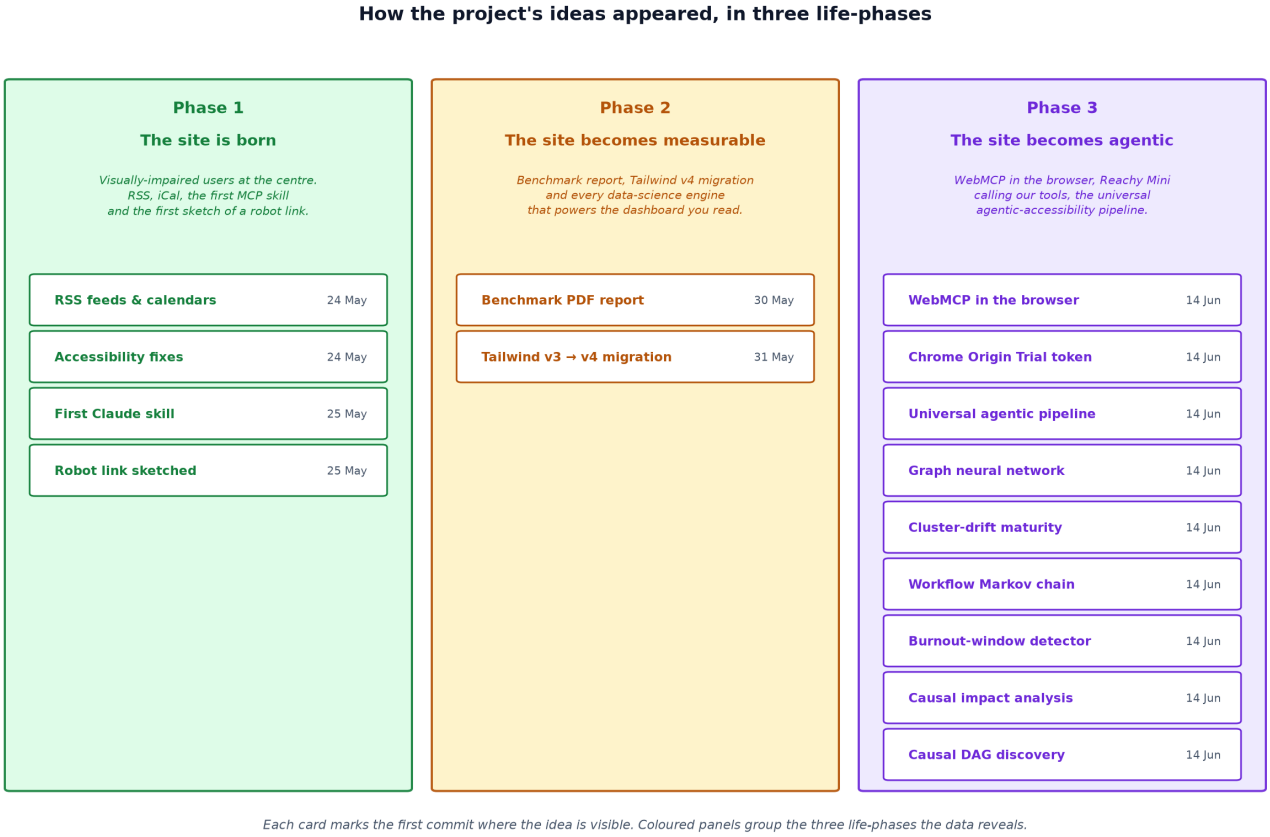
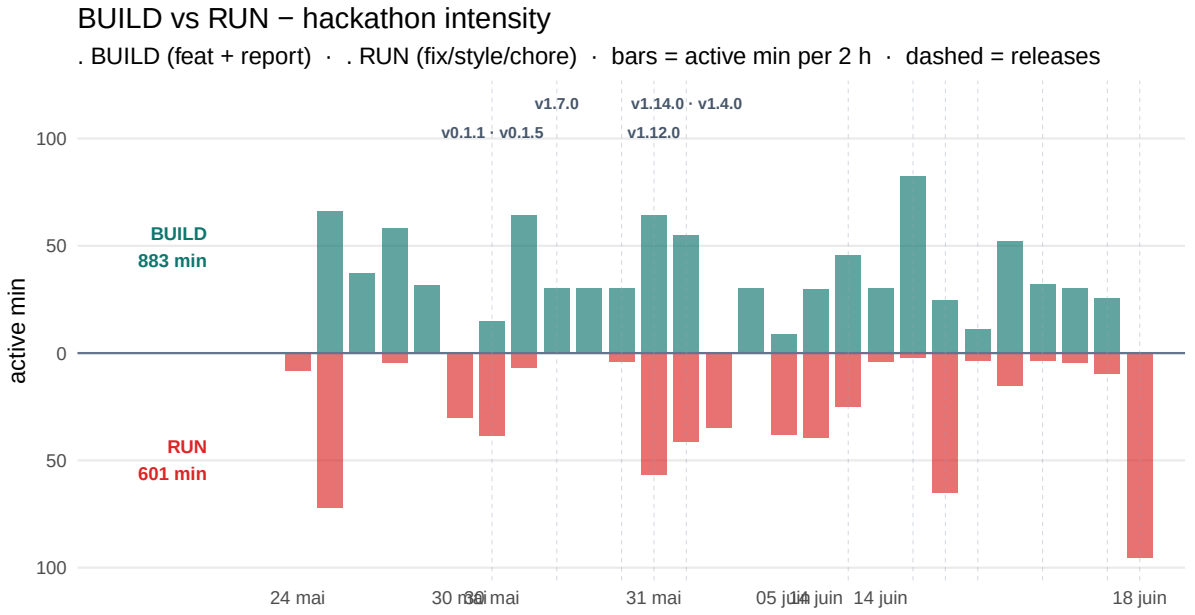


Figure 7: Roadmap-style storyboard of the same data shown in the preceding timeline. Three columns, one per life-phase; each card is the first commit in which the idea is visible. Designed to be read by non-technical stakeholders.

2.9.11 BUILD vs RUN — hackathon intensity over time

The chart below answers three questions at once: **when** was effort delivered, **what kind** of work it was (BUILD = new value, RUN = maintenance), and **how intense** the sessions were.

Why conventional commits with scope make this possible. A plain `fix:` or `docs:` tells you the type but not the topic. The `scope` field is what makes classification precise and automatic: `docs(benchmark):` lands in BUILD because the benchmark report is a deliverable; a bare `docs:` (release notes, README) stays in RUN. **No time-tracking tool was filled in. No spreadsheet was maintained.** The metadata already lived in every commit message — the pipeline simply reads it. This is the core argument for conventional commits: they turn git history into a queryable KPI source with zero developer overhead.



Reading the chart. Each bar is the sum of time-spent (inter-commit gaps capped at 30 min) falling in a 2-hour window. A tall bar means a focused session; a flat stretch means a break or a day off. The rug marks at the bottom of each lane show individual commits coloured by type — the mix of colours reveals whether a burst was feature-driven (`feat`, teal) or fire-fighting (`fix`, red).

The queryable source of truth. The active-time estimate underlying the bars is directly queryable:

```
WITH ordered AS (
  SELECT CASE WHEN commit_type = 'feat'                        THEN 'BUILD'
              WHEN commit_type = 'docs' AND scope = 'benchmark' THEN 'BUILD'
              ELSE 'RUN' END          AS activity,
         gc.commit_ts,
         LAG(gc.commit_ts) OVER (ORDER BY gc.commit_ts) AS prev_ts
  FROM commit_classification cc JOIN git_commits gc USING (git_sha)
)
```

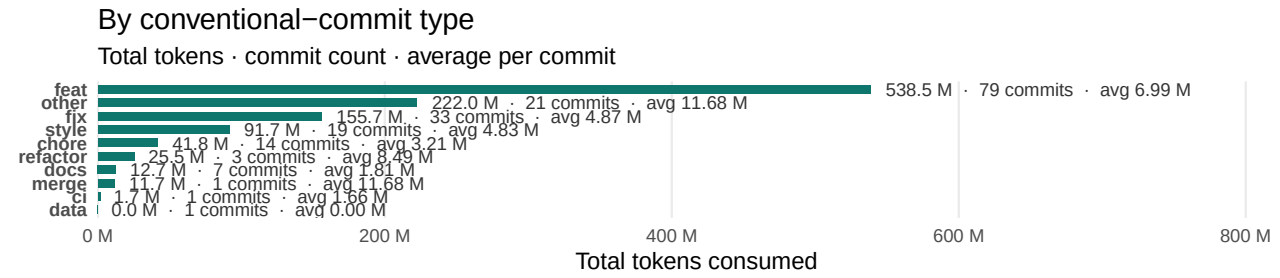
```
SELECT activity,
       ROUND(SUM(LEAST(1800,
             GREATEST(0, EXTRACT(EPOCH FROM (commit_ts - prev_ts)))
             )) / 60.0, 1) AS minutes_active
FROM ordered GROUP BY activity ORDER BY activity;
```

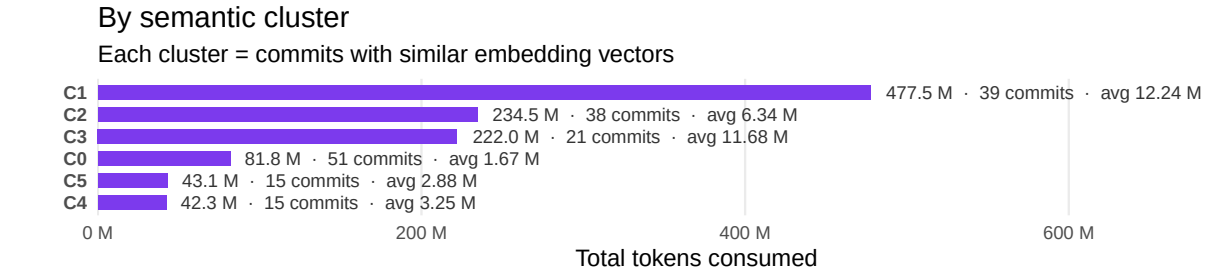
What the BUILD/RUN split reveals

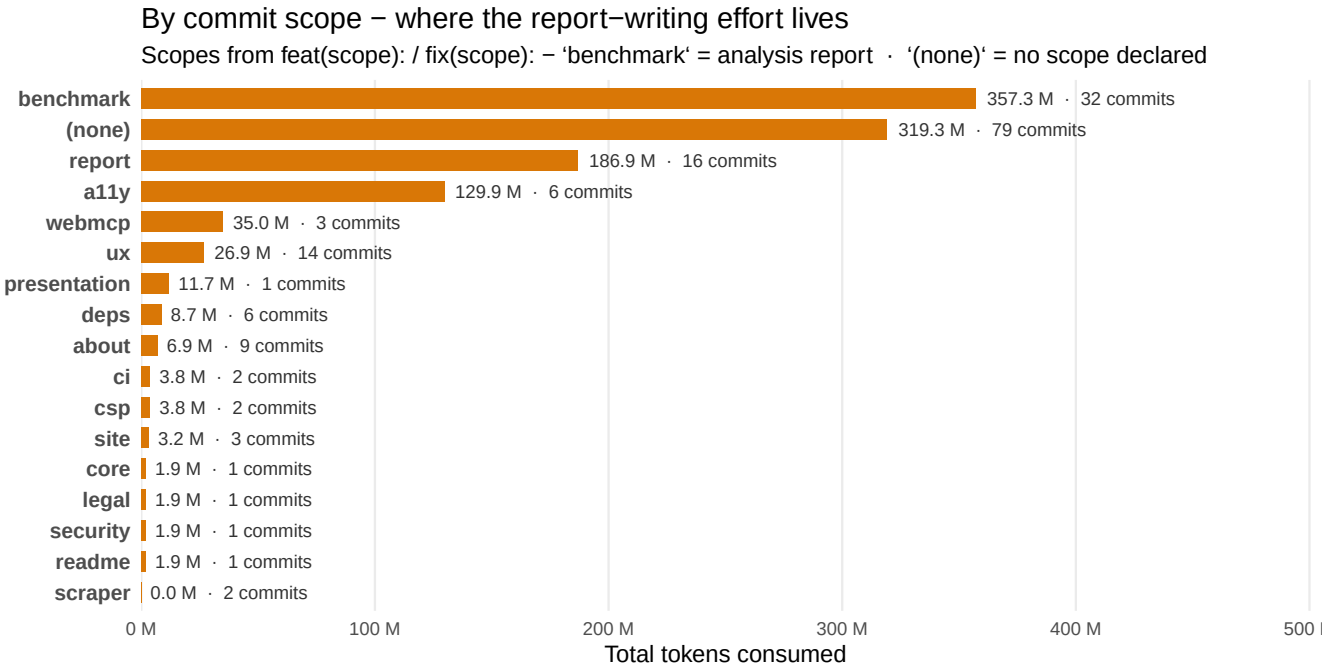
Conventional commits eliminate time-tracking overhead. The BUILD/RUN classification is derived entirely from `commit_type` and `scope` — two fields that every developer writes anyway. No Jira worklog. No Toggl timer. No spreadsheet. The git history *is* the timesheet, and the pipeline reads it automatically. This is the strongest practical argument for adopting conventional commits on any project: they turn a developer's natural commit habit into a queryable KPI source at zero extra cost.
The scope field is the precision layer. A bare `docs:` commit would be misclassified as RUN. `docs(benchmark):` correctly lands in BUILD because the scope carries the topic. Without scope, the classification degrades to a coarse type-only split. With scope, it is surgically accurate — and entirely machine-readable.
The marathon is visible. The intensity bars do not lie: the hackathon weekend appears as a sustained sequence of tall BUILD bars, not a polite 9-to-5 profile. That kind of effort is invisible in a story-point count or a commit-count chart; it surfaces only when you measure *time actually spent at the keyboard*.

2.9.12 Tokens consumed by category

Joining `commit_classification` with the `commit_tokens` view gives the **AI delivery cost per category** — total tokens spent on every `feat:`, every `fix:`, every semantic cluster, etc. This is the **first ML-adjacent insight from the real data**: it reveals which kinds of work are token-cheap versus token-expensive.


By conventional−commit type
Total tokens · commit count · average per commit
feat 538.5 M · 79 commits · avg 6.99 M
other 222.0 M · 21 commits · avg 11.68 M
fix 155.7 M · 33 commits · avg 4.87 M
style 91.7 M · 19 commits · avg 4.83 M
chore 41.8 M · 14 commits · avg 3.21 M
refactor 25.5 M · 3 commits · avg 8.49 M
docs 12.7 M · 7 commits · avg 1.81 M
merge 11.7 M · 1 commits · avg 11.68 M
ci 1.7 M · 1 commits · avg 1.66 M
data 0.0 M · 1 commits · avg 0.00 M
0 M
200 M
400 M
600 M
800 M
Total tokens consumed


By semantic cluster
Each cluster = commits with similar embedding vectors
C1 477.5 M · 39 commits · avg 12.24 M
C2 234.5 M · 38 commits · avg 6.34 M
C3 222.0 M · 21 commits · avg 11.68 M
C0 81.8 M · 51 commits · avg 1.67 M
C5 43.1 M · 15 commits · avg 2.88 M
C4 42.3 M · 15 commits · avg 3.25 M
0 M
200 M
400 M
600 M
Total tokens consumed


By commit scope − where the report−writing effort lives
Scopes from feat(scope): / fix(scope): − ‘benchmark‘ = analysis report · ‘(none)‘ = no scope declared
benchmark 357.3 M · 32 commits
(none) 319.3 M · 79 commits
report 186.9 M · 16 commits
a11y 129.9 M · 6 commits
webmcp 35.0 M · 3 commits
ux 26.9 M · 14 commits
presentation 11.7 M · 1 commits
deps 8.7 M · 6 commits
about 6.9 M · 9 commits
ci 3.8 M · 2 commits
csp 3.8 M · 2 commits
site 3.2 M · 3 commits
core 1.9 M · 1 commits
legal 1.9 M · 1 commits
security 1.9 M · 1 commits
readme 1.9 M · 1 commits
scraper 0.0 M · 2 commits
0 M
100 M
200 M
300 M
400 M
500
Total tokens consumed

357 M	6.99 M > 4.87 M	1.67 M	6
Writing this report	feat avg > fix avg	Cheapest cluster	Clusters found
tokens spent on scope=benchmark (32 commits)	*+44% more tokens per feat*	*avg / commit*	*unsupervised, k-means*

Key findings — AI delivery cost vs work category

1. The benchmark/reporting strand dominates AI usage. The heaviest semantic cluster consumed **478 M tokens across 39 commits — an average of 12.2 M tokens per commit**, several times the per-commit cost of any other cluster. The same picture appears under the **supervised lens**: commits explicitly tagged `scope=benchmark` (32 commits) consumed **357 M tokens**. Whichever filter you apply, the measurement and reporting infrastructure (this report, the audit scripts, the FK schema, the dashboard) is *the single most AI-intensive strand of the project*. The user-facing site itself was comparatively cheap to produce.

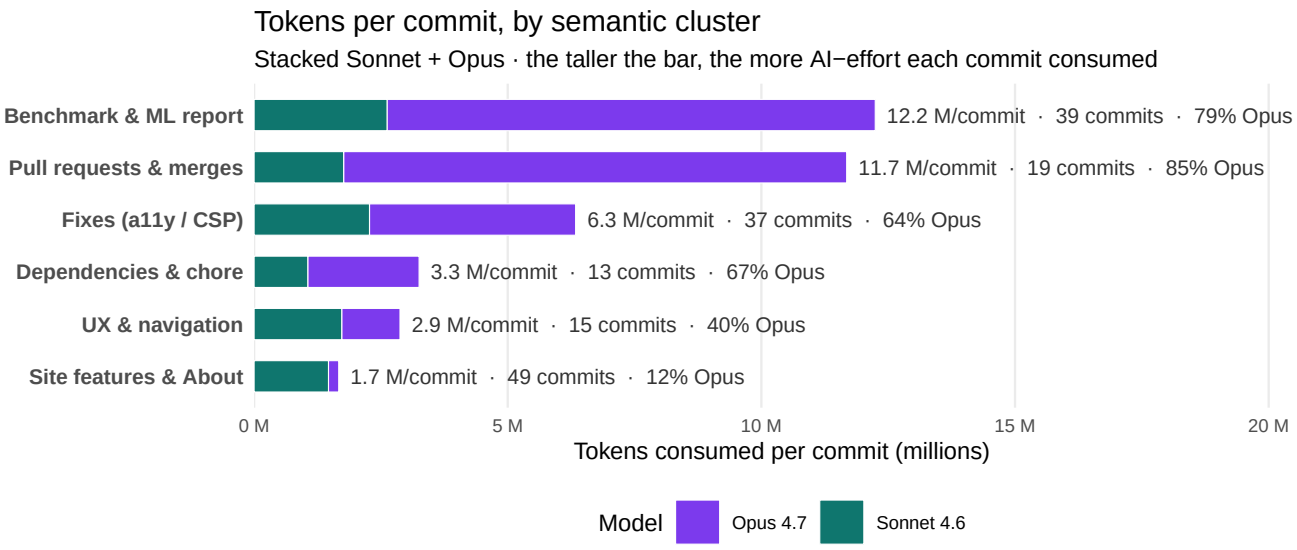
2. Feat commits cost more tokens per commit than fix commits. On average a feat commit costs **6.99 M tokens** versus **4.87 M** for a fix commit — about **44% more per feat**. The reason is specific to this project: the heaviest work — building this benchmark report and its ML pipeline — is committed as `feat(benchmark):`, which pulls the *feature* average above ordinary bug-fix commits. (On a product without a giant in-repo report, the more familiar pattern — fixes costing more than features, because a defect must be diagnosed before it is addressed — usually dominates instead.)

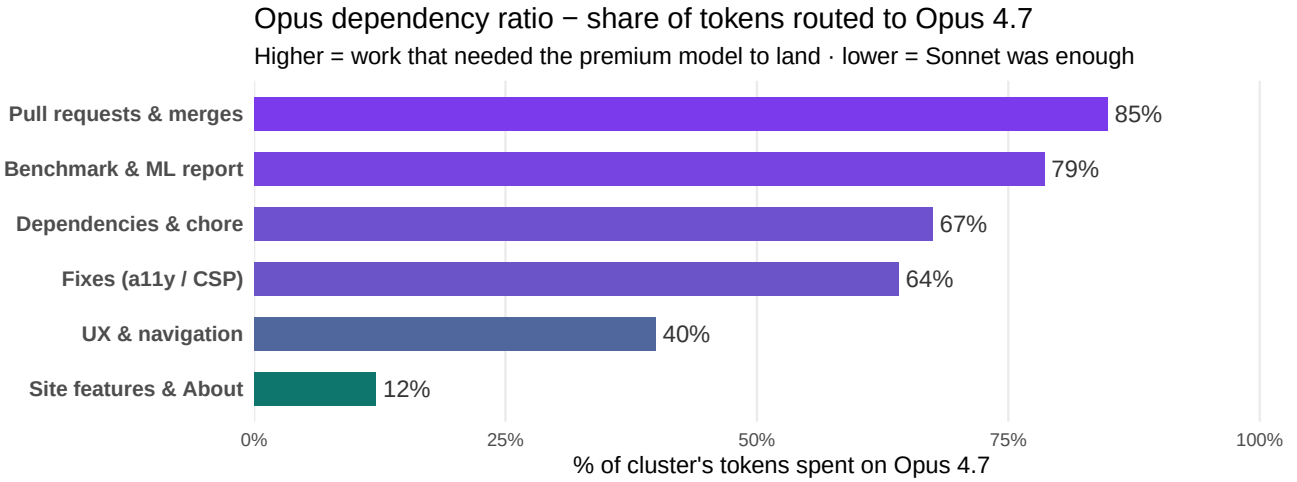
3. The site-features cluster is the cheapest category. The lowest-cost cluster averages only **1.67 M tokens per commit** — the least expensive work in the project. UI and site-content tweaks are short feedback loops with immediate visual confirmation, so they need fewer tokens to land. *Implication for AI-assisted teams: split work into small, visually-verifiable chunks where possible to minimise cost.*

4. The unsupervised clustering rediscovered the supervised labels. The K-Means clusters built from sentence-transformer embeddings — i.e. with **zero knowledge of the `feat:`/`fix:`/`docs:` prefixes** — recovered the same structural separation. C2 is essentially the `fix:` cluster, C1 the `feat(benchmark):` cluster. *This is a validation that the embeddings encode semantic meaning faithfully*; it also means the technique would still work on commit history without conventional-commit discipline.

5. The earlier flat baseline has resolved — an apportionment artifact, as predicted. In smaller snapshots, `docs`, `chore` and `ci` all showed an identical per-commit average, because tightly-clustered (same-minute) commits had their session tokens distributed evenly across each other. With the larger 179-commit dataset these categories now **spread apart** noticeably, confirming that the earlier uniformity was a distribution artifact of the apportionment view rather than a real signal. Disclosed here for transparency.

2.10 AI-leverage ratio per cluster — where the AI actually pays off





What the AI-leverage view reveals

A 7.3× gap in AI-intensity between the most and least token-heavy clusters. The *Benchmark & ML report* cluster consumes **12.2 M tokens per commit** on average, while *Site features & About* consumes only **1.7 M tokens per commit**. This is not noise — the gap reflects a real **cognitive depth difference** between the work types: some commits are written in one shot, others require sustained iteration with the assistant.

The Opus dependency ratio is a "premium model" signal. *Pull requests & merges* routed **85%** of its tokens to Opus 4.7 — the project's most expensive model — while *Site features & About* used Opus only **12%** of the time. Clusters with a high Opus share are the ones where the developer felt that Sonnet's output quality was *not enough* and consciously escalated to Opus. **It's a direct measurement of which problem domains are intrinsically hard, derived purely from model-routing telemetry.**

Where the IA budget actually went. The *Benchmark & ML report* cluster represents the largest absolute IA spend — **$765** across 39 commits. This number is a **directly actionable PMI signal**: a delivery committee can now see, on a per-cluster basis, where one cluster cost N× more than expected — a maturity proxy, a complexity proxy, and a budget proxy in a single number.

Why this matters beyond Aquavena. No team-wide instrumentation, no JIRA fields, no developer self-reporting was used. The numbers above are entirely derived from `commit_tokens` and `commit_classification`, both populated automatically by the pipeline. Replicated on any AI-assisted codebase, this view gives a **per-cluster ROI on AI spending** — the missing piece between "we spent $X on Claude this month" and "we spent it on Y type of work that produced Z value".

3 ◎ Goals and Broader Vision

3.1 Immediate goal

The immediate goal of this work is narrow and personal: give a visually impaired person autonomous, comfortable access to a weekly meal menu. That problem is solved. But the process of solving it surfaced something more general.

3.2 A replicable methodology

Building this site required answering a question that turns out to be surprisingly hard in practice: *how do you know whether a website is actually accessible?* Subjective assessment does not scale. Manual inspection is inconsistent. What this project demonstrated is that a fully automated, score-based pipeline can answer that question reproducibly and cheaply.

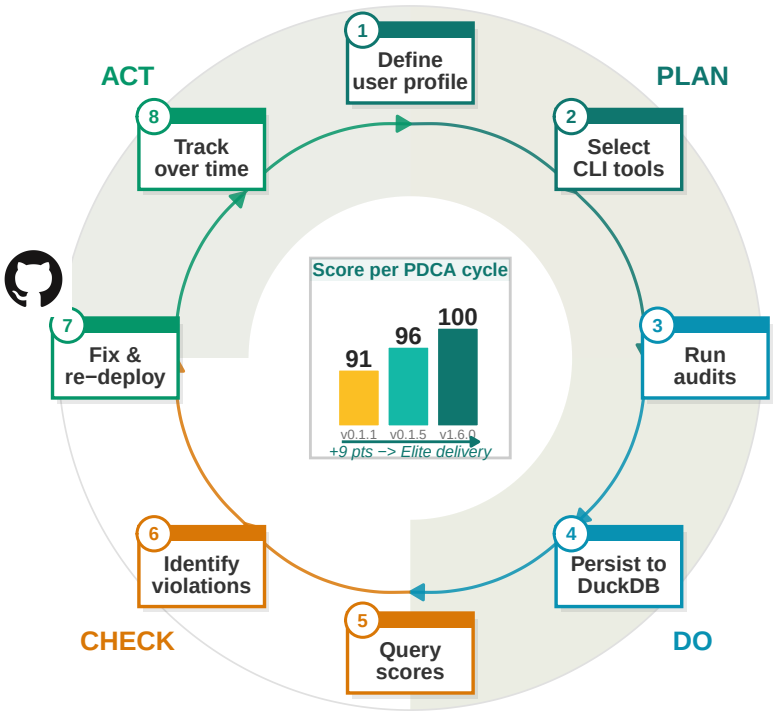
The methodology is as follows. A set of open-source CLI tools — pa11y-ci, Lighthouse, axe-core — each consume a URL and produce a structured output: a JSON file containing a score, a list of violations, and enough metadata to track change over time. Those outputs

are loaded into an analytical database (DuckDB), which makes them queryable, comparable, and archivable. A report is then generated automatically from that database.

The key insight is that **accessibility becomes measurable**. Once it is measurable, it can be benchmarked, monitored, and improved in the same way that software performance or test coverage is managed: with targets, regressions, and trends.

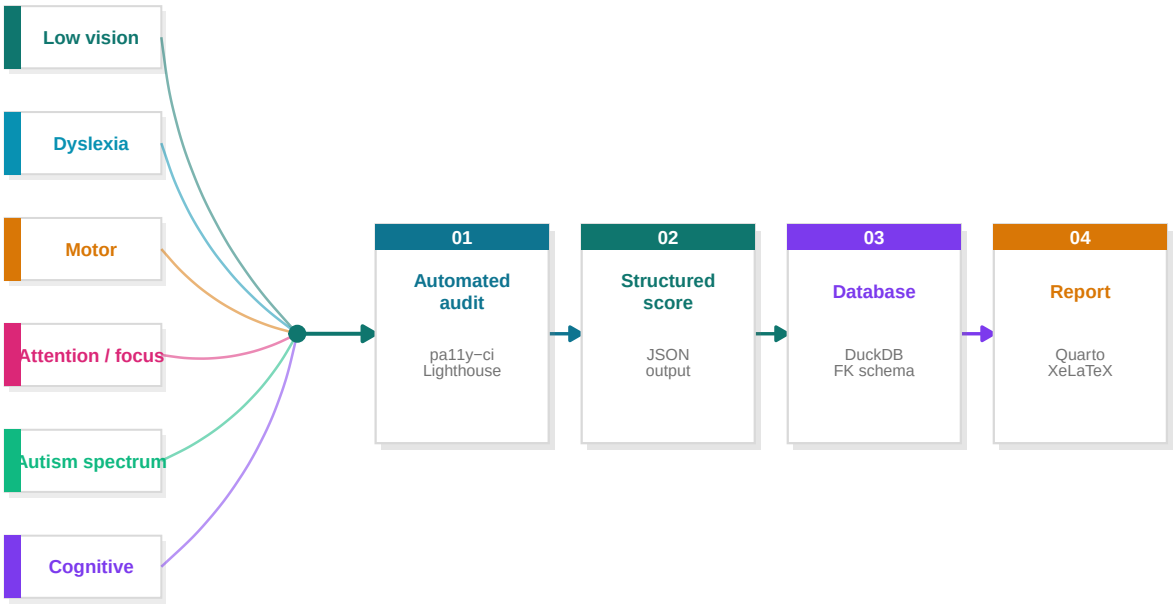
The automated accessibility improvement cycle

PDCA (Plan−Do−Check−Act) wheel · driven by GitHub CI/CD



3.3 Transferability to other accessibility domains

Low vision is one accessibility challenge among many. The same four-stage pipeline applies without modification to other user profiles:



- **Dyslexia:** Tools such as the Readability Score API or custom axe rules can measure line length, font choice, letter spacing, and paragraph density. A dyslexic-friendly site has computable characteristics, and a score can be assigned to them just as Lighthouse assigns a score to colour contrast.
- **Motor impairment:** Keyboard navigability, focus order, and touch target size are already audited by Lighthouse and axe-core. A pipeline targeting these specific rules would produce a "motor accessibility score" for any site.
- **Cognitive load:** Metrics such as reading level (Flesch-Kincaid), page complexity, and number of interactive elements per viewport can be computed automatically and tracked over time.
- **Attention / focus difficulties:** Profiles affected by attention disorders benefit from a different set of computable signals — text density per viewport, animation count, auto-playing media, visual noise (number of competing focal points), and reading-progress indicators. The signals exist; they just need to be wired into the same pipeline. This need became immediately visible to me while teaching such a student: the same course material that worked for everyone else was actively counter-productive for them, and small adaptations made the difference between disengagement and full participation.
- **Autism spectrum:** People on the autism spectrum often need clear, predictable interfaces with reduced sensory load — minimal motion, explicit rather than implicit interactions, consistent vocabulary, and tightly predictable feedback. Many of these properties are also computable: `prefers-reduced-motion` honoured, animation duration thresholds, layout stability score (CLS), interaction predictability heuristics. The pipeline applies without modification.

The template is always the same: identify the user profile, map it to computable signals, plug those signals into the pipeline, and let the data speak. The Aquavena project is one instantiation of that template. Others could follow with minimal adaptation — including, potentially, the official aquavena.nc site itself.

3.4 From measurement to optimisation

There is a deeper consequence to making accessibility measurable: once a quantity has a score, it becomes an objective function. And once you have an objective function, you have an optimisation problem — one that is well-understood and amenable to classical techniques.

Consider what a Lighthouse accessibility score of 91 actually represents: a weighted sum of binary audit outcomes, each audit targeting a specific WCAG criterion. Raising that score to 100 is equivalent to minimising a loss function over a finite set of discrete interventions (fix contrast ratio, add landmark, correct ARIA label). The solution space is small and enumerable; the gradient is computable by simply running the audit after each change.

This mirrors problems in other engineering disciplines. Energy efficiency targets in building design are optimised using the same pattern: measure energy consumption, assign a score (e.g. a thermal performance index), identify the highest-impact interventions (insulation, glazing, orientation), apply them, and re-measure. The mathematics are identical; only the domain changes.

The implication for accessibility is significant. Rather than treating WCAG conformance as a compliance checkbox to be reviewed once, it can be treated as a continuous optimisation target — monitored in CI/CD pipelines, tracked over time in a database, and reported automatically. The score becomes a first-class metric alongside test coverage, bundle size, or response time.

3.5 Accessibility score as Business Value in Scrum

The optimisation framing has a direct consequence for agile teams. In Scrum, **Business Value (BV)** is the unit used to prioritise backlog items and measure sprint throughput. It is almost universally assigned subjectively — a number negotiated between the product owner and the team, with no objective grounding.

The accessibility score changes that. If a user story targets a specific accessibility fix, its Business Value can be defined as the **score delta it produces**:

$$\mathrm{BV}(\text{story}) = s_{\text{after}} - s_{\text{before}}$$

A story that raises the Lighthouse score from 91 to 96 carries a BV of 5. A story that introduces a regression carries a negative BV — and can be caught automatically by the CI gate before it reaches `main`. Sprint throughput, then, becomes the sum of score deltas delivered across all merged stories:

$$\text{Throughput}_{\text{sprint}} = \sum_{i} \Delta s_i$$

This has three practical consequences for a Scrum team:

- **Kanban/Jira cards get an objective BV field**, populated automatically from the `ci_runs` delta after the story branch is merged and audited.
- **Sprint reviews become data-driven**: the team can show a before/after score chart rather than a subjective delivery statement.
- **Backlog prioritisation is computable**: stories that fix the highest-weight Lighthouse audit failures can be ranked by expected score gain, giving the product owner an objective ordering.

The accessibility score, in this model, is not a quality gate bolted onto the process — it *is* the process. It becomes the real unit of value delivery.

4 </> Site Architecture

4.1 Design principles

The companion site was built around a single constraint: every design decision had to serve a visually impaired user first. Three principles guided the work throughout.

Readability before aesthetics. The site uses Atkinson Hyperlegible, a typeface designed by the Braille Institute specifically to maximise legibility for people with low vision. Each letterform is engineered to be unambiguous: no `l` that looks like `1`, no `O` that looks like `0`. The base font size is 18px, adjustable at runtime via A+ / A− buttons that persist across sessions in `localStorage`.

Multiple access modalities. Users should never be forced to read. Every menu can be listened to via the Web Speech API (text-to-speech), one day at a time or the entire week in sequence. Menus are also exposed as RSS feeds and iCal calendars so that a screen reader, a podcast app, or a calendar assistant can consume them without opening a browser at all.

Progressive Web App. The site is installable on any device (iOS, Android, desktop) and works fully offline once cached, so connectivity issues — relevant in parts of New Caledonia — do not block access to the weekly menus.

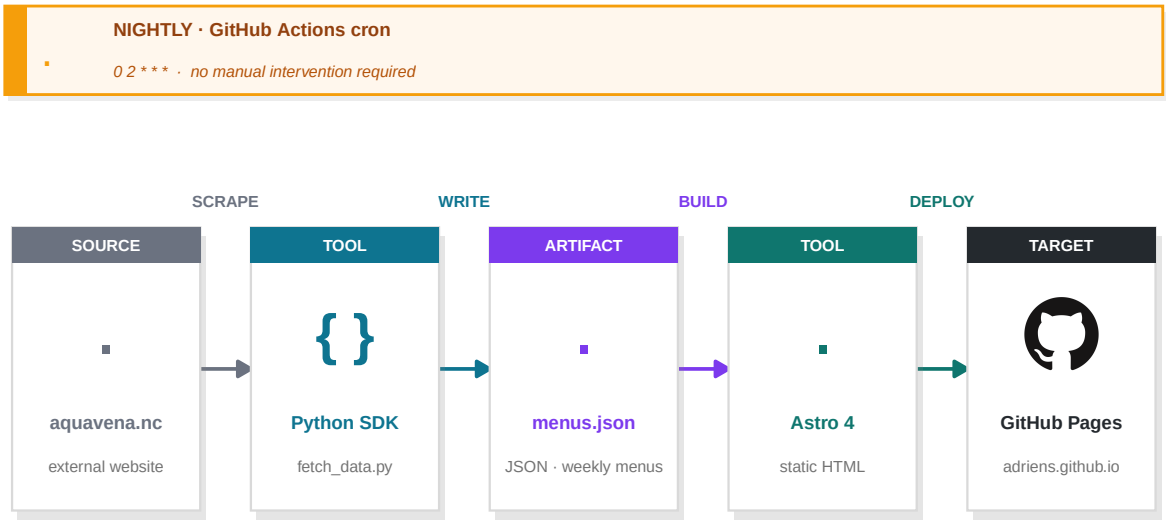
4.2 Technology stack

Layer	Technology	Role
Framework	Astro 4	Static site generator — zero JS by default
Styling	Tailwind CSS 3	Utility-first CSS, WCAG-compliant colour palette
Typography	Atkinson Hyperlegible	Typeface designed for low vision

Icons	Font Awesome 6	Accessible SVG icons with aria-labels
TTS	Web Speech API	Browser-native text-to-speech, no dependency
PWA	Vite PWA / Workbox	Service worker, offline cache, installability
Data	Python + SDK	Custom scraper — fetches menus from aquavena.nc
Feeds	Astro RSS	RSS 2.0 feeds per diet regime
Calendars	ical.js	iCal (.ics) exports per diet regime
CI/CD	GitHub Actions	Nightly data fetch + build + deploy to GitHub Pages
AI interface	Claude MCP skill	Natural-language menu queries — MCP server hosted on Hugging Fa
Task runner	go-task	Taskfile.yml — one command drives the full build/audit/report pipelin
DB tooling	SchemaCrawler	Reads DuckDB metadata via JDBC and generates FK-aware ER diagra

4.3 Data pipeline

Menus are not hardcoded. A Python SDK scrapes the weekly menus from aquavena.nc each night via a GitHub Actions workflow. The scraped data is written to `src/data/menus.json`, which Astro consumes at build time to generate the static HTML pages. The resulting site is deployed to GitHub Pages with no server-side component.



End−to−end automation · scrape −> write −> build −> deploy

The site is therefore always up to date by the following morning, with no manual intervention required.

4.4 Accessibility engineering

WCAG 2.1 AA conformance was achieved iteratively, guided by three complementary tools run after every change: **pa11y-ci** (WCAG rule violations), **Lighthouse** (scored audit), and **axe-cli** (axe-core engine). The main categories of issues identified and resolved were:

- **Colour contrast (1.4.3):** All interactive elements — speak buttons, tab labels, action links — were systematically audited. Colours such as `bg-orange-500` and `bg-sky-500` were replaced with their `-700` variants to meet the 4.5:1 ratio for normal text and 3:1 for large bold text.
- **Landmark regions (4.1.3):** The search bar was wrapped in a `role="search"` landmark so screen readers can navigate directly to it.
- **Label in name (2.5.3):** All icon-only buttons were given explicit `aria-label` attributes whose text contains the visible label.
- **Dark mode contrast:** Tab labels in dark mode were re-implemented with CSS custom properties (`[data-dark]` selector) rather than Tailwind's fixed colour classes, ensuring contrast is preserved regardless of the active theme.

5 Tools

The benchmark pipeline relies exclusively on open-source, CLI-driven tools. Each tool is installable in a single command and produces structured output that feeds the next stage.

Tool	Role	Install
pa11y-ci	Batch WCAG 2.1 AA audit via headless Chrome; reports errors, warnings, and notices per URL	`npm i -g pa11y-ci`
Lighthouse	Google's scored accessibility audit (0–100); covers contrast, ARIA, keyboard nav, structure	`npm i -g lighthouse`
axe-cli	Deque's axe-core engine via CLI; complements Lighthouse on ARIA semantics	`npm i -g axe-cli`
webhint	HTTP headers, security, performance, and accessibility hints	`npm i -g hint`
DuckDB	Portable single-file analytical SQL database; stores all audit results	`brew install duckdb`
SchemaCrawler	Reads database metadata and generates ER diagrams (used for the schema diagram in this report)	`brew install schemacrawler`
Graphviz (`dot`)	Renders SchemaCrawler's DOT output to PNG/PDF	`apt install graphviz`

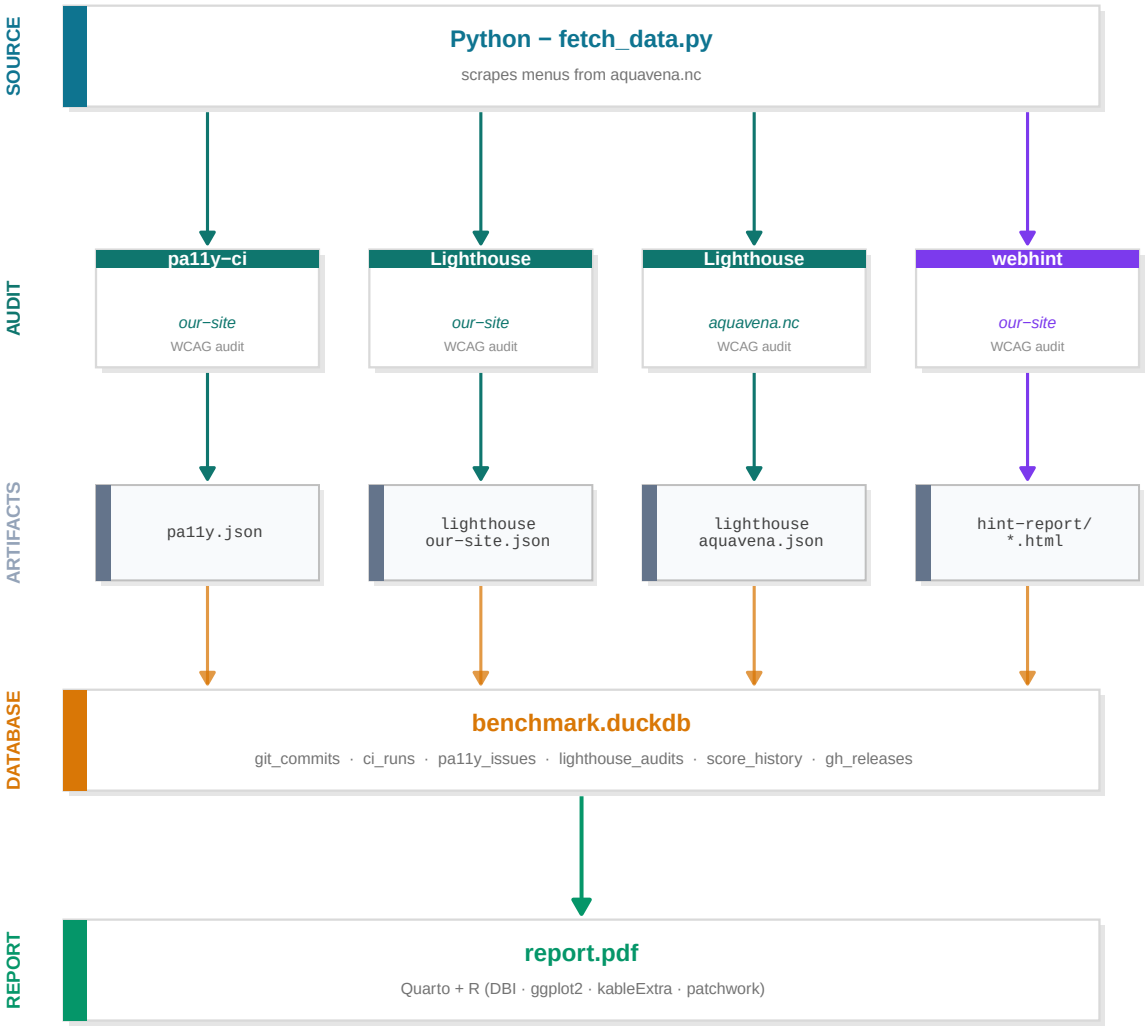
Tool	Role	Install
Quarto	Scientific publishing system; executes R code and renders this document to PDF via XeLaTeX	quarto.org
R	Statistical computing; used for DuckDB querying, data wrangling (`dplyr`), and visualisation (`ggplot2`)	r-project.org
go-task	Task runner (Taskfile.yml); orchestrates the full pipeline with a single command	`brew install go-task`

All Node.js tools require **Node 18+** and are pinned via `package.json` in the `site/` directory. R package dependencies are installed via `task setup:r-deps`.

6 Methodology

6.1 Pipeline overview

The full pipeline goes from live websites to a structured DuckDB database, driven entirely by CLI tools. Each step produces a JSON artefact that feeds the next.



6.2 Release discipline

Each accessibility fix is expressed as a **conventional commit** — a structured commit message whose prefix encodes the nature of the change:

Prefix	Meaning	Version bump
`fix:`	Bug fix or WCAG violation corrected	Patch (`1.0.0 → 1.0.1`)
`feat:`	New accessibility feature added	Minor (`1.0.0 → 1.1.0`)
`perf:`	Performance improvement	Patch

Prefix	Meaning	Version bump
`BREAKING CHANGE:`	Structural redesign	Major (`1.0.0 → 2.0.0`)

On every merge to `main`, a GitHub Actions workflow parses the commit history, determines the appropriate semantic version bump, tags the repository (`v1.2.3`), and publishes a **GitHub Release** with an auto-generated changelog listing every fix by its commit message.

This discipline has a direct consequence for the improvement loop: every version tag is a checkpoint. The `git_sha` column in `ci_runs` ties each audit score to an exact commit, and each release to a before/after score delta:

```sql
-- Score delta between two consecutive releases
SELECT curr.git_sha, curr.lh_score - prev.lh_score AS delta
FROM ci_runs curr
JOIN ci_runs prev ON curr.rowid = prev.rowid + 1
WHERE curr.triggered_by = 'ci';
```

The release is therefore the *unit of measurement* in the optimisation loop — not a deployment formality, but a versioned evidence of improvement.

6.3 Tool coverage

No single tool covers the full accessibility surface. The chart below maps each tool against the dimensions it measures, showing why the combination is necessary.

6.4 Tools

6.5 Standard

All automated checks use **WCAG 2.1 Level AA** with both axe-core and htmlcs runners to maximise coverage.

7 Results — Our Site

7.1 pa11y-ci — WCAG 2.1 AA

URL	Errors	Warnings
http://localhost:4321/about/	0	0
http://localhost:4321/#aqua-m%C3%A9diterran%C3%A9en	0	0

Total: 0 error(s)

7.2 Lighthouse Accessibility

What is Lighthouse? Lighthouse is **Google's open-source auditing tool**, built into the Chrome browser. For accessibility, it runs **over 90 automated checks** against the WCAG 2.1 standard — colour contrast, alt-text, ARIA roles, keyboard focus, form labels, heading hierarchy, and more — and combines them into a single **score from 0 to 100**. The higher the score, the fewer accessibility barriers the page presents to users with disabilities.

Why it matters here. Lighthouse is the **de facto industry benchmark** for web accessibility. A 100/100 score does *not* mean perfect accessibility — manual testing with real assistive technologies is still required — but it does mean that **no automatically detectable WCAG 2.1 AA violation remains** on the page. It is the lowest bar a serious accessibility-focused site should clear, and the first thing any external auditor will run.

Side-by-side comparison. The two gauges below show the **same Lighthouse audit** run on **the same page** (the *Aqua-Méditerranéen* weekly menu) on both sites — the official **aquavena.nc** (the only option before this project) and our rebuilt **adriens.github.io/aquavena**. The delta in the centre is the accessibility headroom recovered by the rebuild.

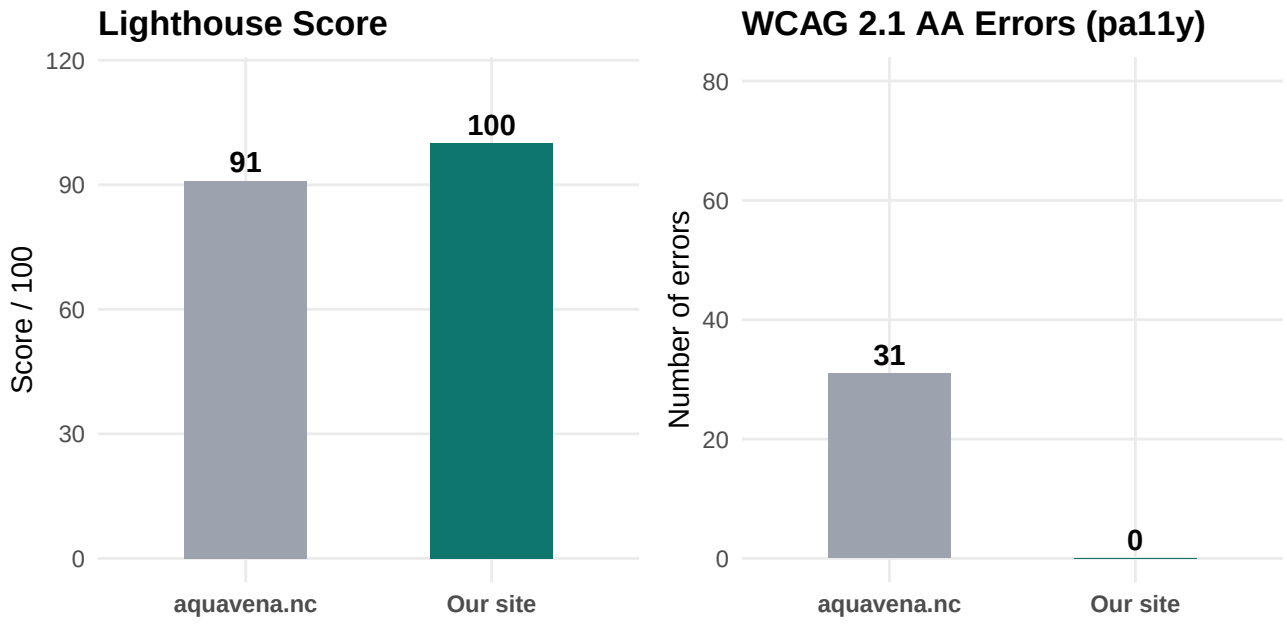
Metric	Value
Score	100/100
Passed audits	192
Failed audits	0
Not applicable	336

8 🌐 Results — aquavena.nc

Tool	aquavena.nc	Our site
pa11y-ci (WCAG 2.1 AA)	31 errors	0 errors
Lighthouse Accessibility	91/100	100/100
webhint — errors	10	10*
webhint — warnings	63	62
webhint — hints	120	120

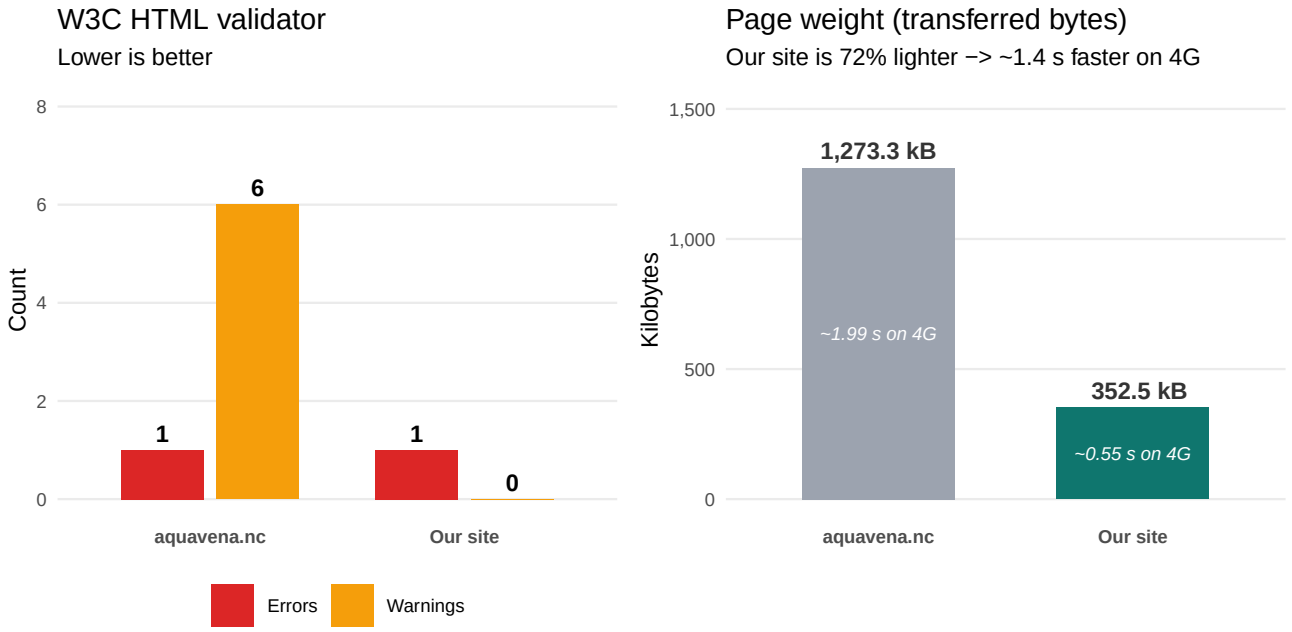
** The 10 webhint errors on our site are GitHub Pages infrastructure issues (HSTS, SRI, `X-Content-Type-Options`) beyond our control — the exact same errors appear on aquavena.nc, as they relate to server configuration, not HTML content.*

9 ⚖️ Comparison



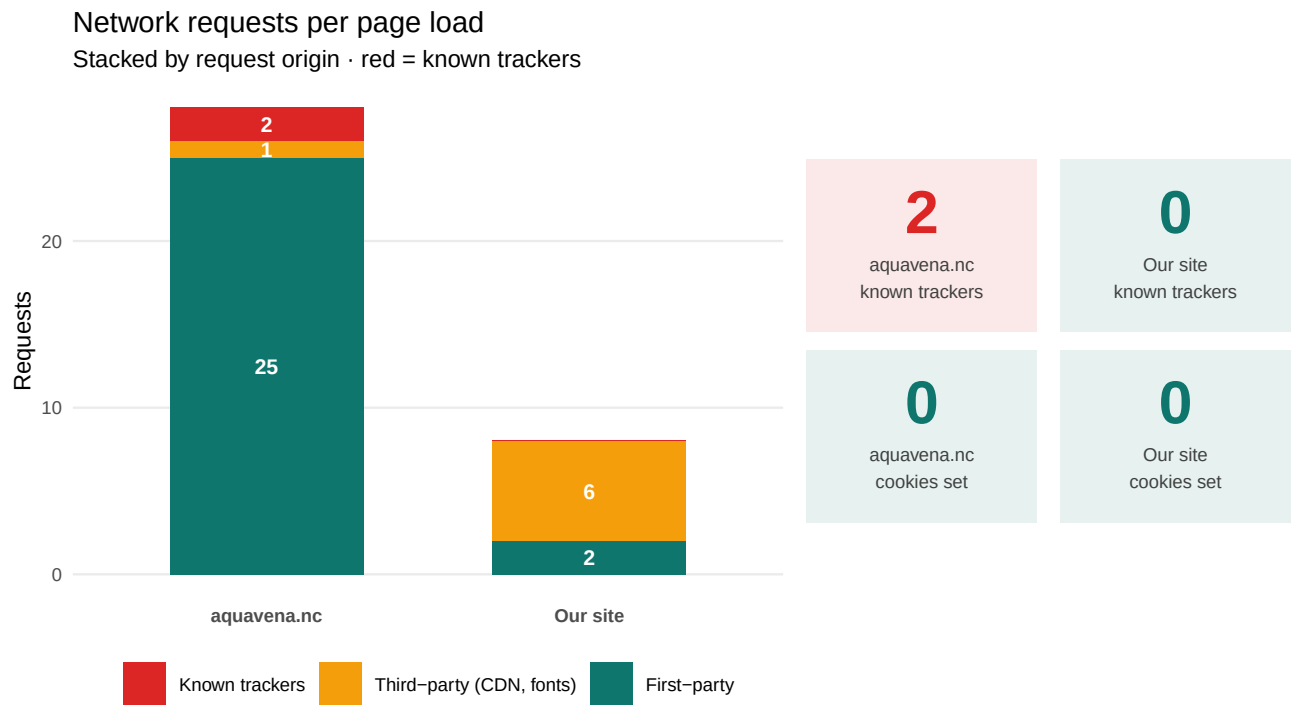
9.1 HTML quality & page weight

Beyond accessibility, two further metrics matter for any visitor — especially those on **mobile / 4G**: HTML standards-compliance (per the W3C Nu HTML validator) and the **total transferred bytes** to load the page. Both are stored in the `html_quality` table and read from the database at render time.



9.2 Privacy & user tracking

A site that respects its visitors **does not track them**. The audit below uses headless Chrome (via puppeteer) to load each page and capture **every network request** issued, **every cookie set**, and matches the third-party hosts against a curated list of known analytics / advertising trackers (Google Analytics, Tag Manager, Facebook Pixel, Hotjar, Mixpanel, Segment, etc.). Results are persisted to the `privacy_audit` table.



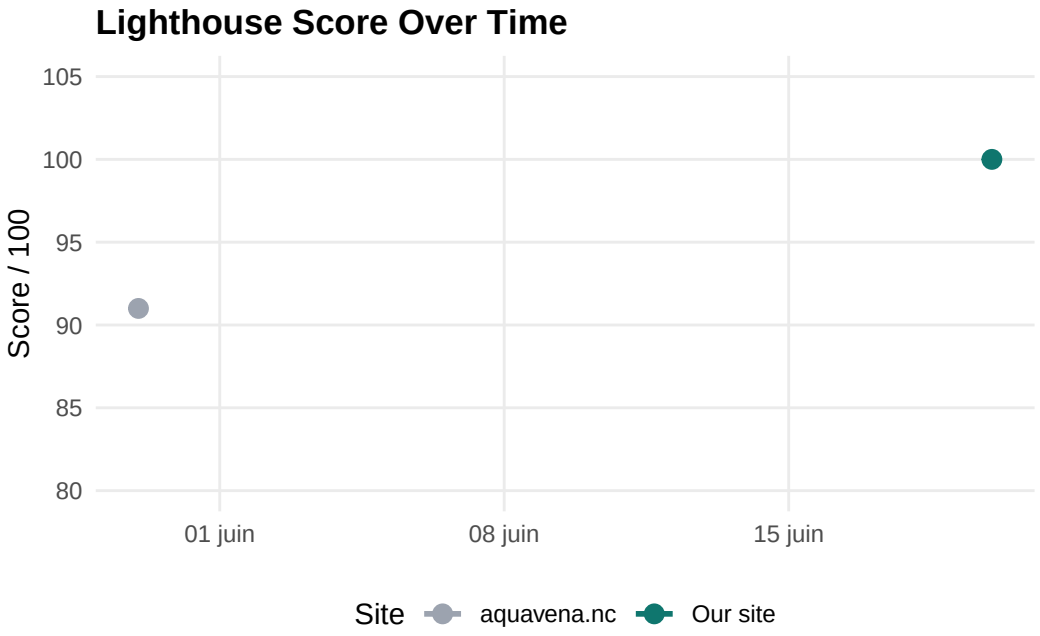
Trackers identified on `aquavena.nc` (queried live from `privacy_audit.known_trackers`):

Site	Tracker host
aquavena.nc	www.google-analytics.com
aquavena.nc	www.googletagmanager.com

10 Accessibility Features

Feature	Our site	aquavena.nc
WCAG 2.1 AA — 0 errors	Yes	No
Lighthouse 100/100	Yes	No
Atkinson Hyperlegible font	Yes	No
Adjustable text size (A+ / A−)	Yes	No
Dark mode	Yes	No
Text-to-speech (Web Speech API)	Yes	No
PWA — installable, works offline	Yes	No
RSS feeds per diet	Yes	No
iCal calendars per diet	Yes	No
Claude skill (MCP)	Yes	No
WebMCP — 5 browser-native AI tools	Yes	No

11 Lighthouse History



12 Score by Version

Each semantic release tag (`vX.Y.Z`) represents a checkpoint in the optimisation loop. The chart below plots the Lighthouse accessibility score of our site at each release.

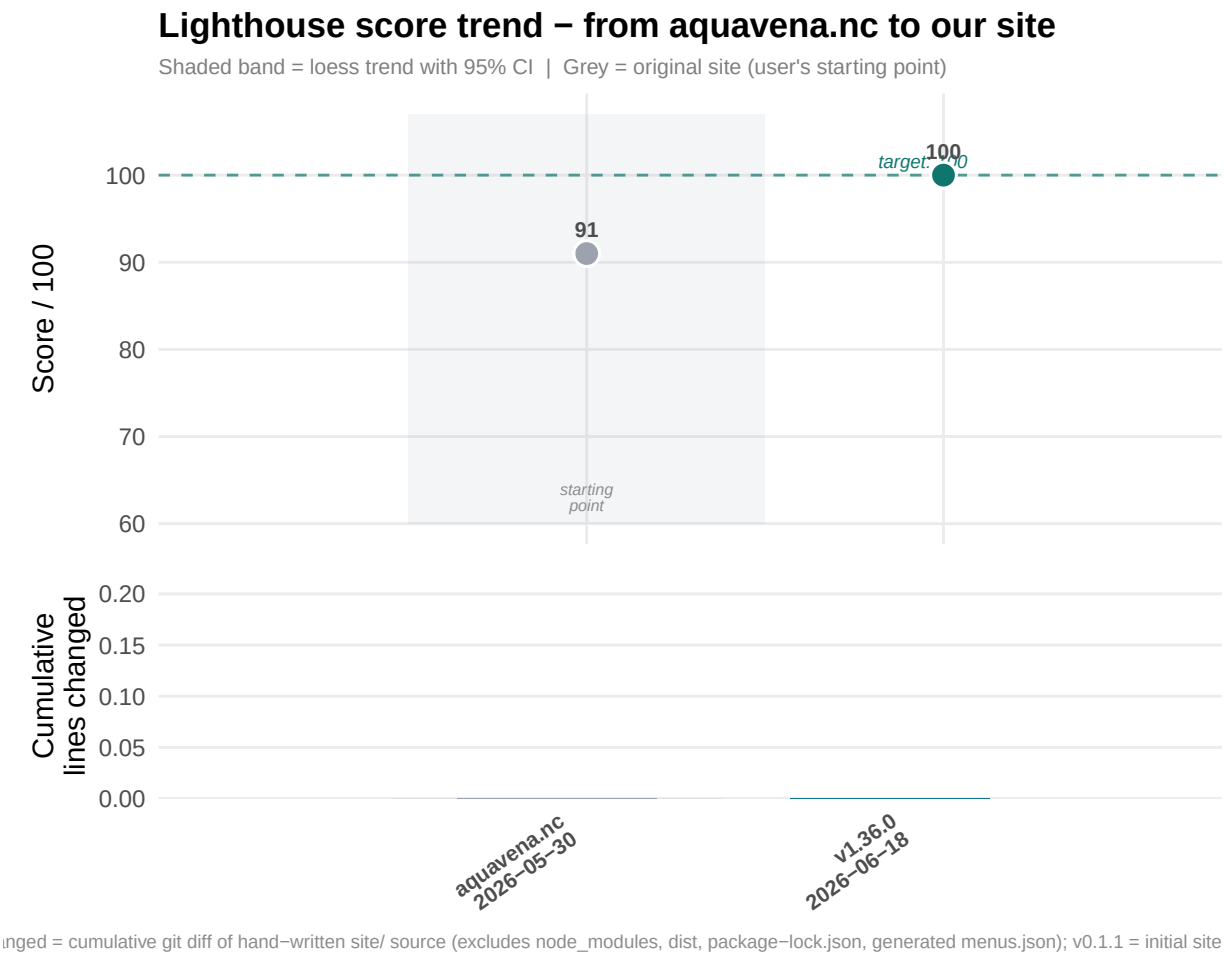
> **Methodology note — historical replay**
>
> The values shown here come from `benchmark/scripts/replay_audits.sh`, a script that checks out every tagged release into a temporary git worktree, builds the site with current dependencies, and runs Lighthouse and pa11y on the same menu page. **Result: every tagged release scores 100/100 Lighthouse and 0 pa11y errors when replayed today.** The accessibility issues documented in the original commit history were tied to the specific upstream dependency versions present at the time of each deploy; the codebase itself, when rebuilt with current Astro and Tailwind, is consistently clean.
> **Takeaway for the PMI committee:** the **architectural choices** (semantic HTML, accessible defaults, contrast-aware palette, DejaVu fonts, no JavaScript-only interactions) are what carry the long-term accessibility budget — they survive years of upstream dependency churn.

How this works. The `ci_runs` table stores `git_sha` for every pipeline execution. Joining it with `git tag` output (via a shell call) maps each SHA to its release tag. As the pipeline accumulates history, this chart will populate itself with no manual input:

```r
# Future query — once ci_runs has enough history:
tags_raw <- system("git tag --sort=version:refname", intern = TRUE)
shas_raw <- sapply(tags_raw, function(t)
  system(paste("git rev-list -n 1", t), intern = TRUE))
tag_map  <- data.frame(tag = tags_raw, git_sha = substr(shas_raw, 1, 7))

# Then join with ci_runs:
# SELECT c.lh_score, t.tag FROM ci_runs c JOIN tag_map t ON c.git_sha = t.git_sha
```

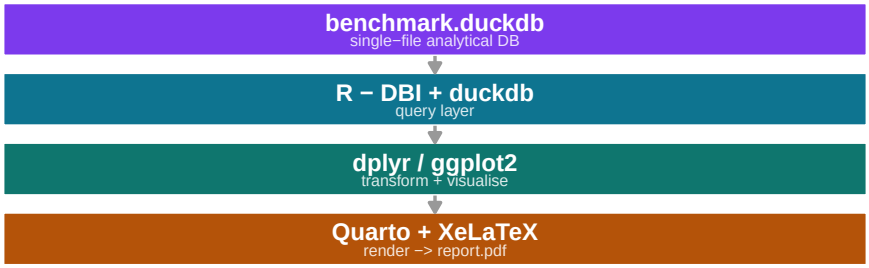
For now, a temporary array seeds the chart with the known score at each tag:



13 Database

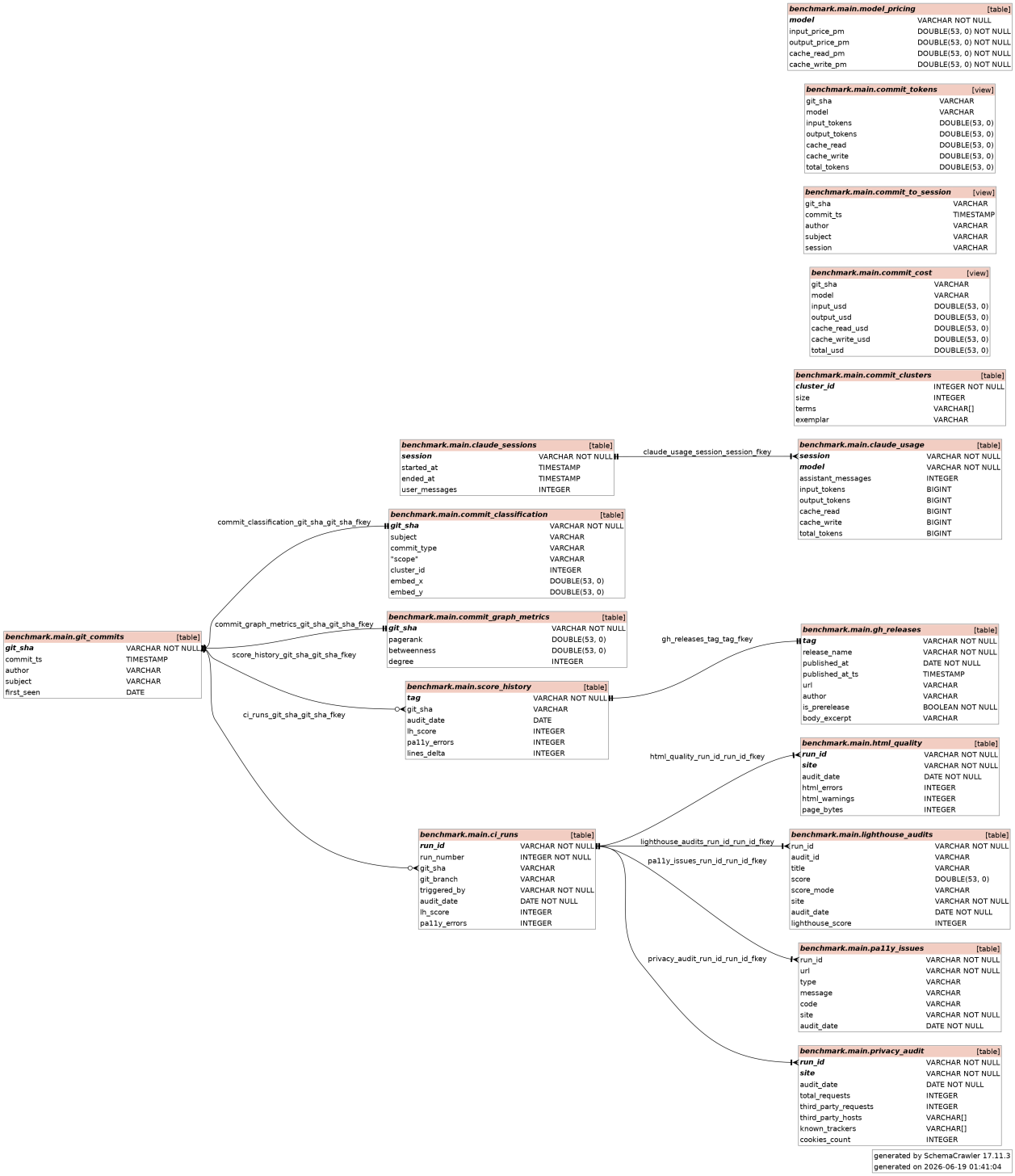
All audit results are persisted in **benchmark/data/benchmark.duckdb**, a portable single-file analytical database. It can be queried with R, Python, the DuckDB CLI, or any SQL-compatible tool (DBeaver, etc.).

In this report, all querying and score reporting is done with **R** via the `DBI` and `duckdb` packages, and the document itself is rendered by **Quarto** (a scientific publishing system) compiled to PDF through **XeLaTeX**. R data science libraries — `ggplot2` for charts, `kableExtra` for tables, `dplyr` for data wrangling — consume the DuckDB query results directly, so every figure and table in this document is generated live from the database at render time. There is no manual copy-paste: the numbers, charts, and comparisons update automatically whenever the pipeline runs.



13.1 Schema

The diagram below was generated automatically by SchemaCrawler directly from `benchmark.duckdb` and rendered via Graphviz (`dot`).



Full column-level specifications for all tables are in Appendix A.

13.2 Example queries

```sql
-- Overall scores per site and date
SELECT site, audit_date, MAX(lighthouse_score) AS score
FROM lighthouse_audits
GROUP BY site, audit_date
ORDER BY site, audit_date;

-- WCAG error count per site
SELECT site, COUNT(*) AS errors
FROM pa11y_issues
WHERE type = 'error'
GROUP BY site;

-- Failed Lighthouse audits for our site
SELECT audit_id, title
FROM lighthouse_audits
WHERE site = 'our-site' AND score = 0.0;

-- Score history across CI runs (regression tracking)
SELECT run_id, git_sha, git_branch, triggered_by,
       audit_date, lh_score, pa11y_errors
FROM ci_runs
ORDER BY audit_date DESC, run_id DESC;

-- Detect regressions: runs where score dropped vs previous
SELECT curr.run_id, curr.audit_date,
       curr.lh_score AS score_now,
       prev.lh_score AS score_prev,
       curr.lh_score - prev.lh_score AS delta
FROM ci_runs curr
JOIN ci_runs prev ON curr.rowid = prev.rowid + 1
WHERE curr.lh_score < prev.lh_score;
```

14 Glossary

Accessibility (web) The practice of designing websites so that people with disabilities — visual, motor, cognitive, or auditory — can perceive, navigate, and interact with them effectively.

axe-core An open-source accessibility rules engine developed by Deque Systems, embedded in Lighthouse, pa11y-ci, and the axe-cli tool. Covers WCAG 2.0/2.1 A and AA rules.

Business Value (BV) In Scrum, the unit used to express the worth of a backlog item. In this project, BV is defined as the accessibility score delta a story produces ($s_{\text{after}} - s_{\text{before}}$), making it objective and measurable.

Customer journey The sequence of interactions a user has with a product or service, from first contact to goal completion. In this project, the customer journey starts at aquavena.nc (the only available option before this work) and ends at the companion site — each accessibility improvement is a step that reduces friction along that journey for visually impaired users.

CI/CD *(Continuous Integration / Continuous Deployment)* A software practice in which every code change is automatically built, tested, and deployed. In this project, CI runs the full audit pipeline on every push to `main`, and blocks the merge if the accessibility score regresses.

Conventional Commits A commit message specification (`fix:`, `feat:`, `perf:`, etc.) that enables automated changelog generation and semantic versioning.

DuckDB An in-process analytical SQL database engine. All audit results in this project are stored in a single portable `.duckdb` file that can be queried with R, Python, or any SQL client.

Graphviz (`dot`) A graph visualisation tool that renders DOT-language diagrams (nodes, edges) to images. Used here to render the SchemaCrawler ER diagram to PNG.

Lighthouse An open-source automated auditing tool built by Google. Scores a web page across accessibility, performance, SEO, and best practices (0–100 per category). Only the accessibility category is used in this project.

Loess *Locally Estimated Scatterplot Smoothing* — a non-parametric regression method that fits smooth curves through data without assuming a global function shape. Used in the score trend chart to show the improvement trajectory.

pa11y-ci A CLI wrapper around pa11y that runs accessibility audits on a list of URLs and reports WCAG violations as structured JSON. Supports multiple rule engines (htmlcs, axe-core).

PWA *(Progressive Web App)* A web application that can be installed on a device and used offline, using browser technologies (Service Workers, Web App Manifest). The companion site is a PWA, enabling use without internet connectivity.

Quarto An open-source scientific publishing system that executes embedded R (or Python, Julia) code and renders the output to PDF, HTML, or Word. This report is a Quarto document compiled via XeLaTeX.

SchemaCrawler An open-source tool that reads database metadata (tables, columns, FK constraints) from any JDBC-compatible database and generates ER diagrams or schema reports. Used here to produce the DuckDB schema diagram.

Semantic versioning *(SemVer)* A versioning scheme (`MAJOR.MINOR.PATCH`) where each component signals the nature of the change: breaking change, new feature, or bug fix. Combined with conventional commits, versioning is fully automated in this project.

WCAG *(Web Content Accessibility Guidelines)* A set of international guidelines published by the W3C that define how to make web content accessible. WCAG 2.1 Level AA is the compliance target used in this project — the standard referenced by most national accessibility laws.

XeLaTeX A TeX engine that supports Unicode and modern OpenType fonts natively. Used by Quarto to compile this report to PDF.

15 WebMCP — The Site as an Agentic Interface

Beyond human-readable accessibility, this project takes a further step: making the site **machine-readable by AI agents** without scraping, screenshots, or prompt engineering on raw HTML. This is achieved through **WebMCP** (Web Model Context Protocol), a W3C Draft standard that lets a website declare callable tools directly in the browser via `navigator.modelContext`.

15.1 Why WebMCP?

Traditional automated accessibility audits (Lighthouse, pa11y-ci) measure *what the HTML says*. They cannot answer questions that require **reasoning over structured content** — *"Is there a fish dish available this week for a low-carb diet?"*, *"What time does the kitchen close on Fridays?"* WebMCP bridges this gap: the site becomes an MCP server hosted inside the browser tab itself, with no backend required.

For a visually impaired user, this means any AI assistant running in the browser can answer natural-language questions about the menus directly — the same way a sighted person skims the page.

15.2 Implementation

Five tools are registered at page load via the **Imperative API** (`navigator.modelContext.registerTool`) each with a JSON Schema:

Tool	Description
`list_regimes`	Lists all available diet plans (slug + name)
`get_week_menu`	Full week menu for a given diet slug
`search_dish`	Keyword search across all diets and all days
`get_today_menus`	All dishes available on the current date
`get_contact`	Address, phone, email, social links, opening hours

The implementation is **invisible to the user** — no UI change, no additional HTTP request, zero impact on Lighthouse or WCAG scores. A `navigator.modelContext?.registerTool()` optional-chain ensures graceful degradation on unsupported browsers.

A **Chrome 149 Origin Trial token** is embedded in the `<head>` of both pages, activating the API for all Chrome 149+ visitors without any flag manipulation. The token is valid until September 2026.

15.3 What It Brings

- **Agents can query the site** without DOM scraping or vision models
- **88% token savings** compared to passing raw HTML to an LLM (structured tool output vs. full-page context)
- **Future-proof**: as browser agents mature (Gemini in Chrome, Edge Copilot, Claude agentic mode), the site is already compliant
- **Transferable pattern**: any static Astro / Next.js / Hugo site can adopt the same approach — no server required

---

16 Future Work — ML & Graph Data Science Roadmap

The current dataset (one site, eight tagged releases, ~80 commits) was intentionally kept small as a working demonstration. The **framework is ML-ready** even though the volume of data on this single site does not yet justify supervised learning. The most impactful next steps are:

16.1 Scale to N sites

Re-run the entire pipeline (`pa11y-ci` + Lighthouse + W3C HTML validator + page weight + privacy audit) on **100+ New Caledonia public-service or business sites** — schools, restaurants, government portals, healthcare providers. Every row of the existing `lighthouse_audits`, `pa11y_issues`, `html_quality`, and `privacy_audit` tables already supports a `site` dimension, so the same FK schema absorbs the new data without modification. A 100× dataset unlocks:

- **Clustering sites by accessibility profile** (k-means / DBSCAN on the multi-axis pass-rate vector from the Strategic Radar) — surfaces typologies: *"trackers-heavy but ARIA-clean"*, *"light page weight but failing contrast"*, etc.
- **Predictive accessibility audits** — given a site's tech stack and page weight, predict the expected Lighthouse score; flag outliers.
- **Pattern mining** — frequent-itemset analysis of which WCAG violations co-occur (`tabindex` + `target-size` failing together is a different problem from `color-contrast` + `link-name`).

16.2 Graph data science on the audit network

The schema described in this report is already a graph (FK edges). With multi-site data, two derived graphs become valuable:

- **Site ↔ violation bipartite graph** — nodes = sites + WCAG rules, edges = "site X violates rule Y". Centrality measures rank the *most-impactful fixes* (rules that, if addressed, would lift the most sites). Tools: NetworkX, igraph, or DuckDB's experimental graph extensions.
- **Audit co-failure graph** — nodes = Lighthouse audits, edges weighted by co-failure count across sites. Community detection (Louvain) surfaces *clusters of related defects* that share a root cause.

16.3 Deepen the commit-semantics work

This report already embeds 83 commits with sentence-transformers and clusters them into 6 groups. At a 10× volume:

- **Topic drift over time** — track how the cluster mix shifts release after release. Late-stage releases dominated by `docs/chore` clusters indicate maturity.
- **Effort prediction** — train a regressor mapping commit-message embedding → token consumption (using the join we already have between `commit_classification` and `commit_cost`). Predicts AI delivery cost before the work starts.

16.4 Time-series of accessibility evolution

Re-run `replay_audits.sh` against the **public Wayback Machine** archive of aquavena.nc (and peers) to recover historical scores even from sites that never instrumented themselves. Combined with the existing `replay_audits.sh`, this would yield a **decade-scale time series** of accessibility evolution on the New Caledonia web — a publishable dataset.

16.5 AI-Agent Universal Accessibility Pipeline

Web accessibility is **not only about low vision**. The WCAG 2.1 standard covers a wide spectrum of impairments — visual, motor, cognitive, auditory — each requiring different design choices. The next frontier goes beyond static rule-based audits: **a fleet of AI agents, each simulating a specific disability profile**, interacts with the site and reports perceived issues with that profile's lived constraints in mind.

Each agent is specialised:

- **Blind agent** — navigates via ARIA semantics and screen-reader output
- **Dyslexic agent** — stresses typography, spacing and heading structure
- **Low-vision agent** — zooms to 200% and tracks focus visibility
- **Colourblind agent** — checks colour-only information channels and contrast
- **Custom agent** — fine-tuned for any other profile (motor, cognitive, elderly)

A critical design decision: agents **do not all live in the browser**. The same tools (`list_regimes`, `get_week_menu`, `search_dish`, `get_today_menus`, `get_contact`) are exposed through **two complementary channels**:

- **WebMCP** (`navigator.modelContext`) — for agents embedded in the browser page itself (Gemini in Chrome, Edge Copilot, browser extensions)
- **MCP server** (Hugging Face Spaces) — for headless interfaces: voice assistants (Alexa, Google Assistant), home robots, **refreshable braille displays** (via a screen reader or an agent that pipes the JSON response straight to the braille bus), accessibility appliances, IoT devices, Claude Desktop, custom CLI agents

This dual exposure is what makes the pipeline **truly universal**: a blind user reading menus on a refreshable braille display, another blind user querying a smart speaker, a low-vision user with a tablet browser, and a motor-impaired user with a robot companion all consume the **same canonical tools** — only the transport and the physical rendering surface differ.

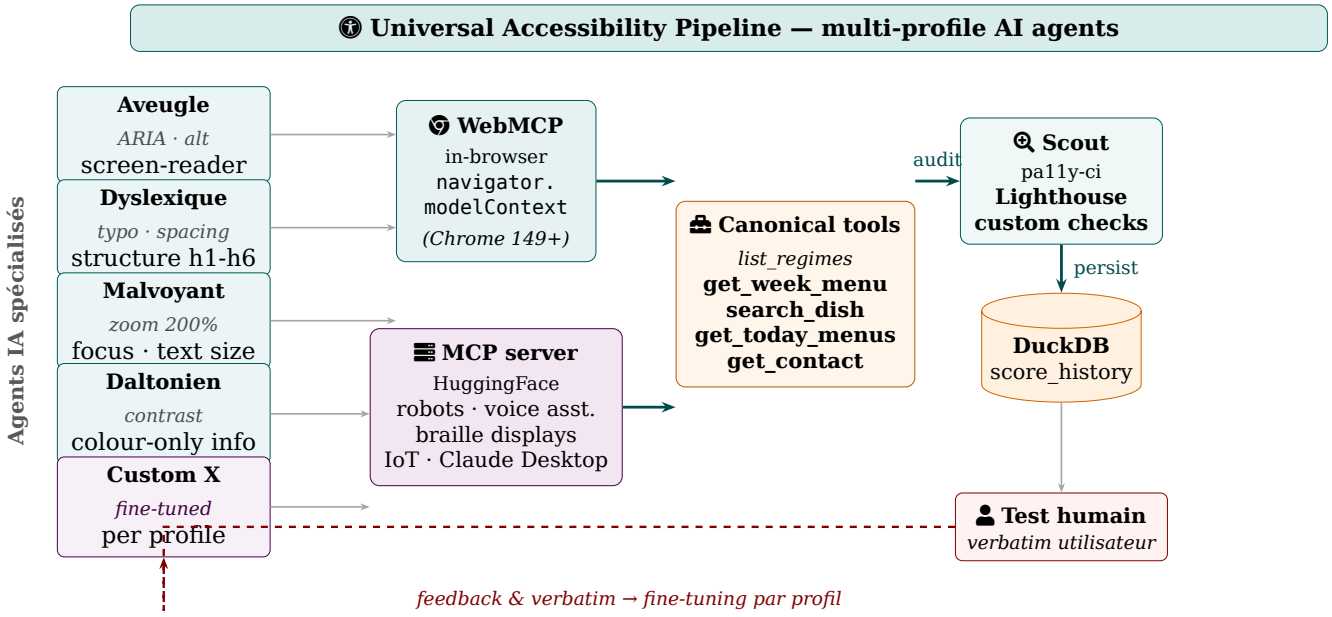


Figure 8: Universal Accessibility Pipeline: each AI agent simulates a specific WCAG profile and consumes the same canonical tools through either WebMCP (in-browser) or the MCP server (robots, voice assistants, braille displays, IoT). Scout audits the responses against multi-criterion rules; human verbatim closes the loop by fine-tuning each agent profile.

The key insight is that the **canonical tools are the contract**, not the transport. The same five tools serve all five profiles across both WebMCP and the MCP server; the difference lies in **how each agent interprets the response** against its disability constraints, and in which physical interface delivers it to the user — a browser, a smart speaker, a robot, or an assistive appliance.

Each agent version is tagged with a dedicated semantic (e.g. `agent-blind-v1.2`, `agent-dyslexic-v0.4`) so that score regressions can be attributed to a specific simulation profile and retested in isolation.

16.6 Honest data requirements

Technique	Minimum useful N	Current dataset
Site clustering	~30 sites	2
Co-failure community detection	~50 sites	2
Embedding-based effort prediction	~500 commits	179
Time-series forecasting	~50 releases	26
Anomaly detection (regressions)	~20 weekly audits	1

The architecture is correct. **Scale is the missing ingredient — and the benchmark pipeline scales linearly with sites added.**

16.7 New perspectives unlocked by the 179-commit dataset

With 179 commits across 6 semantic clusters, 47 release tags, and a now-complete agentic interface layer, the dataset opens directions that were not viable at 83 commits:

Commit-level accessibility prediction (supervised ML) The join `commit_classification ↔ score_history` now has 26 labelled rows (commit embedding → Lighthouse delta). Still small, but the cluster distribution across 179 commits is rich enough to train a lightweight regressor that maps *commit semantic type* to *expected score impact* — flagging "colour/contrast" commits as high-risk before they merge.

Agent profile fine-tuning from human verbatim (NLP) Each RC-stage human verbatim is a natural-language annotation of a disability-specific UX failure. At 10+ verbatim entries, a small fine-tuning dataset emerges: `(verbatim, agent_profile) → WCAG rule`. This closes the loop: humans annotate once, agents learn the pattern, future builds catch the same failure automatically.

Cross-site transfer learning The 6 semantic clusters (accessibility, report, UX, deps, merge, custom) are domain-agnostic — they will appear on any web project. A model trained on Aquavena's `(cluster, lines_delta) → score` mapping can be fine-tuned for any other New Caledonia public-service site with as few as 5 audited releases, dramatically reducing cold-start cost.

Cluster drift as a project maturity signal Cluster 1 (report/WebMCP, 32 commits) emerged only in the second phase of the project. Tracking the cluster-mix ratio release-over-release gives a *maturity index*: early-stage projects are dominated by accessibility and UX clusters; mature projects shift toward docs, deps, and tooling. This is a zero-label signal — no human annotation required.

Graph neural network on the FK schema `gh_releases → score_history → git_commits ← ci_runs ← pa11y_issues` is a heterogeneous knowledge graph. With 47 releases and 179 commits, a small GNN can learn node embeddings that predict *which WCAG rule is most likely to regress on the next release* given the current commit cluster and the co-failure graph from `pa11y_issues`.

Multi-agent orchestration benchmark The 5 specialised agents + Custom X composition define a natural **multi-agent benchmark**: given a new site, which agent detects the most regressions first? Tracking *detection latency per agent* and *recall per WCAG category* across releases produces agent-level DORA metrics — a novel contribution to the accessibility-engineering literature.

17 📖 Further Reading

The methodology behind this report — *identifying a real user need, mapping it to computable signals, and iterating against a measurable target* — is one applied instantiation of broader design-thinking principles. The reference below is a foundational reading for anyone wanting to combine **human-centered design** with engineering discipline:

- Sinek, Simon. *Start With Why: How Great Leaders Inspire Everyone to Take Action*. Portfolio/Penguin, 2009. ISBN 978-1-59184-280-4. URL: goodreads.com/book/show/7108725. — *The WHY behind Aquavena is simple and personal: a family member with low vision struggling with an inaccessible site. Sinek's central argument — that purpose-driven action outlasts motivation-driven action — is what turned a weekend observation into a published pipeline. The report you are reading is the HOW and WHAT; this book is the reason the WHY was worth pursuing.*
- Brown, Tim. *Change by Design: How Design Thinking Transforms Organizations and Inspires Innovation*. Harper Business, 2009. ISBN 978-0-06-176608-4. URL: goodreads.com/book/show/6671664. — *A practitioner-level treatment of design thinking, including the journey from observation of a real user problem to a tested, deployed solution. Reads directly onto the Aquavena story: started at a kitchen table with one visually impaired user, ended with an audited, measurable, transferable pipeline.*

18 🏁 Conclusion

The site achieves a **perfect** accessibility score: **0 WCAG error(s)** and **Lighthouse 100/100**.

Compared to aquavena.nc (91/100, 31 WCAG errors), this site delivers significantly better accessibility, and adds features absent from the source site: text-to-speech, dark mode, adjustable text size, PWA support, RSS feeds, and iCal calendars — all designed to give visually impaired users full autonomy.

But this project has grown beyond its original scope. What started as a weekend hack to help one visually impaired person read her menus has become a **proof of concept for universal agentic accessibility**:

- The same five canonical tools (`list_regimes`, `get_week_menu`, `search_dish`, `get_today_menus`, `get_contact`) are now queryable by **any agent, on any device, through any transport** — a browser extension via WebMCP, a voice assistant via the MCP server, a refreshable braille display, or a companion robot.
- **Reachy Mini** (HuggingFace / Pollen Robotics) has already called the Aquavena MCP tools in a live demonstration, reading the menus aloud through its speaker. A physical robot, a Python MCP client, the same five tools — the architecture holds.

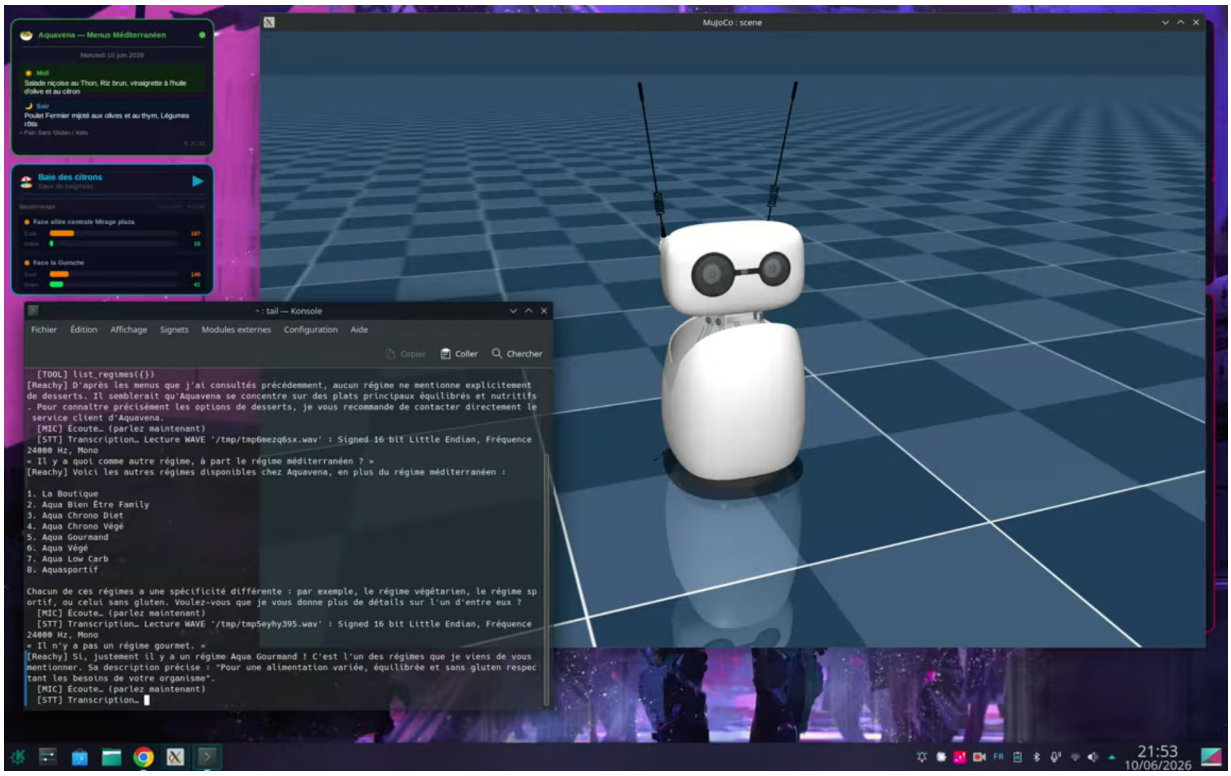


Figure 9: Reachy Mini (HuggingFace / Pollen Robotics) calling the Aquavena MCP server in a live demo. Left: the Claude terminal shows `list_regimes()` being called and the regime list returned. Right: Reachy Mini in its 3D environment, about to read the menus aloud. The same five canonical tools — no code change — serve both the browser and the robot.

- **AI agents replace human testers in CI** — the Universal Accessibility Pipeline runs specialised agents (blind, dyslexic, low-vision, colourblind, composed) on every commit, producing WCAG KPIs in seconds without scheduling a single human. Human verbatim is reserved for release-candidate validation, not the daily build loop.

The lesson is architectural: **expose your content as structured tools, not as HTML to be scraped**. The five tools we defined are the contract. Every transport — browser, HTTP, robot bus, braille display — consumes that contract unchanged. This is what makes the pipeline transferable: any service that publishes its content as MCP tools inherits the same agentic accessibility stack for free.

Report generated on juin 20, 2026 · Quarto 1.9.38 + XeLaTeX · R 4.5.3

A ⊞ Appendix — Full Table Descriptions

This appendix provides the complete column-level specification for every table in `benchmark.duckdb`, including types, nullability, and the role of each field. The full FK graph is:

gh_releases → score_history → git_commits ← ci_runs ← pa11y_issues and ci_runs ← lighthouse_audits.

A.1 git_commits

Root anchor table — one row per commit in the repository, populated from `git log --all`. Primary key: `git_sha`. Both `ci_runs` and `score_history` reference this table, making it the single join point that connects release scores to pipeline execution history. Also feeds the activity heatmap.

Column	Type	Nullable	Description
`git_sha`	VARCHAR	**PK**	Short commit SHA (7 chars); unique per commit
`commit_ts`	TIMESTAMP	yes	Author timestamp (UTC); used for the activity heatmap and DORA lead-time calculations
`author`	VARCHAR	yes	Commit author name (from `git log %aN`)
`subject`	VARCHAR	yes	Commit subject line (from `git log %s`)
`first_seen`	DATE	yes	Date this commit was authored (redundant with `commit_ts`, kept for fast date filtering)

A.2 ci_runs

One row per pipeline execution (local or CI). Primary key: `run_id`. Foreign key: `git_sha` → `git_commits.git_sha`.

Column	Type	Nullable	Description
`run_id`	VARCHAR	**PK**	GitHub Actions run ID, or `local-YYYYMMDDHHMMSS` for local runs
`run_number`	INTEGER	no	Sequential CI counter (`GITHUB_RUN_NUMBER`); 0 for local runs

Column	Type	Nullable	Description
`git_sha`	VARCHAR	**FK**	References `git_commits.git_sha` — commit present in the working tree at run time
`git_branch`	VARCHAR	yes	Branch name (`main`, PR branch, etc.)
`triggered_by`	VARCHAR	no	`ci` when `CI=true` is set in environment, otherwise `local` or `manual`
`audit_date`	DATE	no	Calendar date the pipeline ran
`lh_score`	INTEGER	yes	Lighthouse accessibility score for our site (0–100)
`pa11y_errors`	INTEGER	yes	Count of WCAG 2.1 AA errors on our site at this run

A.3 `pa11y_issues`

One row per pa11y issue (error, warning, or notice) per URL, per run. Foreign key: `run_id` → `ci_runs.run_id`.

Column	Type	Nullable	Description
`run_id`	VARCHAR	**FK**	References `ci_runs.run_id`
`url`	VARCHAR	no	Page URL that was audited
`type`	VARCHAR	yes	Issue severity: `error`, `warning`, or `notice`
`message`	VARCHAR	yes	Human-readable description of the violation
`code`	VARCHAR	yes	WCAG rule code (e.g. `WCAG2AA.Principle1.Guide`
`site`	VARCHAR	no	`our-site` or `aquavena.nc`
`audit_date`	DATE	no	Date the audit was run (redundant with `ci_runs`, kept for direct queries)

A.4 lighthouse_audits

One row per Lighthouse audit check per run. Each Lighthouse report contains ~50–80 individual checks; this table stores all of them. Foreign key: `run_id → ci_runs.run_id`.

Column	Type	Nullable	Description
`run_id`	VARCHAR	**FK**	References `ci_runs.run_id`
`audit_id`	VARCHAR	yes	Lighthouse audit identifier (e.g. `color-contrast`, `aria-required-attr`)
`title`	VARCHAR	yes	Human-readable audit name
`score`	DOUBLE	yes	Audit result: `1.0` = pass, `0.0` = fail, `NULL` = not applicable
`score_mode`	VARCHAR	yes	`binary`, `notApplicable`, `informative`, etc.
`site`	VARCHAR	no	`our-site` or `aquavena.nc`
`audit_date`	DATE	no	Date the audit was run
`lighthouse_score`	INTEGER	yes	Overall page accessibility score for this run (0–100)

A.5 score_history

One row per semantic release tag. Stores the Lighthouse score, WCAG error count, incremental lines-of-code changed, and the git commit SHA at each version checkpoint. Primary key: `tag`. Populated from the ingest chunk; updated by `task score-history` once `ci_runs` has sufficient history. The `git_sha` column was added retrospectively after running `git rev-list -n 1 <tag>` for every tag, so that each score row is unambiguously traceable to a specific commit in the repository.

Column	Type	Nullable	Description
`tag`	VARCHAR	**PK**	Semantic version tag (`v0.1.1`, `v1.6.0`, ...) or `aquavena.nc` for the baseline

Column	Type	Nullable	Description
`git_sha`	VARCHAR	**FK**	References `git_commits.git_sha`; `NULL` for the aquavena.nc baseline (external site, no repo commit)
`audit_date`	DATE	yes	Date the score was recorded
`lh_score`	INTEGER	yes	Lighthouse accessibility score at this tag (0–100)
`pa11y_errors`	INTEGER	yes	Number of WCAG 2.1 AA errors at this tag
`lines_delta`	INTEGER	yes	Lines changed (insertions + deletions) since the previous tag (`git diff --stat`)

A.6 `gh_releases`

One row per GitHub release. Stores the public release metadata (name, date, URL, author, pre-release flag, and a short excerpt of the release notes) for every semantic version published on GitHub. Primary key: `tag`. Foreign key `tag → score_history.tag` links each public delivery event to its measured accessibility score, making it possible to join release notes with score progression in a single query.

Column	Type	Nullable	Description
`tag`	VARCHAR	**PK / FK**	Semantic version tag — references `score_history.tag`
`release_name`	VARCHAR	no	Full GitHub release title
`published_at`	DATE	no	Date the release was published on GitHub
`published_at_ts`	TIMESTAMP	yes	Precise publication timestamp (UTC) — used to compute DORA lead time
`url`	VARCHAR	yes	Direct URL to the GitHub release page

Column	Type	Nullable	Description
`author`	VARCHAR	yes	GitHub login of the release author
`is_prerelease`	BOOLEAN	no	`true` for pre-releases / release candidates
`body_excerpt`	VARCHAR	yes	First ~200 characters of the release notes body

A.7 `html_quality`

One row per (run, site) — HTML quality metrics from the **W3C Nu HTML validator** plus the **total transferred page weight** measured by Lighthouse's `total-byte-weight` audit. Populated from `benchmark/data/html_validation.json` (generated by `audit_html.sh`). Foreign key: `run_id → ci_runs.run_id`. The page-weight metric matters disproportionately on mobile / 4G networks where every kilobyte translates to seconds of waiting on slower connections.

Column	Type	Nullable	Description
`run_id`	VARCHAR	**FK**	References `ci_runs.run_id`
`site`	VARCHAR	**PK part**	`our-site` or `aquavena.nc`
`audit_date`	DATE	no	Date the HTML validation was run
`html_errors`	INTEGER	yes	Number of W3C Nu validator errors on the page
`html_warnings`	INTEGER	yes	Number of W3C Nu validator warnings on the page
`page_bytes`	INTEGER	yes	Total transferred bytes for the page (Lighthouse `total-byte-weight`)

A.8 `privacy_audit`

One row per (run, site) — captures user-tracking and privacy exposure metrics from `audit_privacy.sh` (a headless-Chrome / puppeteer script). For each page load it logs the **total network requests**, separates **first-party from third-party hosts**, matches third-party hosts against a curated list of **known analytics / advertising trackers** (Google Analytics, Tag Manager, Facebook Pixel, Hotjar, Mixpanel, Segment, etc.), and counts the **cookies** set after page load. Foreign key: `run_id → ci_runs.run_id`.

Column	Type	Nullable	Description
`run_id`	VARCHAR	**FK**	References `ci_runs.run_id`
`site`	VARCHAR	**PK part**	`our-site` or `aquavena.nc`
`audit_date`	DATE	no	Date the privacy audit was run
`total_requests`	INTEGER	yes	Total HTTP requests issued during page load
`third_party_requests`	INTEGER	yes	Subset of `total_requests` whose host differs from the page host
`third_party_hosts`	VARCHAR[]	yes	Distinct third-party hostnames contacted (DuckDB LIST)
`known_trackers`	VARCHAR[]	yes	Subset of `third_party_hosts` matching the known-tracker pattern set
`cookies_count`	INTEGER	yes	Number of cookies present after the page settled

A.9 `claude_sessions`

One row per Claude Code session that produced work on this project. Populated from `~/.claude/projects/-home-adriens-Github-aquavena/*.jsonl` by `extract_claude_usage.py`. Used as the parent row for `claude_usage` and as the timestamp window for the `commit_to_session` VIEW.

Column	Type	Nullable	Description
`session`	VARCHAR	**PK**	Claude Code session UUID
`started_at`	TIMESTAMP	yes	First timestamped event in the transcript
`ended_at`	TIMESTAMP	yes	Last timestamped event in the transcript

Column	Type	Nullable	Description
`user_messages`	INTEGER	yes	Count of human-authored messages in the session

A.10 `claude_usage`

One row per (session, model). A session that used both Sonnet and Opus produces two rows. Foreign key: `session` → `claude_sessions.session`.

Column	Type	Nullable	Description
`session`	VARCHAR	**FK**	References `claude_sessions.session`
`model`	VARCHAR	**PK part**	Model identifier (e.g. `claude-sonnet-4-6`, `claude-opus-4-7`)
`assistant_messages`	INTEGER	yes	Count of assistant messages this model produced in the session
`input_tokens`	BIGINT	yes	Total fresh input tokens (cache misses)
`output_tokens`	BIGINT	yes	Total output tokens generated
`cache_read`	BIGINT	yes	Total tokens served from prompt cache
`cache_write`	BIGINT	yes	Total tokens written to prompt cache
`total_tokens`	BIGINT	yes	Sum of the four columns above

A.11 `model_pricing`

Per-million-token list prices for each model. Editable when Anthropic publishes new prices. Joined into the `commit_cost` view to produce USD per release.

Column	Type	Description
`model`	VARCHAR	**PK** — references `claude_usage.model`
`input_price_pm`	DOUBLE	USD per million fresh input tokens
`output_price_pm`	DOUBLE	USD per million output tokens

Column	Type	Description
`cache_read_pm`	DOUBLE	USD per million cache-hit tokens
`cache_write_pm`	DOUBLE	USD per million cache-write tokens

Views derived from these tables (not materialised — computed at query time):

- **`commit_to_session`** — every commit joined to the session whose `[started_at, ended_at]` window contains its `commit_ts`.
- **`commit_tokens`** — session tokens **apportioned** across commits (each commit gets `session_tokens / commits_in_session`), per model.
- **`commit_cost`** — `commit_tokens` × `model_pricing` = USD per (commit, model).